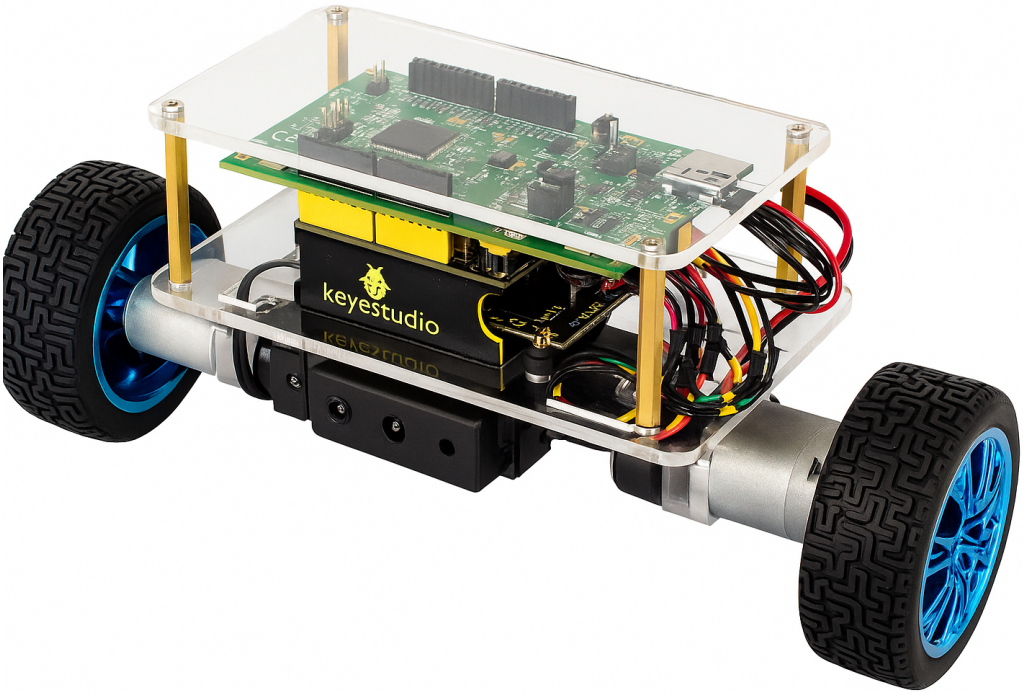
EE/CE 376L/332L

Microcontrollers and Interfacing Lab

Lab Project

Self-Balancing Robot Using ARM Cortex-M4

Version – Tuesday 13th January 2026



Contents

Introduction

1.1 System overview

1.1.1 Task introduction

This lab introduces students to the basics of microcontroller-based system development through hands-on project-based activities. Throughout the semester, students will use the STM32F3 Discovery Board, ARM Cortex-M4 based microcontroller, along with the KeyesStudio Self-Balancing Robot Kit.

Students will begin by learning key development tools: using Git for version control, writing and compiling code with Visual Studio Code, and setting up the environment with STM32CubeCLT. Students will gradually learn about microcontroller architecture, digital and analog interfacing, timers, interrupts, communication protocols such as SPI and I2C, and memory management.

By the end of the course, students will be able to write low-level firmware, connect sensors and actuators, and use control methods such as PID. The course will end with a hands-on project that tests the student's ability to apply what they have learned in a complete microcontroller-based system. The project and lab activities are designed to support students in their capstone projects, where these tools and skills are essential.

Course Structure

The lab is divided into weekly sessions, each focused on a specific concept or peripheral such as GPIO, timers, ADC, and communication protocols. The final weeks are set aside for an open-ended project. A detailed breakdown for each week is given in the next section.

1.1.2 STM32F3 Discovery board

This lab course uses the STM32F3 Discovery development board, which features the STM32F303VCT6 microcontroller from STMicroelectronics' STM32 family, based on the ARM Cortex-M4 core. This board offers a solid platform for developing and testing embedded systems and will be used throughout the course to learn various microcontroller interfacing concepts.

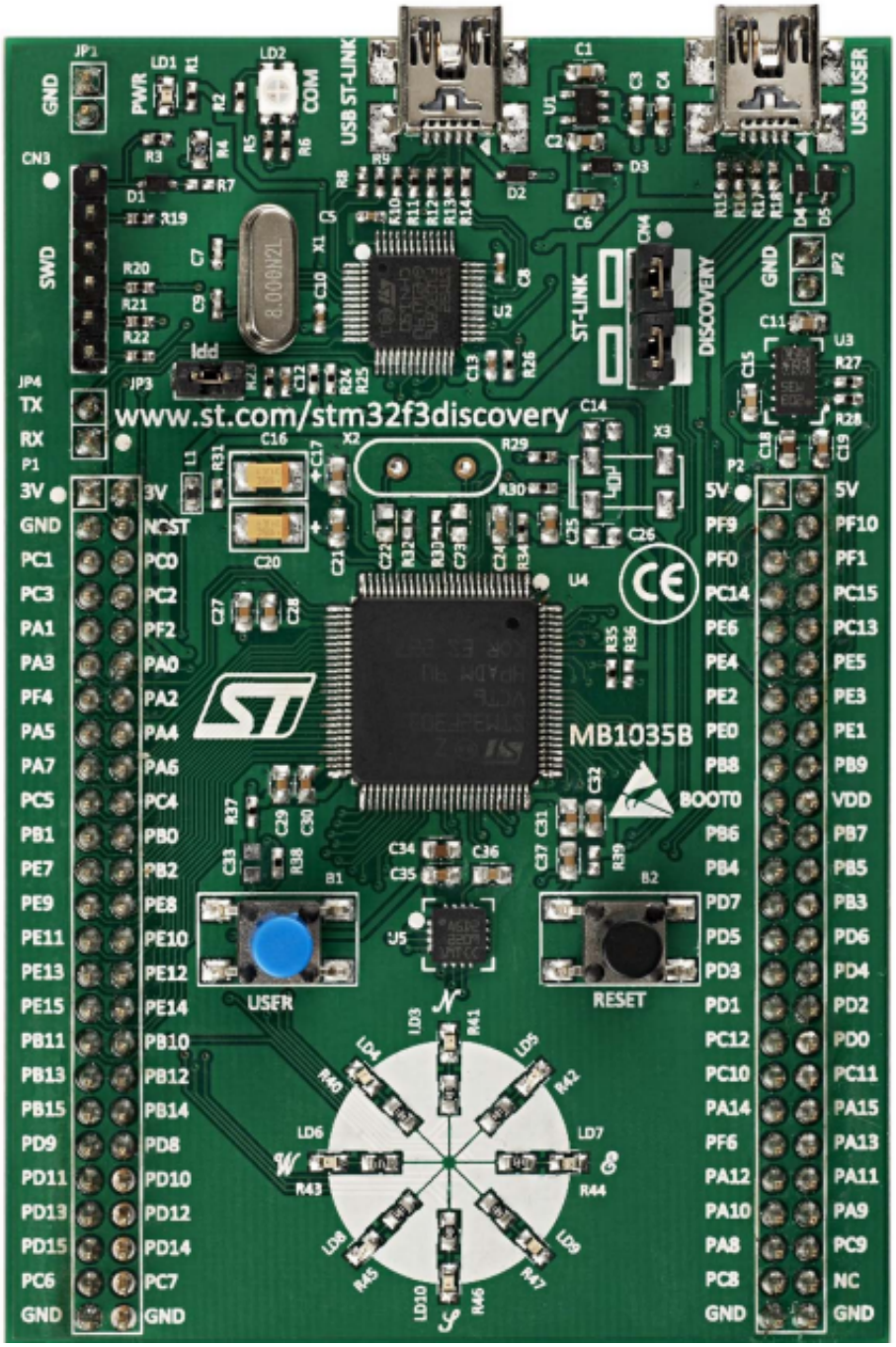


Figure 1.1: STM32F3 Discovery board.

Microcontroller Features

The STM32F303VCT6 has a 32-bit ARM Cortex-M4 core that runs at up to 72 MHz. It includes a hardware floating point unit (FPU) and supports digital signal processing (DSP) instructions. These features help to perform math and signal processing tasks efficiently, which are important for control systems, data collection, communication, and signal conditioning applications.

Key Specifications

Feature	**Details**
Core	ARM® Cortex®-M4 with FPU and DSP support
Clock Frequency	72 MHz
Flash Memory	256 KB (non-volatile program storage)
SRAM	40 KB (volatile data memory)
GPIO	Up to 87 general-purpose input/output pins
Analog Interfaces	4 Analog-to-Digital Converters (ADCs), 2 Digital-to-Analog Converters (DACs)
Timers	Advanced, general-purpose, and basic timers
Communication Interfaces	UART, SPI, I^2C, USB
On-board Debugger	ST-LINK/V2 integrated (via mini-USB)
Operating Voltage Range	2.0 V to 3.6 V (core and I/O logic levels)

Table 1.1: Key specifications of STM32F3 Discovery board.

Power Supply Overview

The STM32F3 Discovery board supports several power supply options, making it flexible for different development and interfacing needs.

Power Supply Sources

The board can be powered through:

- The ST-LINK USB connector
- The USB USER connector
- An external 3V or 5V supply voltage

Power Pins Usage

Pin	Direction	Voltage	Conditions
5V	Input/Output	5V	As output: supplies connected peripherals (max 100 mA). As input: can power the board when USB is not connected.
3V	Input/Output	3V	Same conditions as 5V. Connected peripherals must remain below 100 mA.

Recommended Default Power Setup:

- Use the ST-LINK USB connector to power the board via a PC or laptop.
- Avoid connecting power to 5V or 3V pins unless explicitly required by an experiment and under the supervision of an instructor.

On-Board Peripherals

The Discovery board includes several peripherals that will be utilized in subsequent experiments:

- User LEDs (4)
- User push-button (1)
- MEMS e-compass accelerometer with I2C interface (LSM303DLHC)
- MEMS gyroscope with SPI interface (I3G4250D)
- USB interface (for power, programming, and communication)
- ST-LINK/V2 debugger interface

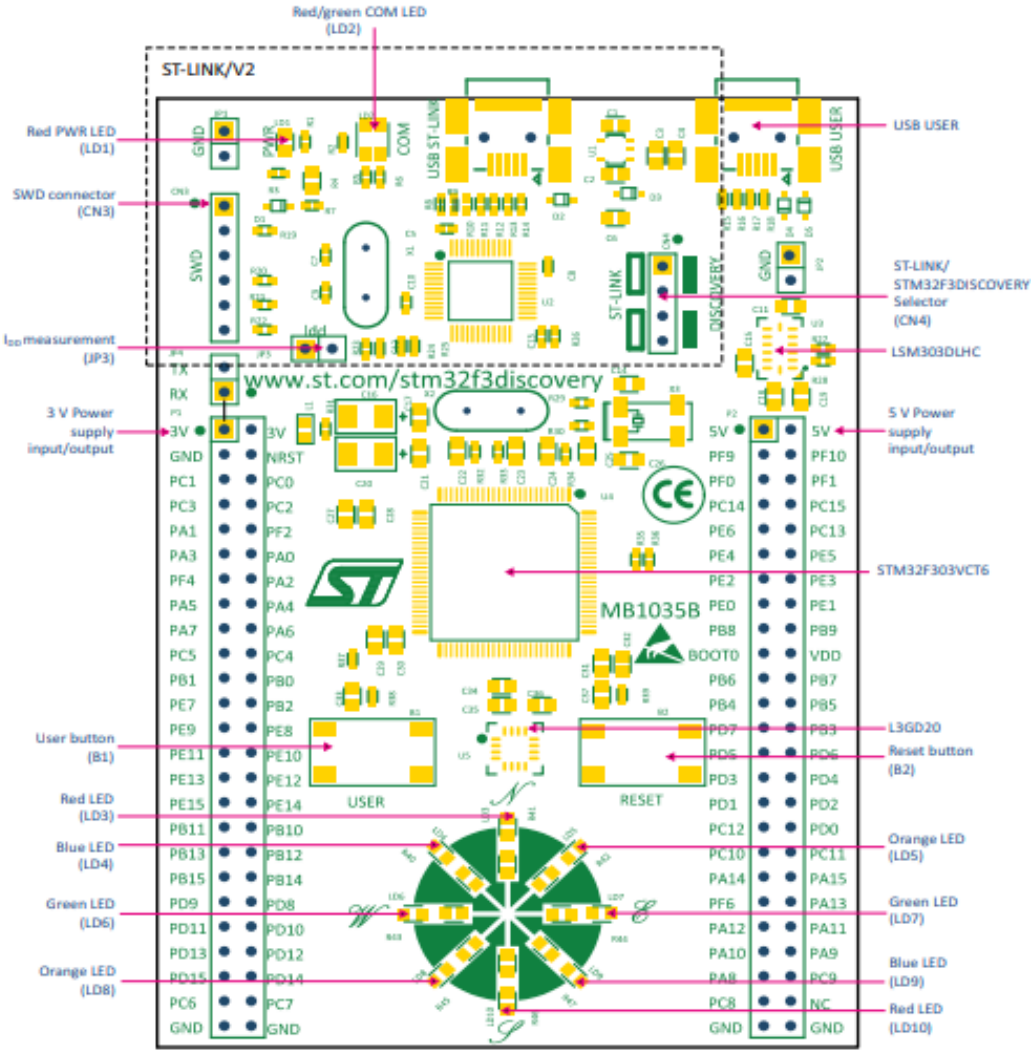


Figure 1.2: Board Layout (Top View)

Board Support Package (BSP)

The Board Support Package (BSP) is a set of software tools from STMicroelectronics that helps developers work directly with the hardware of an STM32 board. In this course, we will use the STM32F3 BSP, which is made for the STM32F3 Discovery Board.

The BSP works with the Hardware Abstraction Layer (HAL) by offering easy-to-use, board-specific functions. This makes controlling peripherals and developing applications simpler and less prone to errors.

Purpose of BSP

The STM32F3 BSP simplifies development by:

- Initializing on-board peripherals such as LEDs, push-buttons, and sensors
- Managing board-specific pin configurations
- Providing tested, reusable drivers for hardware components

Components of the STM32F3 BSP

- LED and push button drivers (e.g., `BSP_LED_Init()`, `BSP_PB_GetState()`)
- Peripheral drivers (for ADC, I2C, SPI, timers, UART etc.)
- Startup and system initialization code
- Configuration files that define clock sources, pin mappings, and memory layout

Example Usage

```c
#include "stm32f3_discovery.h"

int main(void) {
    HAL_Init();                     // Initialize the HAL library
    BSP_LED_Init(LED3);             // Initialize on-board LED3
    while (1) {
        BSP_LED_Toggle(LED3);       // Toggle LED3 state
        HAL_Delay(500);             // Wait for 500 milliseconds
    }
}
```

BSP vs HAL

- **HAL (Hardware Abstraction Layer):** Provides generic peripheral access functions (e.g., GPIO, UART).
- **BSP (Board Support Package):** Provides board-specific functions that utilize HAL internally (e.g., controlling Discovery Board's LED3 without knowing which GPIO pin it is connected to).

Useful Links

This page covers a wide range of BSP functions to control the on-board hardware: `https://documentation.help/STM32F3-Discovery-BSP/`

1.1.3 KeyesStudio Self Balancing Kit

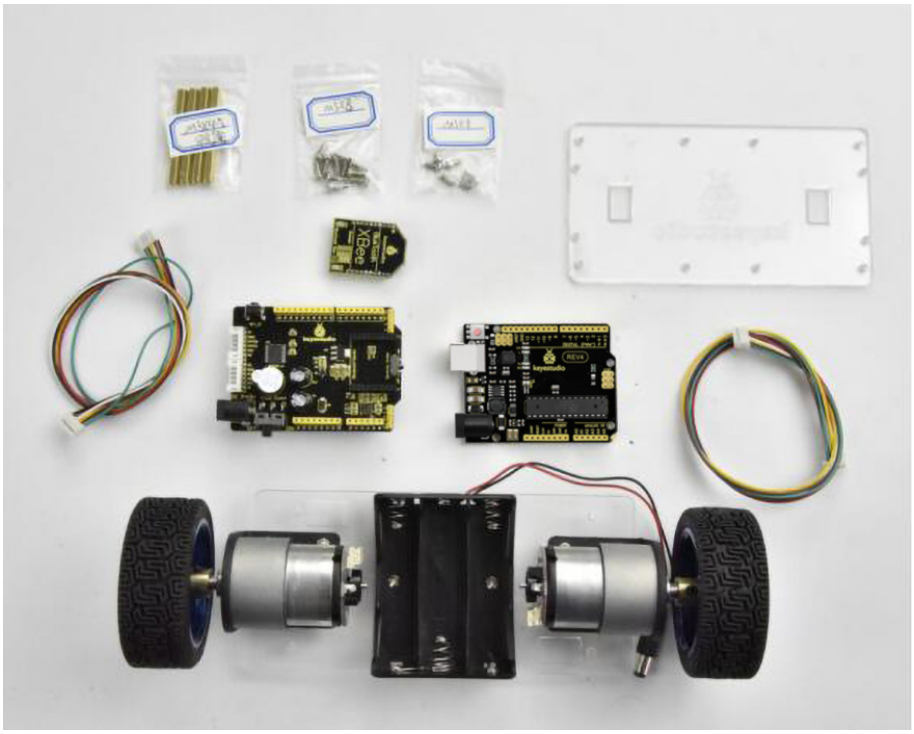


Figure 1.3: Balancing robot kit in it's un-assembled form

The KeyesStudio Self-Balancing Robot Kit is a practical tool used in this course to help students learn about control systems and embedded programming. It includes sensors, motors, and a AVR based microcontroller board that allow the robot to balance itself on two wheels. For this lab, all parts of the kit except the microcontroller board will be used. For the microcontroller, AVR will be replaced with STM32F3 discovery board.

The self-balancing robot stays upright by constantly adjusting its position based on sensor feedback. It uses sensors like gyroscopes and accelerometers to measure its tilt angle. This information is processed by the microcontroller, which then controls the motors to move the robot forward or backward. By quickly correcting its position, the robot maintains balance on two wheels. This process is often managed using a control method called PID (Proportional-Integral-Derivative) control, which helps the robot respond smoothly and accurately to changes in its tilt.

Students will use this kit to apply concepts such as sensor interfacing, motor control, and PID (Proportional-Integral-Derivative) control. Working with the kit gives hands-on experience in designing and testing real-time control systems. And of course it is fun and rewarding to balance a robot on two wheels.

For a step-by-step visual guide to assembling the robot kit, refer to the official video tutorial by Keyestudio: https://www.youtube.com/watch?v=PTaSzV7YRuE

1.2 Balance Car Sheild

The self-balancing robot shield is an expansion board that integrates motor driving, sensing, power management, and communication interfaces commonly used in mobile robotics applications. It includes a dual motor driver, an inertial measurement unit (IMU), power regulation circuitry, and multiple GPIO, UART, and I^{2}C interfaces. The labeled components in Figure **??** provide an overview of the hardware available on the shield.

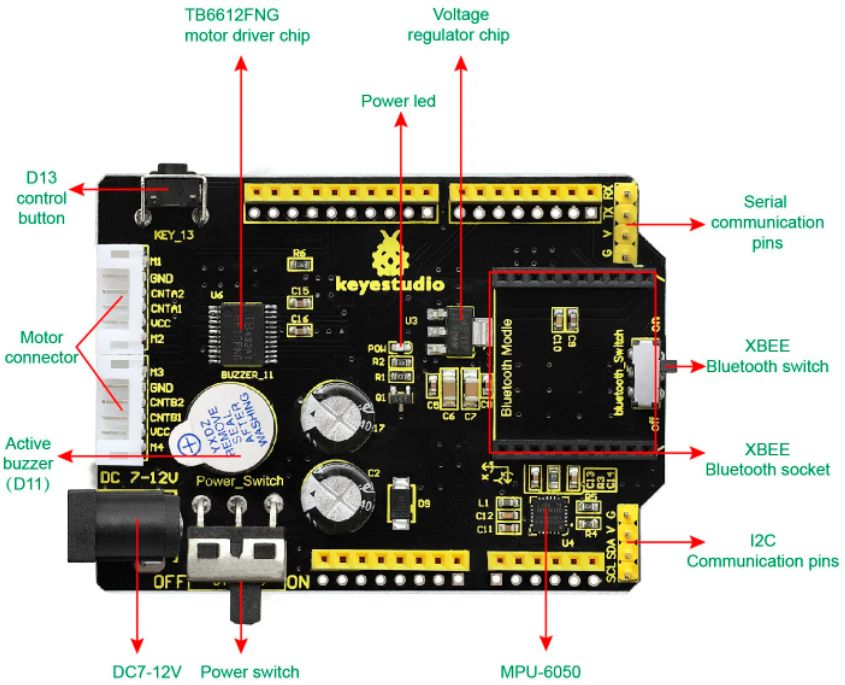
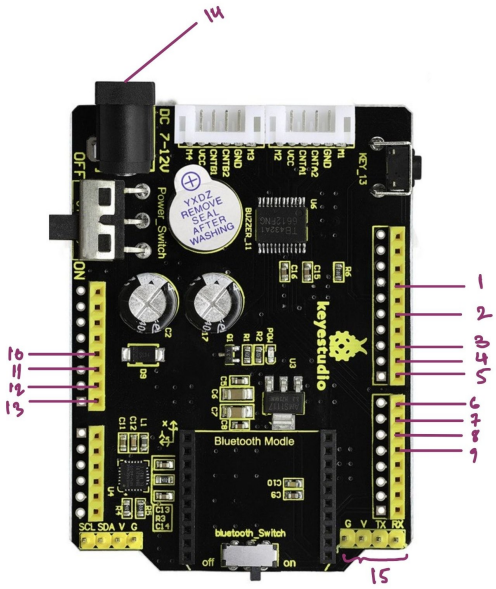


Figure 1.4: Car Shield v3

1.2.1 Useful Pins



#	Pin	Function
1	GND	Ground reference; connect to STM32 GND
2	D12	Direction control for **right motor**
3	D10	PWM speed control for **right motor**
4	D9	PWM speed control for **left motor**
5	D8	Direction control for **right motor**
6	D7	Direction control for **left motor**
7	D6	Direction control for **left motor**
8	D5	Encoder input for **left motor**
9	D4	Encoder input for **right motor**
10	5V	5 V logic supply
11	GND	Ground pin
12	GND	Ground pin
13	Vin	Motor supply input (7–12 V)
14	Vmotor	Dedicated motor power rail
15	TX/RX	UART pins for Bluetooth

Figure 1.5: Pin mapping of the self-balancing robot shield

1.3 Practical Information

1.3.1 Groups

Each lab will be conducted in groups of two students and the project will be completed and graded in groups. The project group registration will happen during the third lab session. Each group will be assigned a unique number that matches a specific workbench, including its computer and lab equipment.

Since the project needs equipment such as oscilloscopes and function generators, most groups will prefer to work in the lab. The workbenches will be used by other groups also. Therefore, make sure to clean your area before you leave.

1.3.2 Lectures

The instructor or research assistant will give a lab lecture before each exercise. These lectures will focus on topics directly related to the exercises and will include helpful information, hints, and tips.

1.3.3 Approval of Exercises

After finishing each exercise, students must show their results to the research assistant to get approval for grading. If a group cannot get approval on the lab day, a late penalty will apply for getting it later. If the group does not get approval within 7 days (before the next lab), they will receive a zero for that lab.

In addition, Habib University's exam rules and policies on submission and plagiarism will be followed.

1.3.4 Lab

The lab is located on the lower ground floor in the Circuits Lab (C-001). Students enrolled in the course will have automatic access.

The lab will be reserved for this course on Tuesdays from 10:00am to 1:00pm for T2 and 2:40pm to 5:40pm for T1. The instructor and research assistant will be present during these sessions. Ramzan timings will differ and will be communicated later by RO.

Students can also use the lab outside these hours if they get prior permission from the lab custodians, Ms. Maham Tabassum.

The lab contains cabinets for temporary storage of group components and materials; however, the security of items stored in these cabinets cannot be guaranteed. Students are advised not to store valuable or sensitive equipment in these cabinets. For secure storage, university-provided lockers located in designated areas may be used at the student's discretion.

Please note that lab equipment, such as oscilloscopes, function generators, power supplies, and multimeters, is shared among courses and groups. Do not remove, relocate, or lock any shared equipment, as it must remain accessible to all students.

Computers and Equipment

The lab is equipped with computers that have all the required software pre-installed. These computers will mainly be used for software development, reading datasheets, and interfacing with microcontrollers. Installing additional software is not allowed. If you think any essential software is missing and would benefit all students, please contact the research assistant to request its installation.

Specialized or group-specific tools and software must be run on your personal laptop. Do not save project files or code on lab computers, as these may be upgraded during the semester without notice, causing loss of all local data. Instead, use the git and github version control system to securely store your work remotely.

The lab has oscilloscopes, multimeters, power supplies, function generators, and other tools needed for experiments and hardware tasks in the course.

1.3.5 Course Learning Outcomes

CLO	Description	LDL
1	Demonstrate programming skills for state-of-the-art microcontrollers for relevant engineering problems	Psy – 3
2	Demonstrate skills in interfacing a microcontroller to various real-world devices	Psy – 3
3	Demonstrate skills in using timers and interrupts for various real-world microcontroller-based applications	Psy – 3
4	Adhere to the deadlines, instructions, and integrity	Aff – 4
5	Demonstrate proactive participation in lab discussion and group project	Aff – 3

1.3.6 Schedule

Week	Lab Experiment	Weightage	CLO
1	Getting Started with Git and VS Code for Version Control	4%	1, 4
2	Getting Started with STM32 Microcontroller: Implementing UART and Programming in C	6%	1, 4
3	Applying GPIO and Digital I/O Control to a 7-Segment Display	6%	2, 4
4	Configuring LED Control Using Timers: Polling and Interrupts	6%	3, 4
5	Using Timers for PWM Generation and Motor Speed Control	6%	3, 4
6	Using Timers to Implement Capture/Compare Mode	8%	3, 4
7	Acquire and Filter Analog Signals Using ADC and CMSIS DSP	6%	2, 4
8	Establish SPI Communication with the L3GD20D Gyroscope Sensor	6%	2, 4
9	Integrate I^2C Communication with the MPU-6050 for Accelerometer and Gyroscope Data Acquisition	6%	2, 4
10	Investigate Dynamic Memory Management and Heap Allocation Techniques in Embedded C	6%	1, 4
11	Open Ended Lab: Design and Validate a Feedback-Controlled Motor Speed System	10%	2, 4
15	Project Evaluation	30%	1, 2, 3, 4, 5

Table 1.3: Weekly Lab Schedule

1.3.7 Documentation, support material and datasheets

All required datasheets, supporting scripts and reference manuals will be available on the LMS. Please avoid printing entire documents, as they are often several hundred pages long and usually only small sections are needed. Using searchable, indexed PDF versions is more efficient than handling physical copies. However, printing important excerpts, such as pinout diagrams or peripheral summaries, can be helpful for quick reference during labs.

The STM32Cube software package also includes detailed documentation and help files. For more information about development tools, peripheral drivers, and debugging, refer to the User Manuals and the STM32CubeMX help system.

Useful Links

- STMicroelectronics – STM32CubeF3 Discovery Board Examples (GitHub)
- STMicroelectronics – STM32F3 Series Reference Manual (RM0316)
- STMicroelectronics – STM32F3 Discovery Kit User Manual (UM1570)
- Keyestudio – Ks0193 Self-balancing Car Wiki Page
- Espruino – STM32F3DISCOVERY Board Reference

1.3.8 Standard components for the lab and fair use policy

You will be provided with all the components needed for the lab at the start of the semester. The components are costly and delicate, therefore use them with extreme care. You are required to return the components at the end of lab. In case some components are missing or damaged, a financial penalty will be imposed.

Penalty for Misplacing/Damaging Lab Equipment Any lab equipment issued to students must be handled carefully and returned in its original condition. If equipment is broken, damaged, or lost, the department will be notified, and the cost will be deducted from the student's safety deposit. If the damage cost exceeds the deposit, the extra charges will be added to the student's semester fees or may result in a financial hold on their university account until the amount is paid. Students are therefore advised to take full responsibility for the equipment assigned to them during the course.

1.3.9 Evaluation

At the end of the semester, all lab and project code must be submitted by pushing it to the assigned Git repositories on the LMS. Students are expected to use Git properly, including writing clear commit messages and making regular updates throughout the course. Labs 1 to 10 will each contribute 6% to the final grade, while lab 11 will contribute 10%. A portion of each lab's grade will be allocated based on proper use of the Git version control system. The project will make up 30% of the final grade.

Groups are required to present their project results during a scheduled demonstration and oral presentation with the course instructor. Each group will have about 15 minutes for their presentation. The source code must be fully uploaded before the presentation. Additionally, a written project report must be submitted before the evaluation day. The report should clearly describe the project objectives, design, implementation and results.

Although formal presentation slides are not required, groups are encouraged to prepare a short (3–5 minute) oral presentation to explain their work and help the evaluators understand it. This will be followed by a question and answer session.

The project grade will be based 80% on completing and functioning required exercises, and 20% on additional features and creativity. Scores will be decided on the evaluation day, considering factors such as overall project functionality, code quality and readability, system organization, hardware setup,

quality of the written report, presentation skills, group participation, and ability to answer questions. Approval of exercises shows that key parts were working at some point but does not guarantee full marks; this will be considered if problems arise during evaluation. Group members will receive a joint score for the project.

Additional details about the evaluation process will be provided closer to the evaluation date.

1.4 Rubrics

1.4.1 Lab Rubrics

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR1	Circuit Layout	Connections between circuit components are mostly wrong. Circuit layout is cluttered. Needs guidance to make correct equipment/ component connections.	Few of the connections made between circuit components are wrong. Circuit layout is not neat and clean as per standards.	Circuit layout and connections are correct but not all connections are neat and clean as per standards.	Neat, clean and correct connections of all the circuit components/ equipment are made as per standard circuit diagram.
LR2	Program/Code /Simulation Model/ Network Model	Program/code/ simulation model/ network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code /simulation model /network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code /simulation model /network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code /simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR3	Troubleshooting	Unable to identify the fault/minimal effort shown in troubleshooting.	Able to identify the fault but unable to remove it.	Able to identify the fault but partially removes it.	Able to identify the fault and takes necessary steps and actions to correct it.

LR4	Data Collection	Measurements are incomplete, inaccurate and imprecise. Observations are incomplete or not included. Symbols, units and significant figures are not included.	Measurements are somewhat inaccurate and imprecise. Observations are incomplete or vague. Major errors are there in using symbols, units and significant digits.	Measurements are mostly accurate. Observations are generally complete. Minor errors are present in using symbols, units and significant digits.	Measurements are both accurate and precise. Data collection is systematic. Observations are very thorough and include appropriate symbols, units and significant digits and task completed in due time.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR6	Calculations	Formulae used and/or calculations are incorrect. Units are not mentioned with results.	Formulae used are correct but there are few mistakes in calculation which lead to incorrect results.	Formulae used and end results are correct. Complete working steps are not shown. Proper units are not mentioned with results.	Formulae used, end results and all calculations are correct with all the intermediate steps clearly shown. Units are mentioned with results.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR8	Equipment/ Instrument Handling	Inappropriate handling of the tools and equipment with minimal accuracy.	Appropriate handling of some of the tools and equipment.	Appropriate handling of most of the tools and equipment.	Appropriate handling of all the tools and equipment.

LR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.
LR10	Analysis	Results are not interpreted or are analyzed incorrectly.	Analysis of the obtained results is incomplete. Conclusions are not drawn.	Results are analyzed and concluded but are not well explained and justification is not provided with the help of reasoning.	Results are well analyzed supported with reasons. Good comparison is made between the theoretical and experimental results with correct and useful conclusion.
LR11	Design	Proposed design is unsubstantiated or does not satisfy most of given constraints. The breakdown of system into features is incomplete and still at lower resolution.	Justification for proposed design is weak, and it only satisfies some constraints. The breakdown of system is missing some features and resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies most given constraints. The breakdown of system into features seems complete, but resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies all given constraints. The breakdown of system into features seems complete and at appropriate resolution, given students' level.

1.4.2 Open Ended Lab Rubric

OE	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
OE1	Problem Definition	Fails to clearly define the problem or research question.	The problem has been defined to some extent but lacks precision or relevance.	The problem or research question has been defined clearly and contextually.	The problem has been defined clearly. Research has been done with precision and relevance.
OE2	Experimental Design	The proposed design is unsatisfactory and does not satisfy most of the given constraints. The breakdown of system into features is incomplete and still at lower resolution.	The justification for the proposed design is weak, and it only satisfies some constraints. The system is missing some features or resolution is not apparent in some places.	The proposed design is developed well through proper analysis, research and statistics that show the breakdown of system into components is complete, but resolution is not apparent in some places.	The proposed design is developed thoroughly through proper analysis, research and statistics that show the breakdown of system into appropriate resolution. The features seem complete and appropriate to satisfy all given constraints.
OE3	Data Collection	Measurements are incomplete, inaccurate and imprecise. Observations are incomplete or not indicated. Symbols, units and significant figures are not used.	Measurements are somewhat inaccurate or imprecise. Observations are missing or sparse. Major errors are there in the use of symbols, units and significant figures.	Measurements are mostly accurate. Observations are generally complete. Minor errors are present in using symbols, units and significant figures.	Measurements are both accurate and precise. Data collection is systematic. Observations are complete. No significant errors in symbols, units and significant figures.
OE4	Analysis & Interpretation	Fails to analyze data effectively or identify meaningful trends or interpretations.	The analysis of data is missing steps or important trends have been missed or provided limited insight.	The data has been analyzed effectively, identifying trends and drawing reasonable conclusions.	The data has been analyzed thoroughly, showing insight and drawing clear, logical, and well-supported conclusions.
OE5	Results, Plots and Report	Figures/graphs/tables are either not included or poorly constructed. Raw data presentation is unclear. Titles, captions, units are not accurate. Data presentation is either not included in the report or not present at all.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is there, but most of the values are either not explained or not well explained.	All figures, graphs, tables are accurate. Titles and captions contain minor errors and are shown. Most of the required data is also presented and meets the prescribed requirements.	All figures/graphs/tables are accurate. Titles and captions are complete and appropriate. Data presentation meets all requirements and is explained in original and meaningful way to engage readers.
OE6	Reflection	Lacks meaningful self-evaluation or reflection on the experiment.	Offers limited self-evaluation without significant insights.	Reflects on the experiment, identifying areas of success and areas for improvement.	Demonstrates critical self-evaluation, identifying successes, weaknesses, and potential areas for refining the experimental approach.

1.4.3 Project Rubric

S. No.	Attribute	Problem	Description
1	Depth of Knowledge Required	WP1	Cannot be resolved without in-depth engineering knowledge at the level of one or more of WK3, WK4, WK5, WK6 or WK8 which allows a fundamentals-based, first principles of analytical approach
2	Range of conflicting requirements	WP2	Involve wide-ranging and/or conflicting technical, non-technical issues (such as ethical, sustainability, legal, political, economic, societal) and consideration of future requirements
3	Depth of analysis required	WP3	Have no obvious solution and require abstract thinking, creativity and originality in analysis to formulate suitable models
4	Familiarity of issues	WP4	Involve infrequently encountered issues or novel problems
5	Extent of applicable codes	WP5	Address problems not encompassed by standards and codes of practice for professional engineering (the problem extends beyond the previous experiences)
6	Extent of stakeholder involvement and conflicting requirements	WP6	Involve collaboration across engineering disciplines, other fields, and/or diverse groups of stakeholders with widely varying needs
7	Interdependence	WP7	Address high level problems with many components or sub-problems that may require a systems approach

STM32F3 Board

2.1 Overview

This lab course will utilize the **STM32F303 Discovery Board**, which features the STM32F303VCT6 microcontroller, a member of STMicroelectronics' STM32 family based on the ARM® Cortex®-M4 core. This board provides a robust platform for developing and testing embedded systems and will be used throughout the course to explore a range of microcontroller interfacing concepts.

2.1.1 Microcontroller Features

The STM32F303VCT6 integrates a 32-bit ARM® Cortex®-M4 core, operating at a clock frequency of up to 72 MHz. It includes a hardware floating-point unit (FPU) and supports digital signal processing (DSP) instructions. These features enable efficient mathematical operations and signal-handling capabilities, which are essential in control systems, data acquisition, and signal conditioning applications.

Key Specifications

Feature	**Details**
Core	ARM® Cortex®-M4 with FPU and DSP support
Clock Frequency	72 MHz
Flash Memory	256 KB (non-volatile program storage)
SRAM	40 KB (volatile data memory)
GPIO	Up to 87 general-purpose input/output pins
Analog Interfaces	4 Analog-to-Digital Converters (ADCs), 2 Digital-to-Analog Converters (DACs)
Timers	Advanced, general-purpose, and basic timers
Communication Interfaces	UART, SPI, I^2C, USB
On-board Debugger	ST-LINK/V2 integrated (via mini-USB)
Operating Voltage Range	2.0 V to 3.6 V (core and I/O logic levels)

2.1.2 Power Supply Overview

The STM32F303 Discovery Board supports multiple power supply options, allowing flexible use across different development and interfacing scenarios.

Power Supply Sources

The board can be powered through:

- The ST-LINK USB connector
- The USB USER connector
- An external 3V or 5V supply voltage

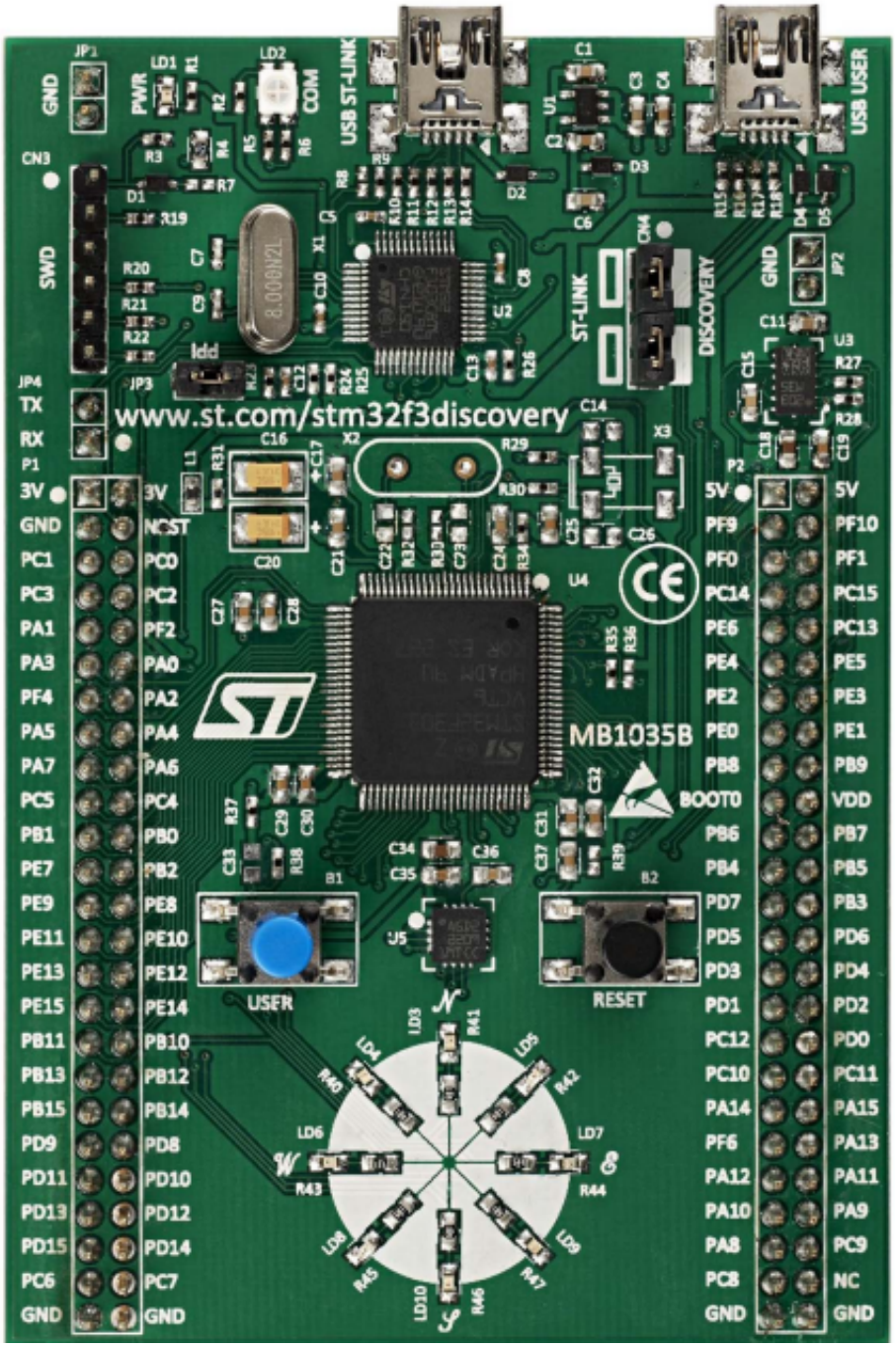


Figure 2.1: STM32Discovery Board.

Power Pins Usage

Pin	Direction	Voltage	Conditions
5V	Input/Output	5V	As output: supplies connected peripherals (max 100 mA). As input: can power the board when USB is not connected.
3V	Input/Output	3V	Same conditions as 5V. Connected peripherals must remain below 100 mA.

Recommended Default Power Setup:

- Use the ST-LINK USB connector to power the board via a PC or laptop.
- Avoid connecting power to 5V or 3V pins unless explicitly required by an experiment and under instructor supervision.

2.1.3 On-Board Peripherals

The Discovery board includes several peripherals that will be utilized in subsequent experiments:

- User LEDs (4)
- User push-button (1)
- MEMS accelerometer (LIS302DL)
- Gyroscope (LSM303DLHC)
- USB interface (for power, programming, and communication)
- ST-LINK/V2 debugger interface

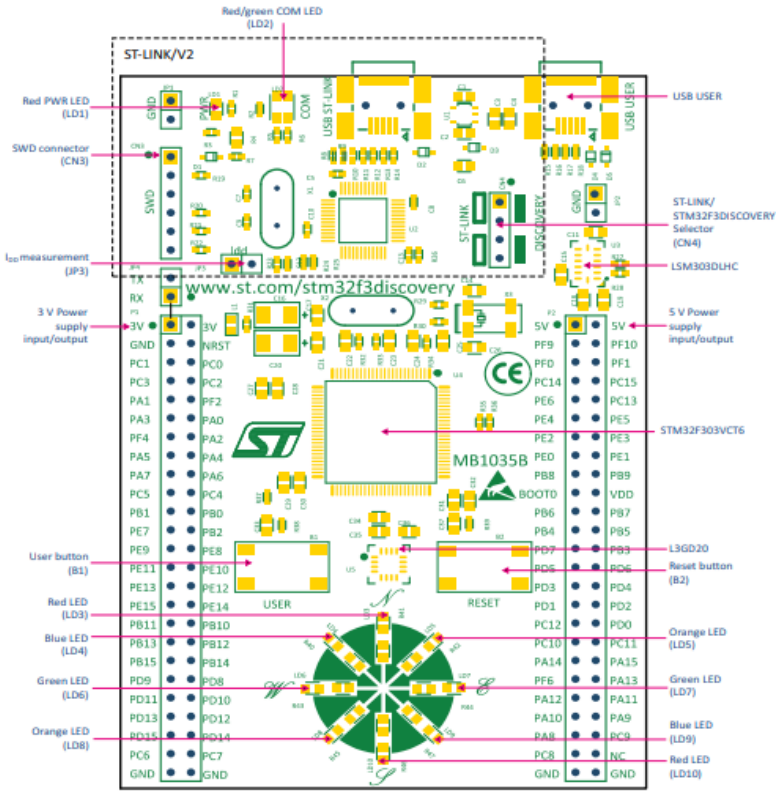


Figure 2.2: Board Layout (Top View)

2.2 Introduction to STM32F303 BSP (Board Support Package)

The Board Support Package (BSP) is a set of software components provided by STMicroelectronics that allows developers to interface directly with the hardware features of a specific STM32 development board. For this course, we will be using the STM32F3 BSP, tailored for the STM32F303 Discovery Board.

The BSP works in conjunction with the Hardware Abstraction Layer (HAL) by providing higher-level, board-specific APIs that make peripheral control and application development more efficient and less error-prone.

2.2.1 Purpose of BSP

The STM32F3 BSP simplifies development by:

- Initializing on-board peripherals such as LEDs, push-buttons, and sensors

- Managing board-specific pin configurations
- Providing tested, reusable drivers for hardware components

2.2.2 Components of the STM32F3 BSP

The BSP typically includes:

- LED and Button Drivers (e.g., `BSP_LED_Init()`, `BSP_PB_GetState()`)
- Sensor Drivers (for accelerometer, gyroscope, etc.)
- Startup and system initialization code
- Configuration files that define clock sources, pin mappings, and memory layout

Example Usage

An example of using the BSP to toggle an on-board LED:

```c
#include "stm32f3_discovery.h"

int main(void) {
    HAL_Init();                     // Initialize the HAL library
    BSP_LED_Init(LED3);             // Initialize on-board LED3
    while (1) {
        BSP_LED_Toggle(LED3);       // Toggle LED3 state
        HAL_Delay(500);             // Wait for 500 milliseconds
    }
}
```

2.2.3 BSP vs HAL

- **HAL (Hardware Abstraction Layer):** Provides generic peripheral access functions (e.g., GPIO, UART).
- **BSP (Board Support Package):** Provides board-specific functions that utilize HAL internally (e.g., controlling Discovery Board's LED3 without knowing which GPIO pin it's connected to).

2.2.4 Useful Links

This page covers a wide range of BSP functions to control the on-board hardware: *(https://documentation.help/STM32F3-Discovery-BSP/)*

General Background Information

3.1 Hardware and electronics

3.1.1 Breadboard

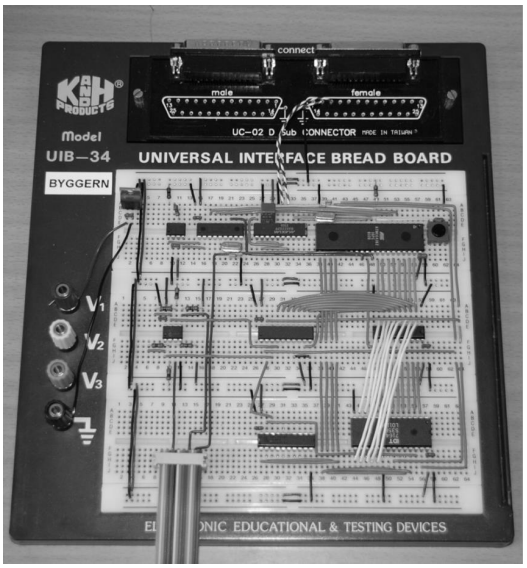


Figure 3.1: Example of well organized breadboard

A breadboard is a construction base for prototyping of electric circuits. Integrated circuits (ICs) and wires can be placed directly into the board without any soldering. This allows one to build and modify a circuit quickly and easily. Holes are interconnected in groups, so that one can place an IC without short-circuiting its pins and access its pins from several holes.

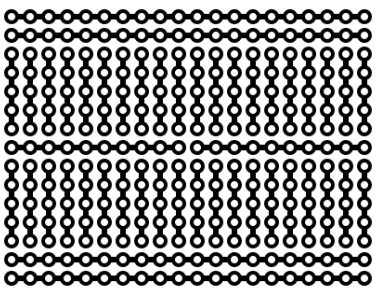


Figure 3.2: Typical hole pattern of a breadboard

Fig. 3.2 shows a typical hole pattern for a breadboard. Try to understand how holes are connected by using the multimeter. When adding and connecting new components it is relatively easy to end up with a chaotic "rat's nest" of tangled wires which is almost impossible to work with. Care must therefore be taken to use cables of proper length and color, and to place them in a systematic manner. Ask the RA/TA if you need more/longer cables if the supplied set or the reserve at the lab doesn't suffice. This extra effort will pay off when new components have to be added to an already densely populated breadboard.

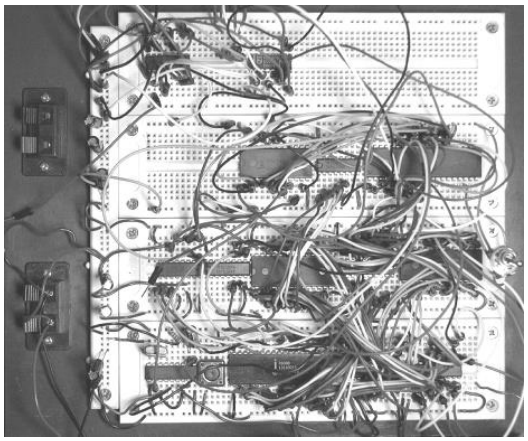


Figure 3.3: Messy assembly

3.1.2 Noise, grounding and decoupling

Although being quick and easy for circuit prototyping, electrical noise is an inherent problem with all breadboards. Thus, proper decoupling of all ICs and ensuring good ground connections, especially to the crystal and microcontroller, is extremely important. Connect all ICs with star-point grounding (all ground connections should originate from the same physical point). The same principle applies for the voltage supply. Try to experiment with grounding if you experience noise problems. You should avoid (large) closed loops of wire while trying to make all connections logical and well-arranged at the same time.

Decoupling

Unfortunately, the supply lines to the circuits are not ideal as they all introduce certain amount of impedance (resistive, inductive, and capacitive) between the power source and the IC, which may lead to undesirable AC effects and noise that affects IC operation. Moreover, the power source itself has a finite response time and can only accommodate changing demands in current supply up to a limited frequency (depending on supply type). As shown in the datasheets, the power consumption of an IC is not constant and they usually draw a highly variable amount of current while operating, e.g. when CMOS circuits change their logical state. This combination will result in noise-like voltage drops occurring at the voltage supply pins of the IC, which can cause considerable trouble and malfunction of the circuit. Furthermore, the problem will typically get worse and more unpredictable when several ICs are connected to the same supply lines.

The most important remedy against these undesirable effects is the decoupling capacitor. The decoupling capacitor is a capacitor that is connected between ground and the voltage supply pins of the IC. Importantly, it must be connected as (physically) close to the IC as possible in order to limit the effect of lead impedances. The decoupling capacitor will then operate as a fast local current source that will counteract voltage drops during transient changes in IC power demand. Decoupling capacitor will work as an effective shunt against noise signals of certain (high) frequencies that may occur on the supply lines. Capacitor value depends on the noise frequencies but 10-100 nF seems to be a good compromise in many situations.

Star-point grounding

Star-point grounding is an important concept. In the connection demonstrated in Figure 3.4, the voltage difference from “real” ground to an IC’s ground terminal will depend on the current consumption of the other ICs along the ground rail because of non-zero impedance of electrical conductors. Thus, the ground potential of each IC is bound to be different between the different ICs in the circuit and will depend on the ground return current, which is clearly an undesirable situation. In addition, if the ground wires are connected as a closed loop as shown in Fig 3.4, known as a ground loop, a particularly critical situation may appear. The circuit will be very vulnerable to induced electrical noise from surrounding magnetic fields (motors, solenoids, transformers etc.) and this type of connection has to be avoided at any cost. Figure 3.5 shows how this problem can be remedied by connecting all ground return wires to a common point close to the supply ground in a star-like connection. It is also desirable to reduce the length of the common ground wire as well as attaining a more balanced wire length. Star-point connection should also be used for the voltage supply.

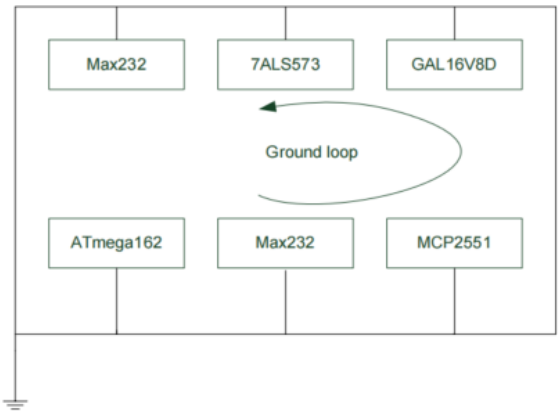


Figure 3.4: Ground loop (wrong)

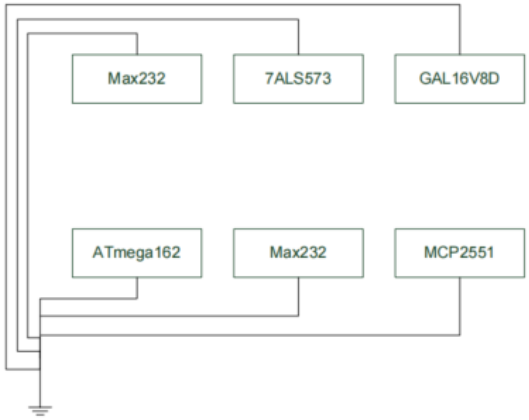


Figure 3.5: Star grounding (correct)

Power drain

Some components, especially the solenoid, drain a lot of transient current when activated. This might lead to voltage drops on the supply lines and ground bounce with subsequent reset of microcontrollers and malfunction of communication lines as a result. Decoupling capacitors may not be able to absorb noise transients of those magnitudes and isolating such devices to a separate voltage supply may prove to be the best solution.

3.1.3 LEDs

LEDs can be a valuable debugging tool for instance when used to display the logical value of a signal. Moreover, they often serve as indicators showing that e.g. the power supply is enabled, the system state is initialized etc. It is also common to connect LEDs to digital output pins and control these from software to indicate different states, values, control flow and so on. For example a LED that toggles for each X iteration of the main program loop, provides a visual indication that the code is in fact executing.

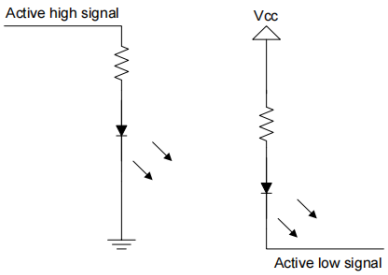


Figure 3.6: Driving a LED. Typical value for the resistor is between 100Ω - 350Ω.

Fig. 3.6 shows two different LED configurations – one gives light when the output signal is high, the other one when the signal is low. Intuitively, a lit diode could indicate a high signal, but most logical circuits can sink more current than they can source, and thus one should consider using the active-low configuration. Another solution is to use a LED driver circuit (buffer/amplifier), but this requires a slightly more complicated circuit. Verify in the datasheet that the IC output is rated for the current source/drain that is needed to operate the LED correctly.

3.1.4 Pull-up and pull-down resistors

Some ICs and circuits are deliberately designed to only be capable of driving their output signals low. Outputs cannot be actively driven to a high level, that is, the output is left floating/high impedance as long as the output is not low. In such cases a high output level has to be ensured by connecting an external resistor, also known as a pull-up resistor, between the output and the logical high voltage level (whatever that might be). Pull-up resistors have many uses and are essential e.g. in open collector logic, level conversion and communication buses such as I2C. The resistor value affects the transient response of the output and the current consumption and has to be selected accordingly. Typically, values in the range 10 k - 100 k are used with CMOS circuits. Pull-down resistors works in the same fashion, only the other way around with the resistor connected between the output and the ground (low) level.

3.1.5 Logical levels (5V or TTL vs 3.3V or CMOS)

In digital electronics, a logic level refers to the voltage range used to represent a logical HIGH (1) or LOW (0). Different families of digital components use different logic levels. The two most common standards are TTL (Transistor-Transistor Logic), typically operating at 5V, and CMOS (Complementary Metal-Oxide-Semiconductor), which often uses 3.3V or lower.

TTL (5V logic): In TTL systems, a logical HIGH is typically any voltage above 2.0V (usually close to 5V), while a logical LOW is any voltage below 0.8V. TTL devices can tolerate 5V inputs and outputs but are not inherently compatible with lower-voltage systems.

CMOS (3.3V logic): Modern CMOS logic, such as that used in STM32F3 microcontrollers, operates at 3.3V. For these devices, a HIGH is recognized at voltages typically above 2.0V, and a LOW is below 0.8V

Incompatibility and Damage Risk: Interfacing 5V TTL outputs with 3.3V CMOS inputs can cause problems. While a 5V signal is typically interpreted correctly as HIGH by a 3.3V system, feeding 5V directly into a 3.3V input may exceed the maximum voltage tolerance of the input pin, potentially damaging the device.

Level Shifting: To interface devices with different logic levels safely, *level shifters* should be used. These are circuits or ICs that translate signals from one voltage domain to another. Alternatives include resistor voltage dividers or open-drain outputs with pull-up resistors, depending on the direction and nature of communication.

STM32F3 Voltage Levels: According to the STM32F303 reference manual, the I/O pins are not 5V-tolerant unless specifically stated. Most general-purpose I/Os operate with a 3.3V logic level, and applying voltages significantly higher than V_{DD} may damage the pin.

Understanding logic levels is crucial when connecting the STM32F3 to external components like UART modules, sensors, or motor drivers. Always consult the datasheets of both the microcontroller and the peripheral to ensure voltage compatibility.

3.1.6 Micro-controllers and STM32F3

A microcontroller is a compact, integrated circuit designed to execute specific control tasks. It typically includes a processor (CPU), memory (RAM and Flash), and peripheral interfaces (such as GPIO, ADC, UART, SPI, etc.) all on a single chip. Microcontrollers are widely used in embedded systems ranging from household appliances to robotics.

In this course, we use the **STM32F3 series** from STMicroelectronics, based on the 32-bit ARM Cortex-M4 architecture. The STM32F3 microcontroller combines high performance with low power consumption and features a wide range of peripherals including analog comparators, fast ADCs, timers, communication interfaces (UART, I2C, SPI), and floating-point support. This makes it well-suited for real-time control applications such as balancing robots, motor control, and sensor fusion.

The specific board used in this lab is the **STM32F3 Discovery**, which provides easy access to on-board peripherals and headers for external modules, making it ideal for both learning and prototyping.

3.1.7 Hardware debugging and general tips

There will be errors and bugs when assembling the breadboard. And that's completely normal ;) Debugging might be difficult, especially for the teaching assistants who don't know the details of your setup. Before asking for help, make sure to go through the following debugging steps, make schematics and ensure that you follow the tips given here and in the relevant exercise. Read the instructions given in the exercise again and make sure you have carried out the steps exactly as they are described there.

1. If you have completed an exercise without testing and then run into problems, you should backtrack and verify that each step is done correctly.
2. The lab is provided with multimeters and oscilloscopes. Use these frequently when assembling and debugging the hardware and software to verify that everything works correctly.
3. Read the datasheets carefully – they contain (almost) everything you need to know.
4. Verify all connections (pin to pin, not wire to wire) using the multimeters continuity test (remember to turn off power supplies first!) the right way.
5. Check that you have connected positive voltage and ground correctly to all components.
6. Ensure that the power supply is on.
7. Measure the voltage between the ground pin and voltage supply pin.

3.2 Microcontroller programming in C

3.2.1 Modularization and drivers

As the robotic system becomes more complex, the number of code files and lines will grow significantly. Without proper structure, the software may quickly become unmanageable, making debugging and further development difficult. In an embedded system such as the STM32F3-based self-balancing robot, different subsystems (e.g., motors, sensors, filters, communication interfaces) interact closely. If care is not taken to modularize the code, unintended interference between components can occur, leading to bugs that are difficult to isolate.

To ensure maintainability, scalability, and reusability, the code should be organized into clear modules or ”drivers”, where each module handles a specific hardware or logical function. For example, in the later stages of the project, your system may consist of:

- A PWM driver to control DC motors
- A GPIO driver for push buttons and LEDs
- A UART driver for serial debugging
- An SPI or I2C driver for communication with sensors (e.g., IMU)
- A sensor driver that interprets IMU readings into angle estimates
- A motor control interface that uses PID to balance the robot

Each driver should consist of a header file (`.h`) and an implementation file (`.c`). The header file exposes a clean and minimal interface — mainly function prototypes, constants, and type definitions — that other modules can use. The implementation file contains the internal logic, which should remain hidden from other modules unless explicitly required.

This modular structure:

- Reduces inter-dependencies between parts of your system
- Makes it easier to isolate, test, and debug individual components
- Encourages code reuse in future projects
- Enables clearer collaboration within teams

Avoid unnecessary global variables and tightly coupled dependencies. For instance, a high-level motor control module should not directly access timer registers. Instead, it should interact with an abstract PWM driver. This decoupling ensures that the motor control logic remains portable even if the underlying PWM hardware or implementation changes.

Similarly, sensor fusion and filtering should not be embedded directly inside low-level drivers. For example, a complementary or Kalman filter that uses IMU data should live in a separate layer above the raw sensor driver. This separation enables you to switch between sensors or filters with minimal changes to the rest of the system.

The STM32Cube HAL already encourages modular programming by providing standardized APIs for each peripheral. You should build your application logic on top of these HAL drivers wherever possible, and avoid mixing high-level logic with low-level hardware control.

For tips on modularizing C code, refer to:

- Modular Programming Tips – Fast Product Development
- C Modules Guide – Icosaedro

3.2.2 Documentation

As far as possible, your variables, functions, and `#define` constants should have self-explanatory names. Clear and descriptive naming eliminates the need for excessive comments and helps make the code easier to read and maintain.

However, it is still essential to include comments and documentation for functions that have non-obvious purposes, as well as for algorithms that may be difficult to follow. Your comments should help others, including your teammates, research assistants, and future you, understand the logic and structure of the program. Well-documented code will also make project evaluation easier and contribute positively to your final grade.

There are automated tools available that can generate professional, navigable documentation directly from your source code. One such tool is **Doxygen**, which can create documentation in HTML, PDF, or other formats by parsing specially formatted comments. More information and setup instructions can be found at: http://www.stack.nl/˜dimitri/doxygen/.

3.2.3 Ports

Most of the pins on STM32F3 microcontrollers can be configured by the application software as digital inputs or outputs using dedicated control registers. Groups of pins are organized into ports, each having up to 16 pins. The STM32F303VCT6 microcontroller on the STM32F3 Discovery Board includes ports GPIOA through GPIOF, each with 16 configurable pins.

Each port is controlled by several 32-bit registers: `MODER`, `OTYPER`, `OSPEEDR`, `PUPDR`, `IDR`, `ODR`, and `BSRR`, among others. Each bit (or pair of bits) in these registers corresponds to one pin on the port. The `MODER` register is used to configure each pin individually as input, output, alternate function, or analog. Pins within the same port can be configured differently and operate independently.

The output data register (`ODR`) is used to control the state of output pins — writing a '1' to a bit sets the corresponding pin to a logical high value, while writing a '0' sets it to a logical low value. STM32 also provides a dedicated bit set/reset register (`BSRR`) that allows atomic setting and clearing of individual output pins. For input pins, internal pull-up or pull-down resistors can be enabled using the `PUPDR` register. A '01' setting enables a pull-up, '10' enables a pull-down, and '00' leaves the pin floating.

The input data register (`IDR`) is read-only, and each bit reflects the current logic level of the corresponding input pin.

Note that most pins have shared or alternate functionality. That is, other peripheral units on the microcontroller may override or be overridden by the GPIO configuration. For example, timer channels, SPI, UART, ADC, and other peripherals are mapped to specific GPIO pins through alternate function registers (`AFR[0]` and `AFR[1]`). It is the responsibility of the programmer to ensure that the correct mode and alternate function are selected when using such peripherals, and that conflicts between overlapping functions are resolved.

3.2.4 Bit Manipulation on STM32F3 Discovery

Microcontroller configuration on STM32 devices is also accomplished by manipulating bits in dedicated control registers. While STM32Cube HAL provides high-level abstractions, understanding and directly accessing peripheral registers offers greater flexibility and efficiency.

The STM32F3 series is based on the ARM Cortex-M4 architecture, and all peripheral registers are memory-mapped. CMSIS (Cortex Microcontroller Software Interface Standard), included via `stm32f3xx.h`, defines register addresses and bit positions using structured pointers.

Using HAL Macros (Recommended for Portability)

HAL provides symbolic macros for GPIO manipulation:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);   // Set PA5
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);                // Toggle PA5
HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5);                  // Read PA5
```

Using Register-Level Bit Manipulation

For advanced or time-critical applications, you can access GPIO and peripheral registers directly.

Examples:

Set PA5 high (output):

```
GPIOA->ODR |= (1 << 5);
```

Clear PA5:

```
GPIOA->ODR &= ~(1 << 5);
```

Toggle PA5:

```
GPIOA->ODR ^= (1 << 5);
```

Check if PA5 is high:

```
if (GPIOA->IDR & (1 << 5)) { /* PA5 is high */ }
```

Set/Reset PA5 using BSRR (atomic):

```
GPIOA->BSRR = (1 << 5);         // Set PA5
GPIOA->BSRR = (1 << (5 + 16)); // Reset PA5
```

Useful macros for direct bit manipulation:

```
#define set_bit(reg, bit)   ((reg) |= (1U << (bit)))
#define clear_bit(reg, bit) ((reg) &= ~(1U << (bit)))
#define toggle_bit(reg, bit) ((reg) ^= (1U << (bit)))
#define test_bit(reg, bit)  ((reg) & (1U << (bit)))
```

Example:

```
set_bit(GPIOA->ODR, 5);     // Set PA5
clear_bit(GPIOA->ODR, 5);   // Clear PA5
toggle_bit(GPIOA->ODR, 5);  // Toggle PA5
```

While the HAL abstracts much of this logic for ease of use, understanding bitwise operations and register-level access is crucial for performance-critical and low-level embedded development on STM32 platforms.

3.2.5 Polling and Interrupts

Embedded systems often need to react quickly to external events. There are two main mechanisms for event detection: **polling** and **interrupts**.

Polling (Busy Waiting)

Polling is a method in which the processor constantly checks the status of a register to determine whether an event has occurred. It is easy to implement but can be inefficient, especially when fast response or multitasking is required.

Interrupts

Interrupts allow the processor to pause its main task and jump to a specific function, called an *interrupt service routine (ISR)*, to handle an event. The processor may determine the source of an interrupt by:

- Polling all interrupt sources (polling interrupts), or
- Receiving the identity directly from the interrupting device (vectored interrupts).

Interrupts are more complex but save processor time and provide faster response. The choice between polling and interrupts depends on application needs.
Timers and Scheduling: Microcontrollers often use timer interrupts to periodically wake the system for task execution. With careful programming, this can mimic a basic OS task scheduler.

By default, all other interrupts are disabled during an ISR, but this can be changed. Use:

```
__enable_irq();  // Enable global interrupts
__disable_irq(); // Disable global interrupts
```

Polling Example (STM32F3)
8 buttons on PORTA, 8 LEDs on PORTD:

```
#include "main.h"

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();  // Set up GPIOA as input, GPIOD as output

    while (1)
    {
        uint16_t buttons = GPIOA->IDR & 0x00FF;      // Read PA0 to PA7
        GPIOD->ODR = (GPIOD->ODR & 0xFF00) | buttons; // Write to PD0 to PD7
    }
}
```

Interrupt Example (STM32F3)
Button on GPIOA PIN0, toggles an LED (e.g., GPIOD PIN13):

```
#include "main.h"

volatile uint8_t BUTTON_PRESSED = 0;

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    while (1)
    {
        if (BUTTON_PRESSED)
        {
            // Respond to button press
            HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
            BUTTON_PRESSED = 0;
        }
        HAL_Delay(10); // Optional debounce
    }
}

// Interrupt handler for EXTI line 0
void EXTI0_IRQHandler(void)
```

```
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0); // Required by HAL
}

// Callback function called by HAL after line interrupt
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
    {
        BUTTON_PRESSED = 1;
    }
}
```

3.2.6 Structs and Unions in C

Although C is not object-oriented, `structs` enable grouping related variables into single entities. For example, a MEMS sensor could be organized in struct as:

```
    typedef struct {
    float     gyroscope_deg_per_sec;        // gyroscope sensor value
    float     tilt_deg;                     // accelerometer sensor value as tilt
    using approx.
    float     gyroscope_static_offset;      // gyroscope fixed offset for
   correction
    float     accelerometer_static_offset;  // accelerometer fixed offset for
   correction
} mems_sensors_data_t;
```

The `typedef` allows using `mems_sensors_data_t` as a data type later in the program.

3.2.7 Useful libraries

Refer to the official STM32 HAL API documentation (UM1786) for function-level details and usage examples.

3.3 Tools and software

3.3.1 STM32CubeIDE

STM32CubeIDE is an advanced C/C++ development platform provided by STMicroelectronics for STM32 microcontrollers and microprocessors. It combines:

- Peripheral configuration,
- Code generation (via STM32CubeMX integration),
- Compilation (based on GCC),
- Advanced debugging.

File Name	Description
`stm32f3xx_hal.h`	Main HAL header file; includes all peripheral HAL drivers for STM32F3.
`stm32f3xx_hal_gpio.h`	GPIO initialization, pin control, input/output toggling.
`stm32f3xx_hal_tim.h`	Timer configuration, PWM generation, delays.
`stm32f3xx_hal_adc.h`	Analog-to-digital converter driver.
`stm32f3xx_hal_uart.h`	UART communication (e.g., serial debugging over USB).
`stm32f3xx_hal_rcc.h`	System clock configuration and peripheral clock control.
`stm32f3xx_hal_i2c.h`	I2C communication (e.g., sensors like accelerometers).
`stm32f3xx_hal_spi.h`	SPI communication (e.g., OLEDs, sensors).
`stm32f3xx_hal_conf.h`	Project-level configuration of HAL modules (included in `main.h`).
`core_cm4.h` (CMSIS)	ARM Cortex-M4 core definitions and intrinsic functions.
`arm_math.h` (optional)	CMSIS-DSP library for signal processing, math routines.

Table 3.1: Commonly used libraries for STM32F3 development

It is built on the Eclipse®/CDT™ framework, supporting a wide ecosystem of Eclipse plugins for extended features. STM32CubeIDE integrates the full functionality of STM32CubeMX.

Key Features:

- **Project Creation and Initialization:** Users can select an STM32 MCU/MPU, development board, or example project. Pinout, clock, peripheral, and middleware configurations are handled via GUI.
- **Code Generation:** Initialization code is automatically generated. Peripheral settings can be modified at any time and regenerated without affecting user-defined code.
- **Build and Stack Analyzers:** Provide insights into memory usage, stack size, and build status.
- **Advanced Debugging Tools:** Includes views for CPU registers, memory, and peripheral states, as well as live variable watch, Serial Wire Viewer (SWV) interface, fault analysis, and RTOS-aware debugging.
- **Debugger Support:** Supports ST-LINK and SEGGER J-Link debug probes.
- **Cross-Platform Support:** Available for Windows®, Linux®, and macOS® (64-bit).
- **Legacy IDE Compatibility:** Allows importing projects from Atollic® TrueSTUDIO® and AC6 SW4STM32.
- **STM32MP1 Series Support:** Includes tools for OpenSTLinux-based MPU development.

With STM32CubeIDE, we can configure, build, flash, and debug STM32 projects using a single unified environment.
Note: For this lab course, we will primarily use **Visual Studio Code (VSCode)** as the development environment for writing and building code. However, STM32CubeIDE may still be used for code generation or reference as needed.

3.3.2 STM32CubeMX

STM32CubeMX is a powerful graphical configuration tool that simplifies the development process for STM32 microcontrollers (MCUs) and microprocessors (MPUs). It allows users to:

- Select STM32 devices or development boards based on desired peripherals,
- Configure pins, peripherals, middleware, clock trees, and power consumption,
- Generate initialization C code for Arm® Cortex®-M projects,
- Generate partial Linux® Device Tree files for Cortex®-A based MPUs.

The tool guides users through a step-by-step process, starting with the selection of an MCU, MPU, or evaluation board. It features:

- A pinout conflict resolver,
- A clock configuration assistant,
- Peripheral and middleware configurators (e.g., GPIO, USART, USB, TCP/IP),
- A power consumption estimator,
- Built-in validation of parameter constraints.

STM32CubeMX also supports STM32Cube Expansion Packages, which can be downloaded from ST's online package manager or imported locally. Advanced users can even create their own packages using the integrated `STM32PackCreator` tool.

Once the configuration is complete, STM32CubeMX generates portable initialization code compatible with multiple IDEs including:

- STM32CubeIDE (GCC),
- IAR Embedded Workbench®,
- Keil MDK-ARM.

Key Features:

- Intuitive graphical interface for microcontroller and board selection.
- Pinout configuration with automatic conflict resolution.
- Peripherals and middleware setup with real-time constraint validation.
- Clock tree configuration with dynamic feedback.
- Power consumption analysis.
- C code generation for Cortex-M or partial Linux Device Tree for Cortex-A.
- Support for STM32Cube Expansion Packages and custom package creation.
- Cross-platform availability: Windows®, Linux®, macOS®.

STM32CubeMX is integrated into STM32CubeIDE but is also available as a standalone application.

3.3.3 VSCode IDE and STM32F3 BSP

Visual Studio Code (VSCode) is a lightweight, fast, and highly customizable code editor widely used for embedded systems development. While STM32CubeIDE provides an all-in-one solution, VSCode is preferred by many developers for its modularity, speed, and powerful extensions.

In this lab, **VSCode will be the primary environment used to write, build, and manage STM32 projects**. The setup requires additional components to support ARM development and integration with STM32 hardware.

Key Components for STM32 Development in VSCode:

- **ARM GCC Toolchain:** Compiler for Cortex-M processors.
- **CMake:** Build system to compile and link code.
- **ST-Link CLI:** Used to flash firmware onto the STM32 board.
- **Cortex-Debug Extension:** Enables GDB-based debugging directly within VSCode.
- **STM32F3 Board Support Package (BSP):** Includes headers and drivers specific to STM32F3 series.

Project Workflow:

1. Use **STM32CubeMX** to generate a project with a CMake for the STM32F3 target.
2. Open the generated project folder in VSCode.
3. Use the terminal or available buttons in VSCode to run, build, and flash commands.

Advantages of Using VSCode:

- Lightweight and faster than full IDEs.
- Highly configurable with extensions, version control, and real-time debugging.
- Seamless integration with Git and GitHub.

Note: While initial configuration may require manual steps, this environment gives more transparency into the embedded toolchain.

3.3.4 Terminal

A terminal application is required when communicating with your embedded system over serial interfaces such as UART (RS-232 or USB-to-serial). These tools allow you to send and receive data between your PC and the microcontroller, often for debugging, logging, or issuing runtime commands.

For Linux (Ubuntu) users, the built-in terminal can be used with serial communication tools such as:

- `screen` – basic serial communication utility
- `minicom` – more advanced, scriptable serial interface
- `picocom` – lightweight, minimal serial monitor
- `gtkterm` – GUI-based serial monitor for GTK environments

To use these tools, you must know the name of the serial device (e.g., `/dev/ttyUSB0` or `/dev/ttyACM0`) and the baud rate configured in your STM32 program. Example command using `screen`:

```
screen /dev/ttyUSB0 115200
```

To exit `screen`, press `Ctrl+A` then `K`.

All students are expected to become comfortable using the terminal for viewing UART output and interacting with the microcontroller throughout the lab sessions.

Version Control: Git and GitHub

4.1 What is Version Control?

Version control is a system that helps track changes to files over time. It acts as a *save history* for your projects, allowing you to revert to previous versions, compare changes, and collaborate with others safely. Using a version control system is essential for efficiently managing embedded projects. It allows you to track changes, collaborate with others, and revert to earlier versions when needed. In this course, we recommend using **Git**, a widely adopted version control system, along with **GitHub** for online storage and collaboration.

4.2 Why is Version Control Needed?

In software development and other fields, files are often modified by multiple users or over time. Without version control:

- **Track progress:** Keep a detailed history of project changes.
- **Undo mistakes:** Restore previous versions when needed.
- **Collaboration:** Multiple contributors can work without conflicts.
- **Testing:** Makes it easy to experiment in isolated branches without affecting the main codebase.
- **Backup:** Ensures your work is safely stored and recoverable.

Version control solves these problems by managing file versions efficiently and providing collaboration tools.

4.3 Available Version Control Tools

Some popular version control systems include:

- CVS (Concurrent Versions System)
- Subversion (SVN)
- Mercurial
- **Git** (Distributed)

Among these, Git has become the most widely used due to its speed, flexibility, and distributed architecture.

4.4 What is Git and Why Use It?

Git is a distributed version control system designed for speed and efficiency, used both in industry and academia. Unlike centralized systems, Git allows every user to have a full copy of the repository history on their own computer, enabling faster operations and offline work. For this lab, we will focus on and use Git extensively. It allows users to:

- Track project changes locally
- Collaborate remotely using platforms like GitHub
- Use branching for safe experimentation
- Efficient handling of large projects.
- Branching and merging capabilities facilitate feature development.

- Integration with platforms like GitHub for remote collaboration.

4.5 What is GitHub?

GitHub is a web-based platform built on top of Git that provides remote hosting of Git repositories along with tools for collaboration, issue tracking, code review, and more. GitHub is a free cloud/remote storage option for Git, however it is not the only option. Paid third part servers such as Amazon and MS Azure also offer remote storage for Git. Key features of GitHub are:

- Online collaboration and code sharing
- Pull requests and code review
- Issue tracking and project management

4.6 How are Git and GitHub Different?

Feature	**Git**	**GitHub**
Type	Command-line tool	Web-based platform
Purpose	Version control	Hosting and collaboration
Internet Required	No	Yes
Example	`git commit`, `git merge`	Creating pull requests, viewing repositories

4.7 Focusing on Git: workflow and commands

For this lab, we will be using Git via command line. There are also graphical tools available but we will not be discussing them and use of such tools for getting lab approval is not allowed. Since we will be developing our firmware using VS Code, VS Code's add-on named "gitgraph" is the visualizing tool allowed in this lab.

4.7.1 What is a Repository?

A repository (repo) is a directory (commonly called folder in Windows) that contains your project files and version history. You can think of a repository like a project folder that not only contains your files but also has a time machine built in. You can look back at previous versions, see who made which changes, and collaborate with others without overwriting each other's work. There are two types of repositories.

- **Local repository:** Stored on your computer.
- **Remote repository:** Hosted on platforms like GitHub.

When you create a Git repository, Git internally creates a hidden folder named `.git`, which contains all the metadata and history of your project:

- Your commits (history)
- References (branches, tags)
- Staging area (index)
- Configuration settings

This structure allows you to:

- Create branches for features
- Merge different development lines
- Track who made changes and when

- Revert back to previous versions safely

4.7.2 Getting Started with Git

To begin using Git on your local machine, you need two things:

1. Install Git: `https://git-scm.com/downloads`
2. Set up your Git identity. Since everything is timestamped and tracks the user who made changes, setting up identity account is a mandatory requirement:

```
    git config --global user.name "Your␣Name"
    git config --global user.email "your.email@example.com"

```

Note that your local and GitHub account details can differ. However, whenever you will be pushing changes from local to GitHub, you will need to have a GitHub account, and inside the command terminal, you will be asked to provide your GitHub credentials. You can also set your remote account credentials globally on your laptop/PC. See Git documentation for this.

4.7.3 Initializing a Repository

To start tracking a project with Git, navigate to your project folder (e.g. mci_stm32f3_projects folder) and run:

```
cd your_project_directory
git init
```

This turns the current directory into a Git repository by creating the hidden `.git` folder.

4.7.4 Cloning an Existing Repository

If you have created a repository in lab and want to work on your laptop at home, you would like to copy the same project instead of re-creating it. To achieve this, you need GitHub, i.e. remote cloud storage. In the lab, you will push your local repository to the cloud. We will see this later on but assuming that your repository is already pushed to the cloud, use:

```
cd your_project_directory
git clone https://github.com/username/repo-name.git
```

This will:

- Download all files
- Set up the complete version history
- Link the local copy to the remote GitHub repository

Standard Way of Initializing a Repository

After you initialize a Git repository using `git init`, it is good practice to immediately set up two important files:

(a) `.gitignore` — to exclude unwanted files from version control

(b) `README.md` — to describe your project and how to use it

These files help make your repository clean, professional, and easier to understand for collaborators or future contributors (including yourself).

1. Create a `.gitignore` File

The `.gitignore` file tells Git which files or directories to ignore. These are usually files that:

- Are generated automatically (e.g., compiled binaries, log files)
- Contain machine-specific or personal configurations
- Should not be shared (e.g., API keys, secrets)

Example: Minimal `.gitignore` for a C Project

```
# Ignore compiled objects and binaries
*.o
*.out
*.exe

# Ignore build folder
/build/

# Ignore logs and editor backup files
*.log
*.swp
```

Commands to use:

```
touch .gitignore
# or manually create the file in your editor
```

After creating it, stage and commit it:

```
git add .gitignore
git commit -m "Add␣.gitignore␣to␣exclude␣compiled␣files"
```

2. Create a `README.md` File

The `README.md` file is a Markdown document that introduces and explains your project. It is the first thing users and collaborators will see on platforms like GitHub.

A Good `README.md` Usually Includes:

- Project name and short description
- How to install or compile the code
- How to use it (commands or examples)
- Contact or authorship info

Example: `README.md` for a C Calculator Project

```
# Simple C Calculator

A command-line calculator that supports basic and scientific operations.

## Compile

```bash
gcc -o calc main.c math.c
```

4.7.5 Add Files to the Staging Area

Once the repository is created, you will create or modify files in your project. Git does not track them automatically. You must tell Git which files to track and include in the next commit. This is done using the `git add` command. The space where files are prepared before committing is called the **staging area** or **index**.

Syntax

```
git add <filename>
git add .
```

- `git add <filename>` adds a specific file.
- `git add .` stages all modified or new files in the current directory.

Example: C Programming Project

Suppose you are working on a C project with the following files:

- `main.c`
- `math.c`
- `math.h`
- `README.md`

After writing your code, you want Git to start tracking these files. Here's how:

```
git add main.c
git add math.c
git add math.h
git add README.md
```

Alternatively, to stage everything at once:

```
git add .
```
```

Tip: Check Staged Files

You can check what's currently in the staging area using:

```
git status
```

This command will show:

- Files staged for the next commit
- Untracked files (not yet added)
- Modified files not yet staged

Please note that all these command (in-fact all commands starting with `git` except `git init`) must be executed inside the repository folder. Once you have added the files to the staging area, you're ready to commit them using `git commit`.

4.7.6 Commit Changes

Once you've added files to the staging area using `git add`, the next step is to **commit** those changes. A commit in Git is like taking a snapshot of your project at a certain point in time. It saves the current state of the staged files into your **local** repository history.

Each commit should have a message that clearly describes what was changed and why.

Syntax

```
git commit -m "Your␣descriptive␣commit␣message"
```

Example: C Programming Project

Suppose you're working on a simple calculator written in C. After editing or creating the following files:

- `main.c`
- `math.c`
- `math.h`

As described earlier, you first stage the files:

```
git add main.c math.c math.h
```

Then commit them:

```
git commit -m "Add␣basic␣calculator␣functions␣and␣main␣program␣structure"
```

This commit will now be saved in the project's history. Git will track the fact that you added new files and what their contents were.

Why Commit Messages Matter

Commit messages are essential for:

- Understanding what changes were made
- Helping teammates follow your work
- Navigating through project history later

Good message examples:

- `"Fix bug in division by zero case"`
- `"Add unit tests for math functions"`
- `"Refactor main.c for better readability"`

Avoid vague messages like:

- `"Update"`
- `"Changes"`

View Commit History

To view a list of past commits:

```
git log
```

This shows:

- Commit IDs
- Author
- Date
- Commit messages

You can scroll through the history to understand how the project has evolved over time.

4.7.7 Link to Remote Repository

So far, all your Git operations have been local — everything is stored only on your computer. To collaborate with others or back up your work online, you need to connect your local repository to a **remote repository**, typically hosted on a platform like GitHub.

Step 1: Create a Remote Repository on GitHub

(a) Go to `https://github.com`

(b) Log in and click the + icon in the top-right corner

(c) Choose `New repository`

(d) Enter a repository name (e.g., `c-calculator`)

(e) **Do not initialize** with a README or license (since you already have local files)

(f) Click `Create repository`

Step 2: Connect Local to Remote

In your terminal, run the following command to add a remote named `origin`:

```
git remote add origin https://github.com/your-username/c-calculator.git
```

- Replace `your-username` with your GitHub username.
- Replace `c-calculator.git` with your repository name.

What does this do? This command tells Git, "Hey, I want to link my local repo to this online repo." The name `origin` is just a convention for the default remote.

Check the Connection

You can verify that your remote is set using:

```
git remote -v
```

This should show something like:

```
origin  https://github.com/your-username/c-calculator.git (fetch)
origin  https://github.com/your-username/c-calculator.git (push)
```

Fixing Mistakes (Optional)

If you accidentally entered the wrong URL or want to change the remote, run:

```
git remote remove origin
git remote add origin https://github.com/your-username/c-calculator.git
```

Why This Step Is Important

Linking to a remote repository allows you to:

- Push your code to GitHub for others to see
- Pull updates made by collaborators
- Back up your work online
- Contribute to group projects and open source

Once the remote is set, you're ready to push your code using `git push`.

4.7.8 Push Local Code to GitHub

After committing your changes locally and linking your repository to a remote on GitHub, the next step is to **push** your code online. This makes your project available on GitHub and allows others (like teammates or instructors) to view or collaborate on your work.

Syntax

For the first push:

```
git push -u origin main
```

For subsequent pushes:

```
git push
```

- `origin` refers to the remote repository name (set earlier).
- `main` refers to the branch you're pushing (the default main branch).
- The `-u` (or `--set-upstream`) option tells Git to remember this remote and branch so you can simply type `git push` next time.

Example

Continuing from the C project example:

```
git push -u origin main
```

If successful, Git will upload all committed changes to your GitHub repository. You'll see output confirming the push and the URL to your project, like:

```
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Writing objects: 100% (7/7), 2.13 KiB | 2.13 MiB/s, done.
To https://github.com/your-username/c-calculator.git
 * [new branch]      main -> main
```

First-Time Authentication (Important)

The first time you push to GitHub, it may prompt for authentication. GitHub no longer allows password-based login from the command line. Instead, use one of the following:

- **Personal Access Token (PAT)** – A secure key you generate on GitHub.
- **SSH key** – A more advanced, key-based authentication method.

Recommended for beginners: Personal Access Token

(a) Go to `https://github.com/settings/tokens`

(b) Click `Generate new token`

(c) Choose scopes like `repo`

(d) Copy the token and use it as your password when Git asks

Tip: Use a Git GUI like GitHub Desktop to avoid typing tokens manually.

How to Check if Push Worked

Go to your GitHub repository page (e.g., `https://github.com/your-username/c-calculator`) — you should see your files and commit message.

Common Push Errors and Fixes

- **Error: `failed to push some refs`** – This happens when your local branch is behind the remote. For example, if two group members are working on a single branch and one of the student has pushed something on the branch whereas the other has not downloaded (pull, we will see it shortly) the chages yet. In this case, we need to push the changes again but before that, we must resolve the conflicts i.e. accept the changes from commit.
- **Error: authentication failed** – Double-check your personal access token or SSH setup.
- **Branch mismatch** – If GitHub created a `main` branch but your local is still `master`, rename it:

```
    git branch -M main

```

Summary

Pushing your code uploads your latest commits to GitHub. You must first add and commit locally, then use `git push` to share changes.

4.7.9 Fetch Changes from Remote

When working collaboratively, other team members might push updates to the remote repository on GitHub. To get these updates without immediately applying them to your current work, you use the `git fetch` command.

What Does `git fetch` Do?

- Downloads new commits, branches, and tags from the remote repository
- Updates your local copy of the remote branches (e.g., `origin/main`)
- Does **not** change your current working files or branches
- Lets you review incoming changes before deciding to integrate them

Usage

```
git fetch origin
```

Here, `origin` is the name of the remote repository (default name for GitHub repos).

Example

Suppose your teammate added a new feature to the `main` branch on GitHub. To update your local repository with their changes, run:

```
git fetch origin
```

This command downloads their changes and updates your local tracking branch `origin/main`.

Check What Changed

To compare your local branch with the remote-tracking branch, use:

```
git diff main origin/main
```

This shows what files and lines differ between your current work and the remote version.

4.7.10 Pull Changes from Remote

While `git fetch` only downloads updates, `git pull` downloads **and** immediately merges those changes into your current branch. It is a convenient way to update your work with the latest changes from the remote repository.

What Does `git pull` Do?

- Runs `git fetch` to get the latest commits
- Automatically merges the fetched changes into your current branch
- Updates your working directory with the new code

Usage

```
git pull origin main
```

This fetches changes from the remote `main` branch and merges them into your local `main` branch.

Example

Continuing with the C project, if your teammate pushed bug fixes to the `main` branch on GitHub, you can update your local code and incorporate those fixes by running:

```
git pull origin main
```

Git will automatically merge the changes. If there are conflicts, Git will notify you, and you'll need to resolve them manually.

When to Use `git fetch` vs `git pull`

- Use `git fetch` if you want to review incoming changes before merging.
- Use `git pull` if you want to quickly update your local branch and working files.

Summary

When fetching from remote, merge conflicts can arrive which could be tricky for beginners to fix. However, `git fetch` and `git pull` do roughly the same task but are designed in a way to help the user to avoid the merge conflicts. `git fetch` is a safe way and gives you on overview of the scale of changes and conflicts before you decide to merge them locally using `git fetch`. In contrast, `git pull` is a faster way of merging the changes from remote branch in to you local branch but it could end in merge conflict which needs to be fixed manually. `git pull` tries to resolve maximum conflicts automatically and is a powerful tool, however as a beginner, it is suggested to used `git fetch`. Once you get an idea of conflicts, migrate to `git pull` gradually.

Command	Action	Effect on Local Repo
`git fetch`	Download updates from remote	Updates remote-tracking branches, does NOT change current files or branches
`git pull`	Download updates AND merge into current branch	Updates remote-tracking branches and merges changes into your working files

Using these commands effectively helps you keep your local repository in sync with the shared project on GitHub.

4.7.11 Checking Differences or Changes made in the Files

During development, it is important to see what changes you've made to your code. The `git diff` command lets you view the exact line-by-line differences between various versions of your files.

What Does `git diff` Show?

- Which lines were added, removed, or modified
- Changes not yet staged (by default)
- Differences between branches or commits
- Changes between local and remote repositories

Common Usages

(a) **View unstaged changes (working directory vs staging area):**

```
    git diff

```

(b) **View staged changes (staging area vs last commit):**

```
    git diff --cached

```

(c) **Compare local branch to remote branch:**

```
    git diff main origin/main

```

Example: C Project Context

Imagine you edited the following function in `math.c`:

```c
// Old version
int add(int a, int b) {
    return a + b;
}

// New version
```

```
int add(int a, int b) {
    printf("Adding␣%d␣and␣%d\n", a, b);
    return a + b;
}
```

Before committing the change, you can inspect it using:

```
git diff math.c
```

You'll see output like:

```
diff --git a/math.c b/math.c
index e69de29..a1b2c3d 100644
--- a/math.c
+++ b/math.c
@@ -1,3 +1,4 @@
+    printf("Adding %d and %d\n", a, b);
     return a + b;
```

This shows the line you added. Lines starting with '+' are additions, and lines starting with '-' are deletions.

Tips

- Use `git diff <filename>` to focus on one file
- Run `git diff --stat` for a summary of which files changed
- Use `git difftool` to open a visual comparison (if configured)

Why Use `git diff`?

- To review changes before committing
- To inspect changes pulled from teammates
- To debug or find where things went wrong

4.7.12 Create and Switch Branches

In Git, branches are like parallel versions of your project. You can use them to develop features, fix bugs, or try out experiments without affecting the main code. Once a branch is ready, you can merge it back into the main branch.

Why Use Branches?

- Keep feature development isolated from stable code
- Let team members work independently
- Avoid breaking the main project while testing a new feature
- Have a separate branch for debugging and testing purposes

Create and Switch to a New Branch

With newer versions of Git, the recommended command to create and switch to a branch is:

```
git switch -c feature-name
```

`-c` stands for "create." This creates the branch and switches to it in one step.

Example: C Project Feature Branch

Suppose you are adding a new scientific mode to your calculator project. You can create and switch to a new branch like this:

```
git switch -c scientific-mode
```

This does two things:

(a) Creates a new branch named `scientific-mode`

(b) Moves your working directory to that branch

Now, any changes you make will be isolated to this branch.

Switch Between Branches

To switch to an existing branch (for example, back to `main`), run:

```
git switch main
```

List All Branches

To see all local branches and check which one you're currently on:

```
git branch -v
```

The current branch will be marked with an asterisk (*).

Summary of Branch Commands

- `git switch -c new-branch` – Create and switch to a new branch
- `git switch existing-branch` – Switch to an existing branch
- `git branch` – List local branches

Branches allow multiple developers to work in parallel without interfering with each other. Once development is complete, you can merge the branch into `main` (covered next).

4.7.13 Merging Branches

After completing work in a feature branch, the next step is to integrate those changes back into the `main` branch (or another base branch). This is done using the `git merge` command.

What Does `git merge` Do?

- Combines the changes from one branch into another
- Preserves the full commit history
- Creates a merge commit if the branches have diverged

Steps to Merge a Feature Branch into Main

Let's say you've been working on a branch called `scientific-mode`, and now it's time to merge it into `main`.

Step 1: Switch to the branch you want to merge into

```
git switch main
```

Step 2: Merge the feature branch

```
git merge scientific-mode
```

Git will attempt to merge the changes automatically. If the two branches changed different parts of the project, Git will combine the changes and create a new merge commit.

Example: Merging a C Feature Branch

Suppose you implemented scientific calculator functions in `scientific-mode`. To include them in your main project:

```
git switch main
git merge scientific-mode
```

Git may respond with:

```
Updating abc123..def456
Fast-forward
 math.c | 20 ++++++++++++++++++++
 1 file changed, 20 insertions(+)
```

This means the merge was successful.

Handling Merge Conflicts (Expanded)

A **merge conflict** occurs when both branches have made changes to the same part of a file, and Git cannot decide which version to keep.

Example Conflict in `math.c`:

Imagine both branches changed the 'add()' function in different ways.

`main` branch:

```
int add(int a, int b) {
    return a + b;
}
```

`scientific-mode` branch:

```
int add(int a, int b) {
    printf("Adding␣%d␣and␣%d\n", a, b);
    return a + b;
}
```

When merging, Git will show a conflict like this in `math.c`:

```
int add(int a, int b) {
<<<<<<< HEAD
    return a + b;
=======
    printf("Adding %d and %d\n", a, b);
    return a + b;
>>>>>>> scientific-mode
}
```

The lines between `<<<<<<< HEAD` and `=======` come from your current branch (usually `main`). The lines between `=======` and `>>>>>>> branch-name` come from the branch you are merging (`scientific-mode` in this case).

How to Resolve the Conflict:

(a) Open the conflicted file(s) in a code editor.

(b) Manually edit the file to keep the desired version. For example:

```
// Final resolved version
int add(int a, int b) {
    printf("Adding %d and %d\n", a, b);
    return a + b;
}
```

(c) Remove all the conflict markers (<<<<<<<, =======, >>>>>>>).

(d) Save the file.

(e) Stage the resolved file:

```
git add math.c
```

(f) Finalize the merge with a commit:

```
git commit
```

Git will open a default merge commit message. You can leave it as is or write your own message describing the merge resolution.

Best Practices for Avoiding Conflicts

- Pull or fetch the latest changes from the remote repository before starting new work:

```
git pull origin main
```

- Communicate with teammates about which files each person is editing.
- Break large changes into smaller, isolated commits and branches.
- Review changes using `git diff` before merging.

Summary

- `git switch main` – Move to the branch you want to merge into.
- `git merge branch-name` – Combine the changes.
- If conflicts occur, manually edit the files and complete the merge with `git add` and `git commit`.

Merge conflicts are a normal part of collaboration. With practice, resolving them becomes straightforward and helps maintain clean, reliable code.

4.7.14 Optional: Remove a Tracked File

Sometimes you may want to remove a file from your Git repository — for example, if it was accidentally added, or if it's no longer needed in the project. Git provides commands to remove tracked files cleanly.

Remove and Delete the File from Disk

The simplest way to remove a file from both Git and your working directory is:

```
git rm filename
```

This will:

- Delete the file from your working directory
- Stage the deletion for the next commit

Example:

If you want to remove a temporary test file named `temp.c`, run:

```
git rm temp.c
git commit -m "Remove unused temp.c file"
```

Remove from Git but Keep the File on Disk

If you want to stop tracking a file (for example, a log or binary file), but keep the file itself in your local folder:

```
git rm --cached filename
```

Example:

To stop tracking a build output file named `output.log`, but leave it on disk:

```
git rm --cached output.log
git commit -m "Stop tracking output.log"
```

You can then add the file to your `.gitignore` to prevent Git from tracking it again in the future.

Common Use Cases

- Accidentally committed large or private files
- Remove auto-generated files (e.g., compiled binaries)
- Clean up outdated test or debug files

Summary

- `git rm file` — removes the file from Git and deletes it from disk
- `git rm --cached file` — removes the file from Git but keeps it locally
- Always commit the change after using `git rm`

This command helps keep your repository clean and focused on the files that matter.

Recommended Git Workflow

- `git status` – Check the status of changes.
- `git add <file>` – Stage specific files.
- `git commit -m "message"` – Save a snapshot.
- `git fetch/pull` – Sync with the latest version from GitHub.
- `git push` – Upload your commits to GitHub.

4.7.15 Summary of Essential Git Commands

The table below summarizes the key Git commands covered in this guide, arranged in the typical order of usage when working on a project. These commands form the foundation for effective version control and collaboration.

Command	Description
`git init`	Initialize a new Git repository in the current directory
`git status`	Show the current state of the working directory and staging area
`git add <file>`	Stage a file to be included in the next commit
`git rm <file>`	Remove a tracked file and delete it from disk
`git rm --cached <file>`	Stop tracking a file without deleting it locally
`git commit -m "message"`	Record the staged changes in the repository with a message
`git remote add origin <URL>`	Link a local repository to a remote GitHub repository
`git push -u origin main`	Push local commits to the remote `main` branch
`git fetch`	Download new data (commits, branches) from the remote repository
`git pull`	Fetch and automatically merge changes from the remote
`git diff`	Show differences between files (unstaged by default)
`git diff --cached`	Show staged changes ready to be committed
`git switch -c <branch>`	Create and switch to a new branch
`git switch <branch>`	Switch to an existing branch
`git branch`	List all local branches
`git merge <branch>`	Merge another branch into the current branch
`git log`	Show a history of commits

This table can be printed and kept as a handy reference while working on collaborative coding assignments or projects.

Further Learning Resources

- Interactive Git tutorial: `https://learngitbranching.js.org`
- GitHub's beginner guide: `https://docs.github.com/en/get-started/quickstart`
- Video Tutorial: `https://www.youtube.com/watch?v=RGOj5yH7evk`

Project

Project Description: Self-Balancing Robot

The goal of this project is to design and build a two-wheeled self-balancing robot using the **STM32F3 Discovery Board** and the **Keyestudio Self-Balancing Robot Kit**.

The robot will rely on an Inertial Measurement Unit (IMU), usually combining an accelerometer and a gyroscope, to estimate its tilt angle. A feedback control algorithm, usually a PID controller, will run on the STM32F3 to continuously adjust motor speeds and maintain balance.

Hardware Components

- STM32F3 Discovery Board (STM32F303VCT6)
- Keyestudio Self-Balancing Robot Kit
- Motor driver (via Keyestudio balancing shield)
- IMU sensor (MPU6050 or the onboard sensor on STM32F3 Discovery kit)
- Battery pack (Li-ion) or power supply via long wires

Key Objectives

- Use IMU data to estimate the robot's tilt angle as feedback for control.
- Implement and tune a position PID controller on the STM32F3 for balance
- Control motor speeds using PWM signals based on the controller output
- Achieve stable performance under small disturbances or external forces
- Demonstrate creativity through meaningful extensions or improvements beyond the minimum functional requirements.

Controller Requirements and Guidelines

A position-based PID controller is a mandatory requirement for this project. Students may also choose to explore a speed-based PID controller using motor encoder feedback as input; however, implementation of the position-based controller must still be demonstrated.

Students are encouraged to consult online resources, reference materials, and other external sources (Keyestudio wiki website) to support their design and implementation. Any such sources must be clearly documented and acknowledged during the final project viva.

All design decisions, such as the choice of control strategy, parameter tuning approach, and hardware configurations, must be well justified and clearly explained during the viva. The ability to defend and explain these decisions is an important part of the evaluation process.

5.1 Project Completion 1 and 2

Project Completion Weeks 1 and 2 are allocated for focused, self-directed development of the final project. These sessions are unsupervised, allowing teams uninterrupted time to test, debug, and refine their systems at their own pace.

Although no formal instruction is scheduled during these weeks, students are expected to make substantial progress toward project completion. TAs or RAs may be available on request for occasional support.

This time should be used effectively to integrate all system components, verify functionality on real hardware, and resolve any outstanding technical issues. Progress during these sessions will directly influence the final project evaluation.

5.2 Project Evaluation

Evaluation Criteria	Description	%	CEP/ OEL	CLO
Core Functionality: Self-Balancing	Robot must stay balanced using IMU feedback and PID control, and show recovery from minor disturbances.	30	*WP1*	1
Subsystem Integration and Modular Code	Proper integration of IMU, motor drivers (PWM), control logic, and communication. Code must be modular, readable, and well-structured with clear device drivers (.c/.h) for motor control, angle estimation, PID, and main logic.	15	*WP4*	2
Real-time Plotting and Debugging	Demonstrates a systematic debugging process using UART logs, real-time sensor plots, oscilloscope (if needed), and terminal tools. Sensor plots should be smooth and responsive.	15	*WP1*	1
Version Control	Effective use of Git for collaboration. Clear comments, function headers, and a well-written README. Frequent, meaningful commits.	10	–	4
Creativity and Extensions	Adds creative or innovative features beyond the basic requirements.	10	*WP5*	1
System Architecture, Block Diagram & Flowcharts	Includes clear hardware architecture (module-level diagrams and interfaces) and firmware flowcharts.	10	*WP7*	3
Individual Contribution	Active participation, shared responsibilities, good communication, and evidence of balanced workload within the team.	10	–	5

Lab 1: Getting Started with Git and VS Code for Version Control

1.1 Objective

Familiarize students with:

- Git, GitHub, and version control.
- VS Code installation and configuration.
- STM32CubeMX and CubeCLT setup.
- Writing and debugging a basic `Hello World` program on an STM32 microcontroller.

1.2 Getting Started

In this lab, you will set up your development environment to program STM32 microcontrollers. You will start by installing Ubuntu (unless you are using lab PCs where it's already installed), then install and configure essential software tools such as Visual Studio Code and Git. Next, you will download and set up STM32CubeMX and STM32CubeCLT for project generation and toolchain integration. Finally, you will create, build, and flash a simple "Hello World" program that transmits data over UART.

Note: If you are working on lab PCs, you can skip the Ubuntu installation step as it has already been completed for you.

1.3 Installing and Setting Up Ubuntu

You may install Ubuntu in three ways depending on your setup:

1.3.1 As Primary OS (Erase Windows)

(a) Download ISO: ubuntu.com/download

(b) Create bootable USB using Rufus (Select GPT, UEFI).

(c) Shrink a Windows partition via Disk Management (25 GB minimum).

(d) Disable Secure Boot and Intel RST from BIOS.

(e) Boot into USB and choose "Erase disk and install Ubuntu".

1.3.2 As Secondary OS (Dual Boot)

(a) Same ISO and Rufus setup.

(b) Shrink existing partition.

(c) Disable Secure Boot and Intel RST.

(d) During install, choose "Install Ubuntu alongside Windows Boot Manager".

1.3.3 Using VirtualBox (Temporary Use)

(a) Install VirtualBox.

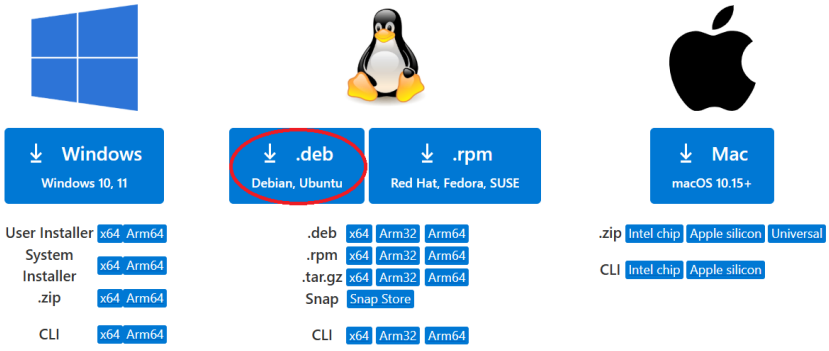
(b) Create a VM with 2–4 GB RAM and 20+ GB disk.

(c) Attach Ubuntu ISO and install.

1.4 Installing VS Code and Git

1.4.1 VS Code

(a) Download from code.visualstudio.com



(b) The downloaded file will appear in your Downloads folder. You can extract its contents and proceed with the setup.

(c) Open the terminal on PC and change the directory to the path your downloaded file is in. You can change the path using the command 'cd path'.

(d) Run the command

```
sudo snap install code --classic
```

(e) Install with default settings.

1.4.2 Installing Git

To install git, run the command

```
sudo apt install git -y
```

1.4.3 GitHub CLI Setup

- Open the terminal and type in the command:

```
sudo apt install gh
```

- When prompted, enter your password and press **Enter**.
- After installation finishes, type:

```
gh auth login
```

- When the terminal prompts you to choose the authentication method:
  - Use arrow keys to select **GitHub.com** and press **Enter**.
  - Select the preferred login method (usually **Web browser**) and press **Enter**.
- A URL and a code will be displayed in the terminal.
  - Open the provided URL in your web browser.
  - You will see a GitHub login page asking for the code.
  - Enter the code shown in the terminal and click the **Continue** or **Authorize** button.
  - After authorization, return to the terminal.
- Back in the terminal, you should see a success message.
- Next, create a new repository and clone it locally by typing:

```
gh repo create my-project --public --clone
cd my-project
```

- Press **Enter** after each command.
- To verify, you can list the files by typing:

```
ls
```

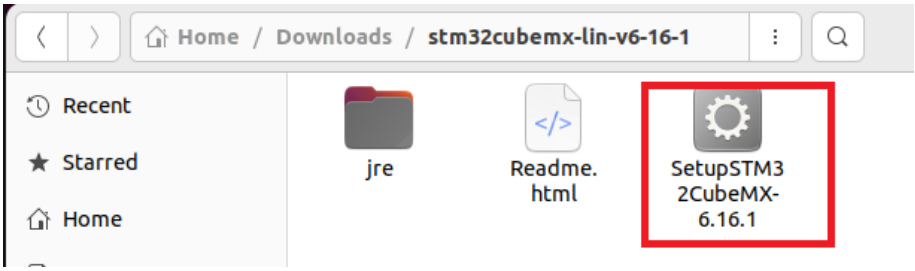
- You should now be inside your new GitHub repository folder, ready to start your project.

1.5 STM32CubeMX and ST-Link Drivers Download

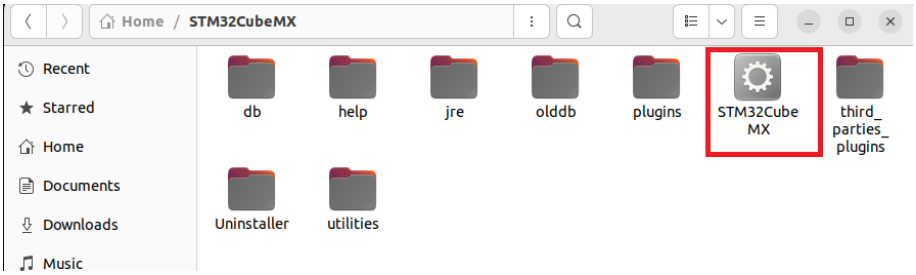
1.5.1 3.1 Download STM32CubeMX

- Download STM32CubeMX (Graphical Configuration Tool) from: `https://www.st.com/en/development-tools/stm32cubemx.html`

- The file will appear in your *Downloads* folder. Extract the file in a folder of your choice e.g. inside *home* directory. STM32CubeMX is an executable file and could be run using software installer or double clicking it.



- Once the installation is done, a folder should be created in the destination. Copy the path to the 'SetuPSTM32CubeMX-6.16.1' file. We will need this path later.



1.5.2 Install STLink Drivers

(a) Open the terminal and run the following commands to install the required drivers for stlink.

```
sudo apt install stlink-tools
sudo udevadm control --reload-rules
```

(b) When prompted, enter your password and press enter. Allow the installation to complete. These packages are necessary for flashing STM32 projects.

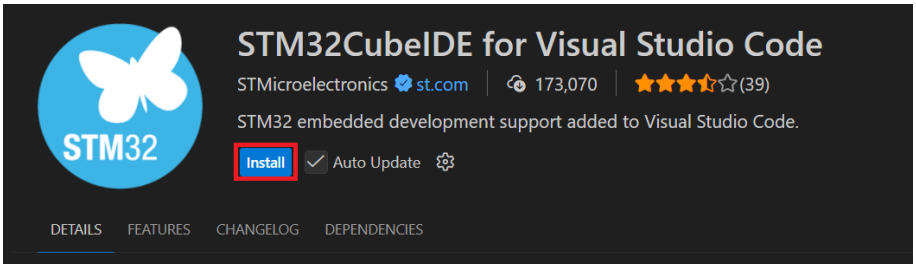
1.5.3 VS Code Extension

Make sure you have already installed `STM32CubeMX` rom the official STMicroelectronics website.

(a) Open VS Code and go to the Extensions view (you can press **Ctrl+Shift+X**).

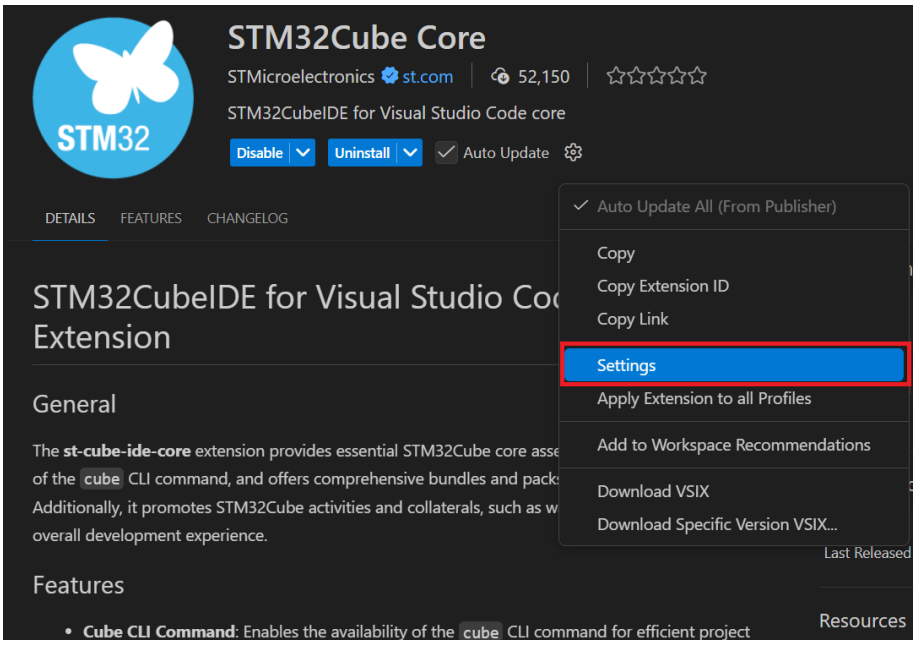
(b) Search for and install the following extensions:

- `STM32CubeIDE for Visual Studio Code`

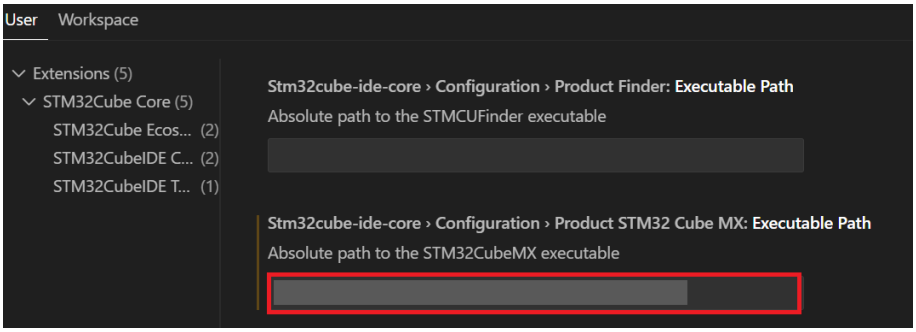


Note: Uncheck the 'Auto Update' option

(c) After installation, search up the extension STM32Cube Core and go to its settings.



(d) Under the STM32 extension configuration, set the path to your mx file.



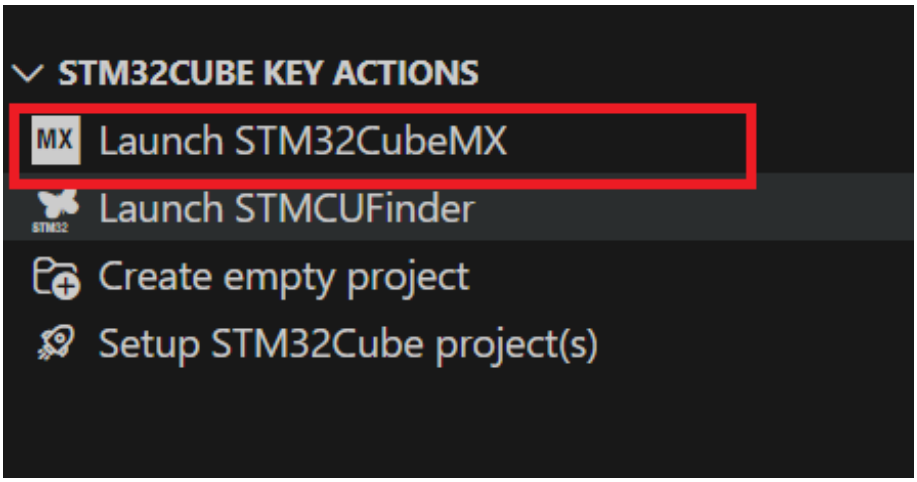
(e) Close the settings. VS Code is now ready to generate, build, and flash STM32 projects using the Cube tools.

1.6 Task 1 - Hello World Program

1.6.1 Launching STM32CubeMX from VS Code

To create your first STM32 project:

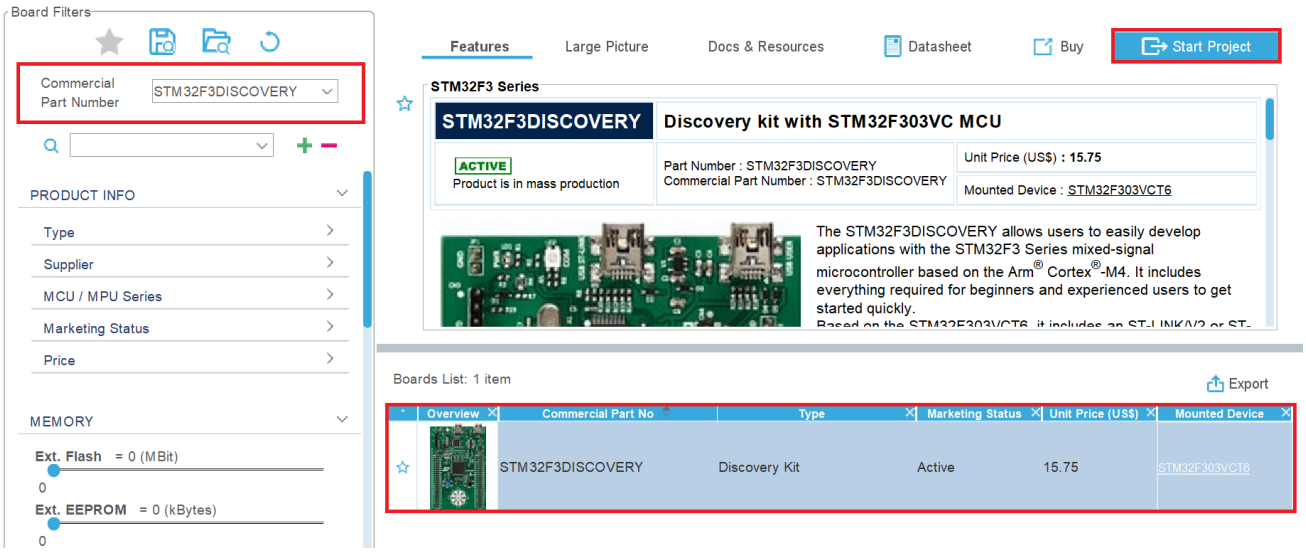
- Open VS Code.
- On the left-hand sidebar, click on the butterfly icon (**STM32 Extension Icon**).
- From the panel that appears, click on **Launch STM32CubeMX**.



- This will open the STM32CubeMX interface for project configuration.

1.6.2 STM32CubeMX Configuration

(a) Click "Access to Board Selector" and select **STM32F3 Discovery** board.



(b) In the dialog box for initializing all peripherals in their default mode, select Yes.

(c) Enable **USART1**: **Connectivity → USART1 → Asynchronous**.

(d) Set baud rate: **115200**.

(e) Enable Virtual COM Port: **Middleware and Software Packs → USB_DEVICE → Class for FS IP: Communication Device Class (Virtual Port COM)**.

1.6.3 Generate and Import Project

(a) Go to "Project Manager" tab.

(b) Choose project name and set toolchain to **CMake**.

(c) Click "Generate Code".

(d) Open the folder in VS Code and trust the author.

(e) If prompted to install bundle packages, click yes.

1.6.4 Write Hello World via UART

In `main.c`, scroll to:

```
/* USER CODE BEGIN 3 */
while (1)
{
    ...
}
```

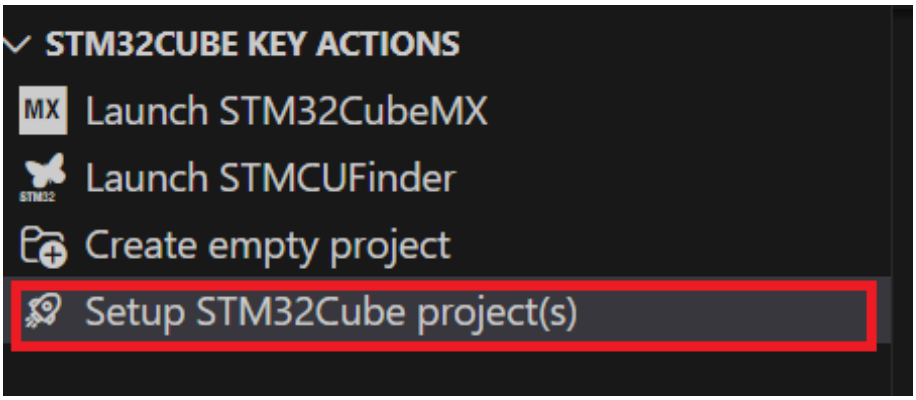
Add UART transmit code. Refer to the HAL documentation uploaded on LMS.

Hints:

- Use `HAL_UART_Transmit()`.
- Include appropriate C string headers.
- Ensure you're using the right handle: likely `&huart1`.

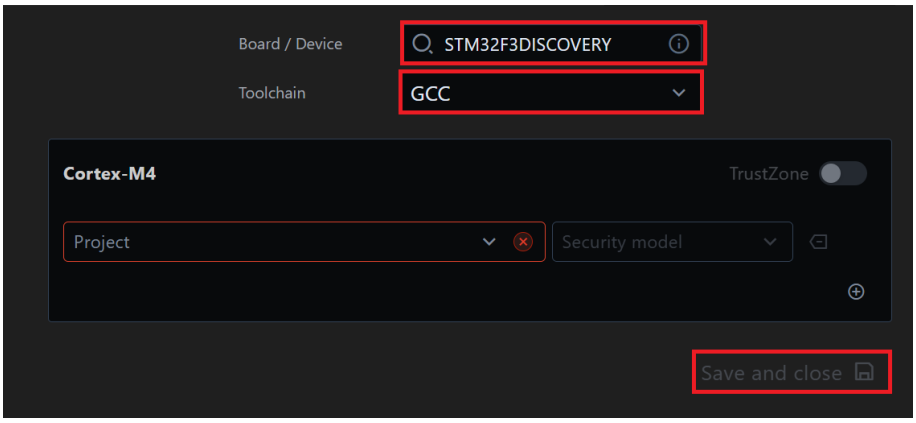
1.6.5 Flash and Test

- Connect your board to the computer.
- From the STM32Cube Panel, Go to 'Select STM32Cube Project(s)'.



.

- Set the Board/Device as 'STM32F3Discovery' and set the Toolchain as 'GCC'. Your Folder Name should show under Cortex-M4 otherwise you can select it from the dropdown. Click on 'Save and Close.'



.

- Build the project: **Ctrl + Shift + B** or from the toolbar at the bottom of the vs code window. Select 'Debug' if prompted.
- From the left panel, click on the Run and Debug option and press 'Run and Debug'. Ensure the debugger selected is **"STM32Cube: STM32 Launch ST-Link GDB Server".**
- Once the code has flashed, the following options will appear

.

- Click on the continue icon or press F5 to run the entire code. Use the stop icon to end the instance created.
- Open terminal and enter the following commands

```
sudo apt install screen
sudo screen /dev/ttyACM0 115200
```

- Press Reset on the STM32 board and observe the message on the terminal.

All code must be pushed to your GitHub repository and pulled in front of the RA for result verification. Failure to push the code will result in a penalty.

1.7 Evaluation Questions

(a) In the Hello World task, why is HAL_UART_Transmit() placed inside the while(1) loop, and what could go wrong if it's placed outside?
(b) You used the command screen /dev/ttyACM0 115200 in this lab. What is the purpose of this command, and what would happen if the baud rate specified does not match the one configured in STM32CubeMX?

1.8 Assessment Rubric

CLO	Task Component	LR2 – Code	LR5 – Results	LR9 – Report	LR7 – Viva
1	Task 1	/40	/20	/10	–
	Evaluation Questions	–	–	–	/10
4	Git	/20			
Total	**/100**				

Lab 2: Getting Started with STM32 Microcontroller: Implementing UART and Programming in C

Objective

This lab introduces you to the fundamental architecture and programming workflow of the STM32F3 microcontroller, using UART as a communication and debugging tool.

2.1 Overview of Microcontroller Components

2.1.1 Core Components

CPU (Central Processing Unit): The CPU is the brain of the microcontroller. It processes data, performs calculations, and makes logical decisions. It follows a basic cycle: fetch, decode, and execute instructions.

Memory: Microcontrollers typically include two types of memory:

- **Program Memory (Flash/ROM):** A non-volatile memory that stores the program code. It retains data even when power is turned off.
- **Data Memory (RAM):** A volatile memory used for storing variables and temporary data during execution. This memory is cleared when power is lost.

I/O Ports: The pins you see on a microcontroller board act as input/output ports. These ports are used to interact with the external environment.

- **Input:** Receive data from sensors, buttons, or switches.
- **Output:** Send data to LEDs, motors, displays, etc.

Timers and Counters:

- **Timers:** Measure time intervals and are used in applications like delays or LED blinking.
- **Counters:** Count external events or pulses, useful in event tracking or signal processing.

Communication Interfaces: Microcontrollers often need to communicate with other devices. Common interfaces include:

- **UART (Universal Asynchronous Receiver/Transmitter):** Serial communication used for debugging or connecting to other serial devices.
- **SPI (Serial Peripheral Interface):** A high-speed interface used to connect master–slave devices such as displays or sensors.

ADC and DAC:

- **ADC (Analog-to-Digital Converter):** Converts analog signals (e.g., from a temperature sensor) into digital values for processing.
- **DAC (Digital-to-Analog Converter):** Converts digital values to analog signals, useful for audio output or waveform generation.

2.1.2 Popular Microcontrollers

- Arduino Uno/Nano
- ESP32
- STM32 (used in this lab)
- Raspberry Pi Pico

- PIC Microcontrollers

2.1.3 Applications

Microcontrollers are used across numerous industries, including:

- Consumer Electronics
- Automotive Systems
- Industrial Automation
- Healthcare Equipment
- IoT (Internet of Things) Devices

According to market analysis, the global microcontroller market was valued at USD 30.63 billion in 2021 and is projected to reach USD 51.13 billion, highlighting the growing importance of this technology.

Important Note

As you begin working with microcontrollers in this lab, keep in mind that understanding both hardware components and software programming is crucial for successful design and implementation of embedded systems.

2.2 STM32F3 Microcontroller Introduction

This lab will utilize the STM32F303 Discovery Board, which features the STM32F303VCT6 microcontroller, a member of STMicroelectronics' STM32 family based on the ARM® Cortex®-M4 core. This board provides a robust platform for developing and testing embedded systems and will be used throughout the course to explore a range of microcontroller interfacing concepts.

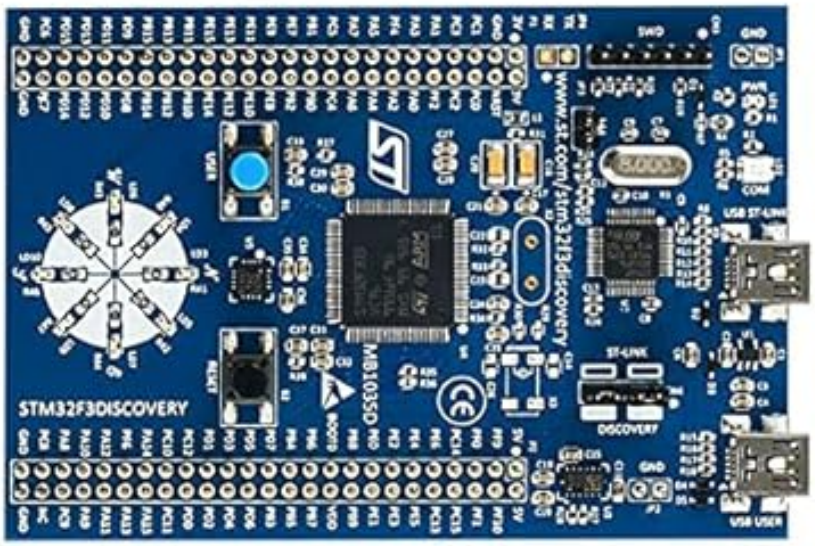


Figure 7.1: STM32F303 Discovery Board

2.2.1 Initial Power Verification

Upon connecting the STM32F303 Discovery Board to a computer via USB, two red LEDs (labeled LD1 and LD2) should illuminate. This indicates that the board is receiving power and that the on-board ST-LINK debugger is active.

2.2.2 Microcontroller Features

The STM32F303VCT6 integrates a 32 bit ARM® Cortex M4 core, operating at a clock frequency of up to 72 MHz. It includes a hardware floating point unit (FPU) and supports digital signal processing (DSP) instructions. These features enable efficient mathematical operations and signal handling capabilities, which are essential in control systems, data acquisition, and signal conditioning applications.

Feature	Details
Core	ARM® Cortex®-M4 with FPU and DSP support
Clock Frequency	72 MHz
Flash Memory	256 KB (non-volatile program storage)
SRAM	40 KB (volatile data memory)
GPIO	Up to 87 general-purpose input/output pins
Analog Interfaces	4 ADCs, 2 DACs
Timers	Advanced, general-purpose, and basic timers
Communication Interfaces	UART, SPI, I^2C, USB
On-board Debugger	ST-LINK/V2 integrated (via mini-USB)
Operating Voltage Range	2.0 V to 3.6 V (core and I/O logic levels)

Table 7.1: Key specifications of the STM32F303VCT6

2.2.3 Power Supply Overview

The STM32F303 Discovery Board supports multiple power supply options, allowing flexible use across different development and interfacing scenarios.

Power Supply Sources

- ST-LINK USB connector
- USB USER connector
- External 3V or 5V supply

Pin	Direction	Voltage	Conditions
5V	Input/Output	5V	As output: supplies peripherals (max 100mA). As input: can power the board when USB is not connected.
3V	Input/Output	3V	Same conditions as 5V. Connected peripherals must remain below 100mA.

Table 7.2: Power pins usage on the STM32F303 Discovery Board

Recommended Default Power Setup

For all lab experiments in this course, students are advised to:

- Use the ST-LINK USB connector to power the board via a PC or laptop.
- Avoid connecting power to the 5V or 3V pins unless explicitly required by an experiment and under instructor supervision.

2.2.4 On-Board Peripherals

The Discovery board includes several peripherals that will be utilized in subsequent experiments:

- User LEDs (4)
- User push-buttons (2)
- MEMS accelerometer (LIS302DL)
- Gyroscope (LSM303DLHC)
- USB interface (for power, programming, and communication)
- ST-LINK/V2 debugger interface

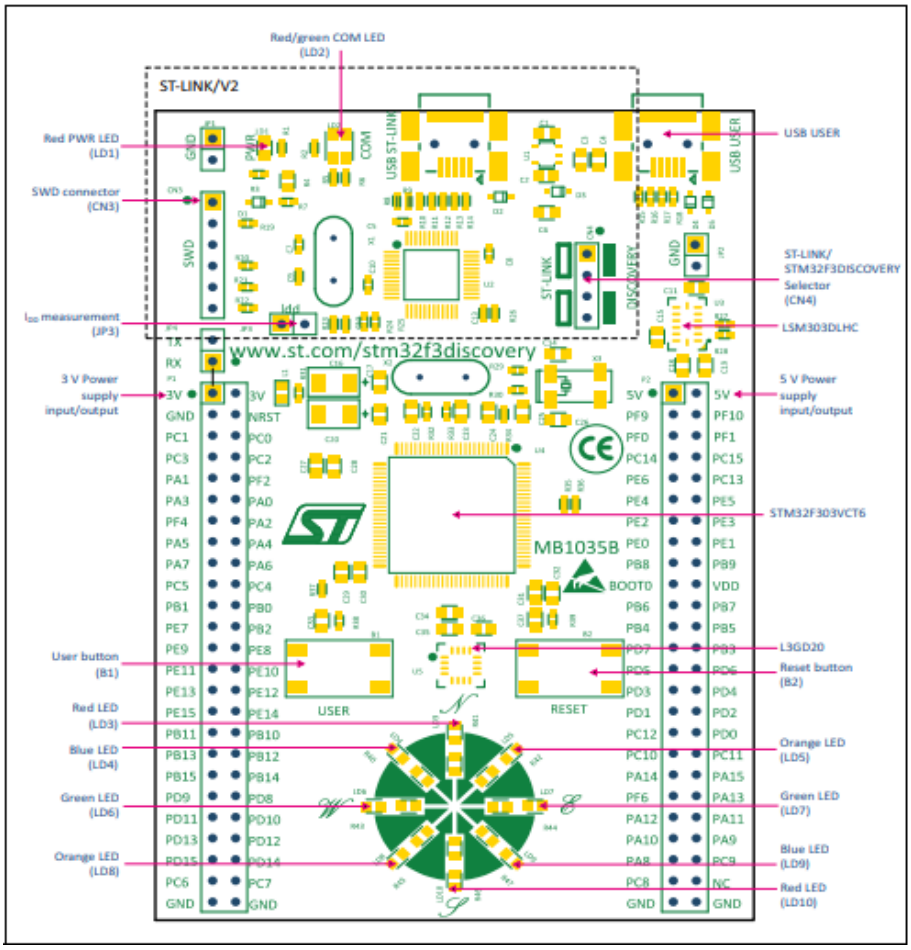


Figure 7.2: Discovery Board Layout (Top View)

2.3 STM32Cube HAL and BSP

The Board Support Package (BSP) is a set of software components provided by STMicroelectronics that allows developers to interface directly with the hardware features of a specific STM32 development board. For this course, we will be using the STM32F3 BSP, tailored for the STM32F303 Discovery Board.

The BSP works in conjunction with the Hardware Abstraction Layer (HAL) by providing higher-level, board-specific APIs that make peripheral control and application development more efficient and less error-prone.

2.3.1 Purpose of BSP

The STM32F3 BSP simplifies development by:

- Initializing on-board peripherals such as LEDs, push-buttons, and sensors
- Managing board-specific pin configurations
- Providing tested, reusable drivers for hardware components

2.3.2 Components of the STM32F3 BSP

The BSP typically includes:

- LED and Button Drivers (e.g., `BSP_LED_Init()`, `BSP_PB_GetState()`)
- Sensor Drivers (for accelerometer, gyroscope, etc.)
- Startup and system initialization code
- Configuration files that define clock sources, pin mappings, and memory layout

2.3.3 Example Usage

An example of using the BSP to toggle an on-board LED:

```c
#include "stm32f3_discovery.h"

int main(void) {
    HAL_Init();              // Initialize the HAL library
    BSP_LED_Init(LED3);      // Initialize on-board LED3
    while (1) {
        BSP_LED_Toggle(LED3); // Toggle LED3 state
        HAL_Delay(500);       // Wait for 500 milliseconds
    }
}
```

2.3.4 BSP vs HAL

HAL (Hardware Abstraction Layer):
Provides generic peripheral access functions (e.g., GPIO, UART).

BSP (Board Support Package):
Provides board-specific functions that utilize HAL internally (e.g., controlling Discovery Board's LED3 without knowing which GPIO pin it's connected to).

Useful Links

This page covers a wide range of BSP functions to control the on-board hardware: `https://documentation.help/STM32F3-Discovery-BSP/`

2.4 UART Communication Using External USB-UART Module

2.4.1 Objectives

This lab introduces you to UART-based communication using an external USB–UART module connected to the STM32F303 Discovery Board. By the end of this task, you should be able to:

- Understand the theory behind UART/USART communication.
- Set up UART wiring correctly with respect to MCU and USB modules.
- Configure USART2 in STM32CubeMX.
- Generate and import a project using STM32 HAL drivers.
- Send data from the STM32F303 MCU to a terminal emulator and verify the output.

2.4.2 Theory and Background from Lab 01

What is UART/USART?
UART (Universal Asynchronous Receiver/Transmitter) is a hardware communication protocol used to enable asynchronous serial communication between digital devices. USART (Universal Synchronous/Asynchronous Receiver/Transmitter) extends this functionality by allowing both synchronous and asynchronous modes.
In asynchronous mode (used in this lab), the transmitter and receiver must agree on the same baud rate and character format. Communication involves only two main lines: TX (transmit) and RX (receive), along with a shared ground (GND). This makes UART suitable for low-pin-count, point-to-point communication, such as between a microcontroller and a PC.

Why Use USART2 Instead of USART1?
In Lab 01, the STM32 communicated using USART1 through the onboard ST-LINK's virtual COM port. However, this method hides much of the physical-level understanding. By using USART2 and an external USB-UART converter, you:

- Gain visibility into physical UART signaling.
- Understand pin-level wiring and baud rate matching.
- Develop the flexibility to interface with other devices and modules (e.g., GPS, Bluetooth).

Figure 7.3: UART module

Asynchronous vs. Synchronous Communication
We use asynchronous mode because:

- No separate clock line is needed.
- It reduces wiring complexity.

- Standard USB–UART modules support only asynchronous mode.

Synchronous mode requires an additional clock line and is better suited for high-throughput or multi-device communication (like SPI).

Baud Rate Selection
The baud rate defines how fast data is transferred. Common rates are 9600 or 115200 bits per second.

- 9600 is stable and broadly supported.
- 115200 is faster and also widely used.

In either case, the MCU and the receiving terminal must be configured to the same baud rate.

2.4.3 Hardware Setup: USB–UART Module to STM32F303

Wiring Overview

To transmit data from the STM32F303 to your PC using an external USB–UART adapter:

USB–UART Pin	STM32F3 Pin	Function
GND	GND	Shared electrical ground
TXD	PA3	UART data to MCU (USART2_RX)
RXD	PA2	UART data from MCU (USART2_TX)

Table 7.3: USB–UART to STM32F303 wiring

Why TX to RX and RX to TX? UART requires crossover: data sent by one device's TX is received on the other's RX.

2.4.4 STM32CubeMX Configuration

(a) Open STM32CubeMX and create a new project.

- Select the STM32F3DISCOVERY microcontroller.

(b) Enable USART2 in Asynchronous Mode:

- In Pinout & Configuration, under **Connectivity**, Click on `USART2` → Mode: Asynchronous.
- Assign pins: PA2 as `USART2_TX`, PA3 as `USART2_RX`.
- Under **Connectivity**, go to USB and enable Device (FS)

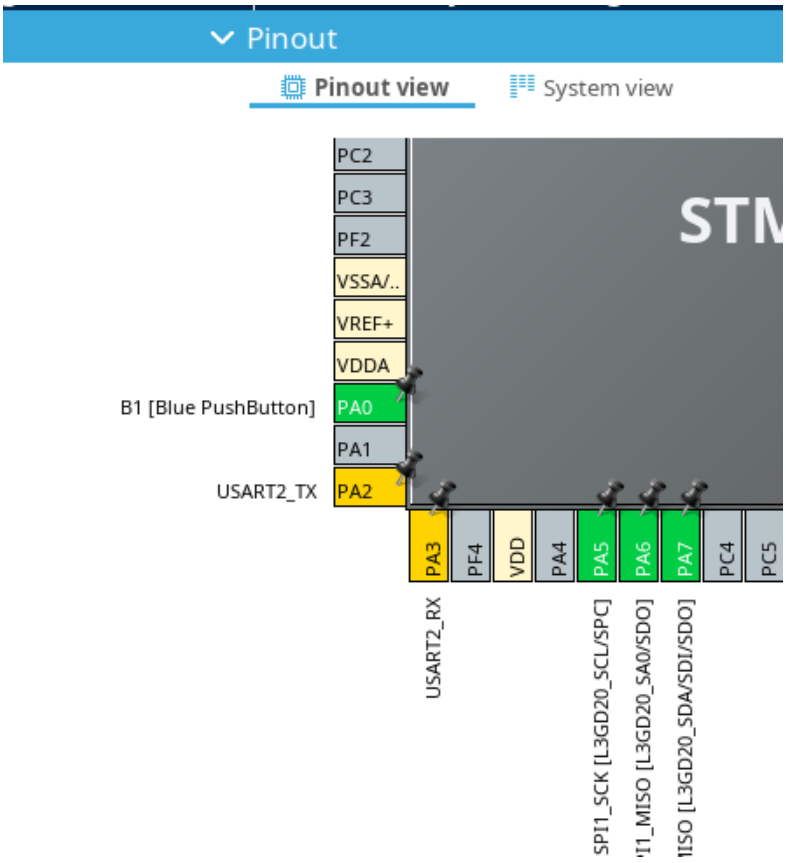


Figure 7.4: TX/RX Pins

(c) Configure Baud Rate:

- In the Configuration tab, select `USART2 Configuration`.
- Set Baud Rate to 9600 or 115200 bps.
- Word Length: 8 Bits, Stop Bits: 1, Parity: None.
- Press Enter.

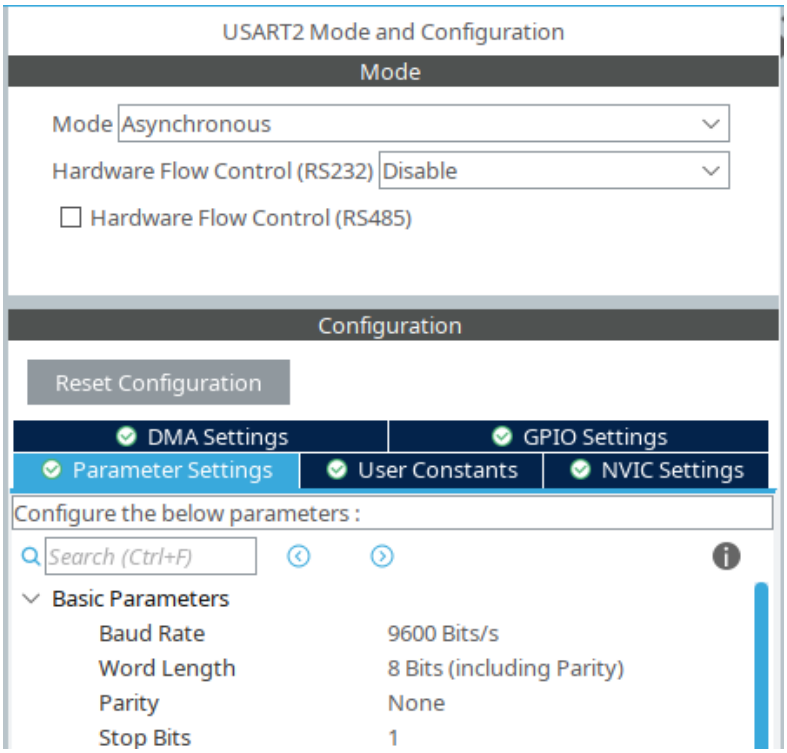


Figure 7.5: USART Paramters Configuration

(d) Disable USB CDC Class:

- Go to `Middleware` → `USB_DEVICE` and disable the CDC class.

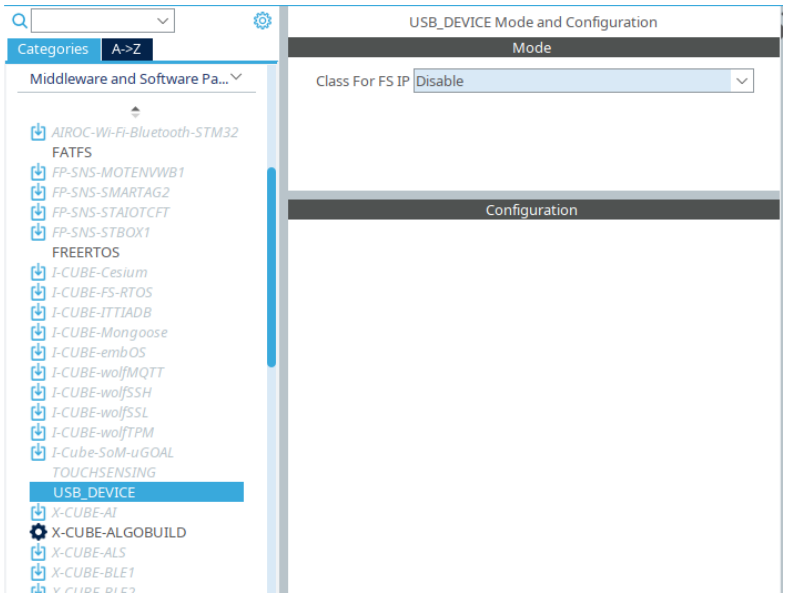


Figure 7.6: Disable the Class for FS IP in USB_DEVICE

(e) Project Settings and Code Generation:

- Name the project.
- Select Toolchain: CMakefile.
- Generate the code.

2.4.5 Common UART Functions for STM32

Function	Explanation
`HAL_UART_Init()`	Initializes the UART peripheral with specified parameters (baud rate, word length, stop bits, parity, etc.).
`HAL_UART_Transmit()`	Sends data over UART in blocking mode. This function waits until all data is transmitted.
`HAL_UART_Receive()`	Receives data over UART in blocking mode. This function waits until the expected number of bytes is received.
`HAL_UART_Transmit_IT()`	Sends data over UART using interrupt mode. The function returns immediately, and transmission is handled in the background.
`HAL_UART_Receive_IT()`	Receives data using UART interrupts. Data reception occurs asynchronously.
`HAL_UART_Transmit_DMA()`	Sends data over UART using DMA (Direct Memory Access) for efficient, non-blocking transmission.
`HAL_UART_Receive_DMA()`	Receives UART data using DMA. Useful for high-speed, continuous data reception.
`HAL_UART_IRQHandler()`	Handles UART interrupt requests. Typically called inside the actual interrupt vector defined in `stm32f3xx_it.c`.
`__HAL_UART_CLEAR_FLAG(huart, FLAG)`	Macro used to clear specific UART status flags, such as overrun (UART_FLAG_ORE), framing error (UART_FLAG_FE), etc. Requires UART handle and flag as arguments.
`__HAL_UART_ENABLE_IT()`	Enables UART interrupt sources (like RXNE, TXE).
`__HAL_UART_DISABLE_IT()`	Disables specific UART interrupts.

Table 7.4: Common UART Functions in STM32F3 HAL Library

2.4.6 Task #0

After completing the setup described above, write a simple C program to print "Hello, World!" on your terminal using the `HAL_UART_Transmit()` function. This time, the communication will occur through the external USB–UART module instead of the built-in ST-LINK virtual COM port. Ensure USART2 is properly initialized in your `main()` function and the baud rate on your terminal matches the one configured in CubeMX.

In Lab 1, we used `screen /dev/ttyACM0 115200` since the board was connected as a Virtual COM Port (VCP). Here we are using a CP2102 USB–UART module, which appears as `USB0`. So run: `screen /dev/ttyUSB0 115200`. If this doesn't work, add sudo before the command: `sudo screen /dev/ttyUSB0 115200` If you set a different baud rate, replace `115200`.

After successful transmission of a basic string using the external UART module, proceed with the following programming exercises. Each task is designed to reinforce fundamental programming concepts in C, while demonstrating how to use serial communication for displaying output.

2.5 Task 1: Writing a Custom `myPrintf` Function for UART Communication

The aim of this task is to deepen your understanding of UART communication and C programming by developing a custom formatted output function, `myPrintf`, which transmits strings via UART using STM32 HAL libraries. This function should behave similarly to the standard `HAL_UART_Transmit`.

Objective: You are required to implement the following function prototype:

```
void myPrintf(const char *fmt, ...);
```

This function should:

- Accept a format string and a variable number of arguments,
- Format the input string accordingly,
- And transmit the resulting string over UART2 using `HAL_UART_Transmit`.

Implementation Guidelines:
To complete this task, you will need to:

- Create an appropriate character buffer to store the formatted output string.
- Use C variadic function macros (`va_list`, `va_start`, `va_end`) to handle the variable arguments passed to `myPrintf`.
- Format the string using `vsnprintf` to safely construct the final output.
- Transmit the result using `HAL_UART_Transmit`, casting the buffer appropriately and using the correct length.

You are provided with the following function skeleton as a starting point:

```

#include "stdarg.h"  // For variadic argument macros
#include "stdio.h"
#include "string.h"

void myPrintf(const char *fmt, ...) {
    // TODO: Step 1 - Declare and initialize a character buffer.

    // TODO: Step 2 - Initialize the variadic argument list.

    // TODO: Step 3 - Format the final string using vsnprintf.

    // TODO: Step 4 - Transmit the string over UART using HAL_UART_Transmit.
}
```

Hints and Considerations:

- Consider using a buffer size of at least 100 characters initially; this can be adjusted based on expected output.
- Understand how `vsnprintf` differs from `sprintf` and why it is preferred.
- Recall that `HAL_UART_Transmit` requires a `uint8_t*` pointer, so a cast from `char*` is necessary.

You may test your function using the following example:

```

int x = 42;
float y = 3.14;
myPrintf("Value of x = %d, y = %.2f\r\n", x, y);
```

2.6 Task 2: Verifying a Mathematical Identity

Objective: Write a program to prove the algebraic identity:

$$(a+b)^2 = a^2 + 2ab + b^2$$

Instructions:

(a) Declare two integer variables `a` and `b`.

(b) Compute the left-hand side (LHS) as `(a + b) * (a + b)`.

(c) Compute the right-hand side (RHS) as `a*a + b*b + 2*a*b`.

(d) Use `printf()` to display the values of LHS, RHS, and whether the identity holds.

2.7 Task 3: String Encryption and Decryption

Objective: Implement a basic character-wise encryption and decryption scheme.

Instructions:

```
a = 2, b = 3
LHS: 25
RHS: 25
Identity Verified: Yes
```

Figure 7.7: Expected output format for Task 1

(a) Prompt or define a static string, e.g., `char str[] = "Microcontrollers";`.

(b) Define a numerical encryption key (e.g., your student ID).

(c) Encrypt each character by shifting its ASCII value using the key:

```
ch = ch + (key % 256);
```

(d) Print the encrypted string.

(e) Decrypt it by reversing the operation:

```
ch = ch - (key % 256);
```

(f) Print the decrypted string and confirm it matches the original.

2.8 Task 4: Matrix Multiplication

Objective: Reinforce loop structures and multi-dimensional arrays in C.

Instructions:

(a) Declare two 2×2 matrices, `A` and `B`, with predefined integer values.

(b) Multiply the matrices using nested loops and store the result in matrix `C`.

(c) Print all three matrices: `A`, `B`, and the resultant `C`.

```
Matrix A:
1 2
3 4
Matrix B:
5 6
7 8
Matrix C (A*B):
19 22
43 50
```

Figure 7.8: Example output for Task 3

2.9 Task 5: Armstrong Number Finder

Objective: Identify all Armstrong numbers between 100 and 999.

Definition: A three-digit number is called an Armstrong number if the sum of the cubes of its digits is equal to the number itself. For example:

$$153 = 1^3 + 5^3 + 3^3 = 153$$

Instructions:

(a) Use a `for` loop to iterate through the range 100 to 999.

(b) For each number, extract its three digits, compute the sum of their cubes, and compare to the original number.

(c) Print each Armstrong number found.

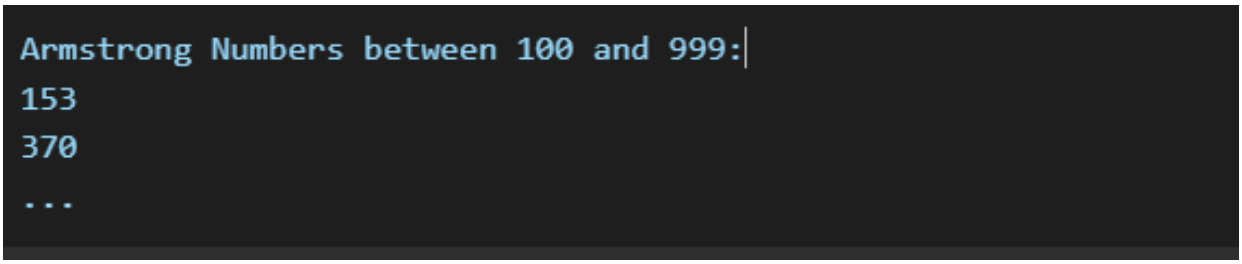


Figure 7.9: Expected output for Task 4

2.10 Tips

- Always verify TX and RX connections; UART requires crossover wiring.
- Ensure baud rate matches on STM32 and terminal emulator.
- Build the program again after any code changes.
- Reset the board after flashing the program.

2.11 Evaluation Question

(a) Explain the role of USART2 in UART communication on the STM32F3 Discovery board. Why must TX and RX lines be crossed when connecting to an external USB-UART module?

2.12 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
1	1	/5	/10	-	**/5**	–
	2	-	/5	/5		
	3	–	/10	/5		
	4	–	/10	/5		
	5	–	/10	/5		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	–
	Total	**/100**				

Lab 3: Applying GPIO and Digital I/O Control to a 7-Segment Display

Objective

This lab introduces General Purpose Input/Output (GPIO) functionality using the STM32F303 Discovery board. You will configure GPIO pins for input and output, interface with push buttons and a 7-segment display, and write programs that visually represent digits via GPIO-driven output.

3.1 Configuring GPIO Pins for Input and Output

GPIO (General Purpose Input/Output) pins are digital pins that can be configured either to receive data (input) or to send data (output) between the microcontroller and external components. These pins can be used to perform many tasks, including driving LEDs, reading buttons, or interfacing with sensors and actuators.
Each GPIO pin can be individually controlled and is software-configurable in terms of direction, logic state, and electrical characteristics. In the STM32 family, GPIO pins can also be set to alternate functions to be used by peripherals like UART, SPI, I^2C, PWM, ADC, etc.

3.1.1 Logic Levels

Digital logic levels determine what voltage corresponds to a logical HIGH (1) or LOW (0). For the STM32F303 Discovery board:

Signal	Voltage Level
Logic LOW	0 V to ~0.8 V
Logic HIGH	2.0 V to 3.6 V (typ. 3.3 V)

Table 8.1: GPIO digital logic levels

Warning: Even though the board has a 5 V power rail, the GPIO pins are not 5 V-tolerant by default unless explicitly mentioned. Applying 5 V directly to an input pin may damage the microcontroller.

3.1.2 STM32F3 GPIO Architecture

The STM32F303VCT6 microcontroller features a well-organized and scalable general-purpose I/O (GPIO) system. GPIO pins on the STM32 are grouped into logical units called ports, labeled `GPIOA`, `GPIOB`, `GPIOC`, `GPIOD`, and `GPIOE`. Each port typically consists of 16 GPIO pins, designated from `Px0` to `Px15`, where `x` represents the port letter (e.g., `PA0`, `PA1`, . . . , `PA15`).

Rationale Behind Port-Based Organization

- **Efficient Grouping of Pins:** By organizing pins into ports, the microcontroller allows developers to logically group and manage related I/O lines. This enables cleaner code and more modular hardware design.
- **Memory-Mapped Register Structure:** Each GPIO port is associated with its own block of memory-mapped registers. This structured layout simplifies the internal hardware decoding logic and allows software to access or modify pin states by reading from or writing to well-defined addresses.

- **Support for Batch Operations:** Since each port has a 16-bit wide data register, the entire port can be accessed in a single operation. This enables bulk read/write operations, which are both faster and more efficient than handling pins individually.
- **Scalability and Peripheral Mapping:** Ports are designed to interface flexibly with multiple peripheral modules (such as USART, SPI, or I^2C), and the alternate functions of many GPIO pins are mapped in a port-wise manner. This aids in flexible hardware routing and peripheral selection through the Alternate Function I/O controller.

Example: Accessing or modifying the state of GPIO pins using low-level bit manipulation is made possible by this architecture. For instance, writing 0x00FF (hexadecimal for 255) to the output data register of GPIOA sets the lower eight pins (PA0–PA7) to logic HIGH, while leaving the upper eight unchanged.

3.2 Handling Push Buttons and Seven Segment Display

3.2.1 Seven Segment Display

A seven-segment display is a widely used electronic display device for displaying decimal numerals. It consists of seven individually controlled LEDs (labeled `a` through `g`) arranged in a figure-eight pattern. An eighth LED is often included as a decimal point (DP).

Types of Seven-Segment Displays

Type	Common Terminal	Control Logic
Common Cathode	All cathodes tied together	Logic HIGH (1) turns ON segment
Common Anode	All anodes tied together	Logic LOW (0) turns ON segment

Table 8.2: Seven-segment display types

In a common cathode display, the common terminal is connected to GND, and segments are lit by sending a HIGH signal. In a common anode display, the common terminal is connected to VCC, and segments are lit by sending a LOW signal.

Connecting to STM32

(a) Verify type (common anode vs. cathode) via datasheet or multimeter.
(b) Connect each segment (`a`–`g`, and DP if needed) to a separate GPIO pin.
(c) Connect the common terminal:
  - Common cathode: COM to GND.
  - Common anode: COM to 3 V.
(d) Control segments via `HAL_GPIO_WritePin()` according to display logic.

Example

Turn on an LED connected to `PD1`:

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_1, GPIO_PIN_SET);
```

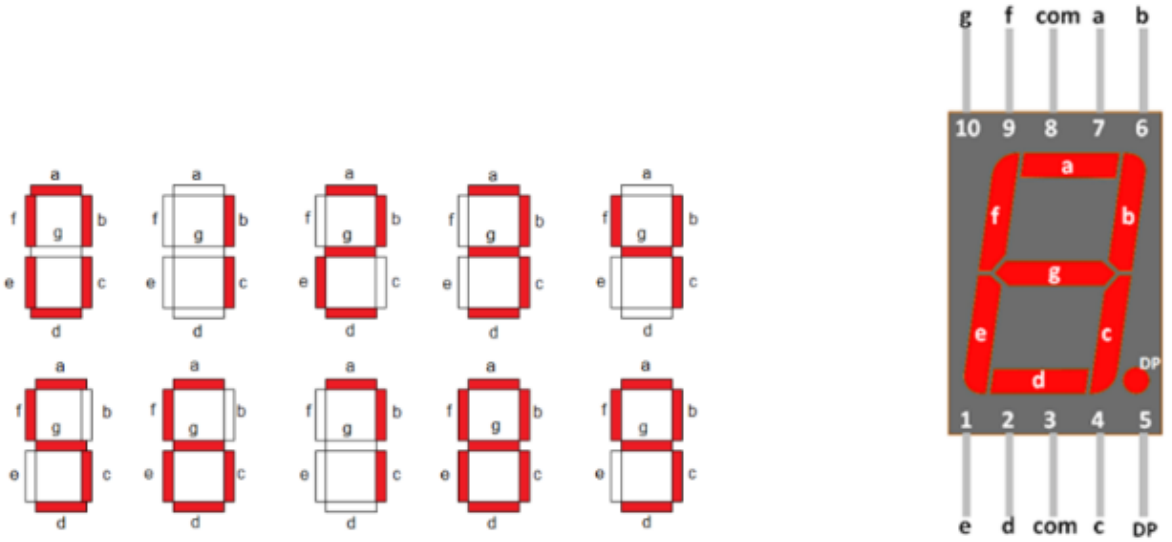


Figure 8.1: Seven Segment Pins

3.2.2 Push Buttons

Push buttons are basic digital input devices used to trigger events. When pressed, a button closes an electrical circuit, allowing current to flow.

External Push Button Configuration

A typical external push button is connected to a GPIO input pin. It is best practice to use a pull-down resistor to keep the pin at a defined LOW state when the button is not pressed.

Connection Steps

- One leg of the button to the GPIO input pin (e.g., `PD7`).
- Other leg to 3 V pin of the STM32F303VCT6.
- 10 kohm pull-down resistor between GPIO pin and GND (*or* enable internal pull-down via CubeMX).

Example Code

```
if (HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_7)) {
    // functionality when button is pressed
}
```

On-board User Button

The STM32F3 Discovery board includes a user push button (B1) connected to `PA0`. It is pre-configured in CubeMX. To detect its press, use:

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0)) {
    // functionality when B1 is pressed
}
```

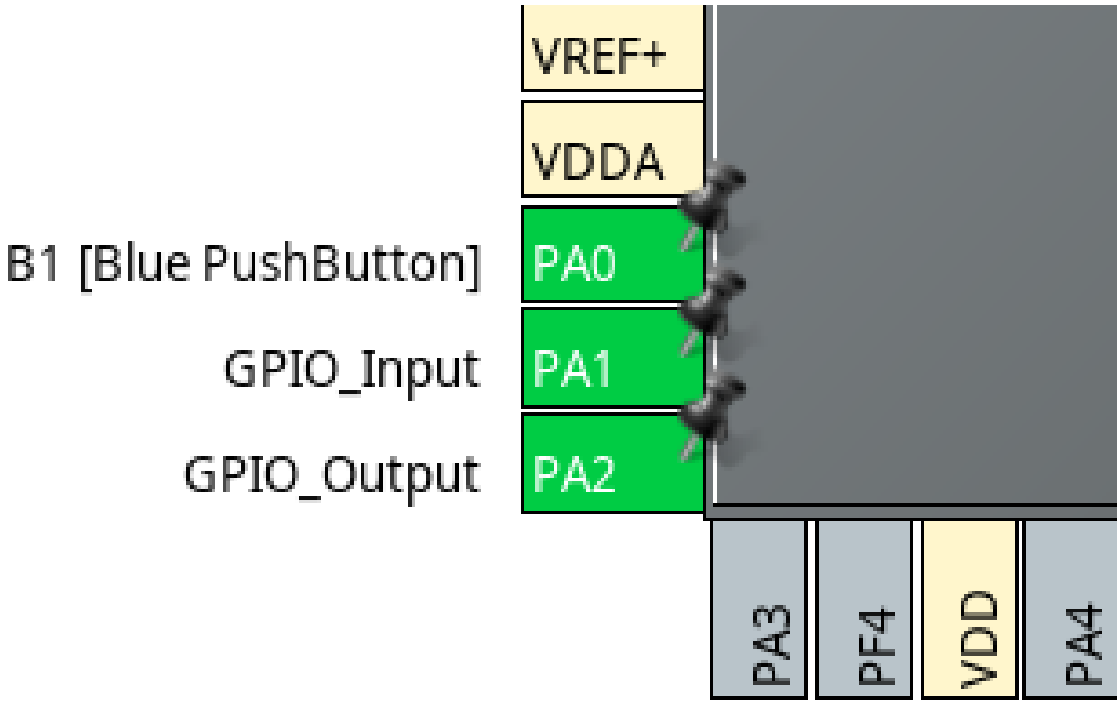


Figure 8.2: Configuring GPIO Pins in CubeMX

3.3 Task 1: Display Hex Digits 0–F on the Seven-Segment Display

Objective: Cycle through all hexadecimal digits (0 to F) on a single-digit seven-segment display, with a 2-second delay between each digit.

Instructions:

(a) Configure GPIO pins connected to the display segments as `GPIO_OUTPUT` in CubeMX.

(b) Implement a loop to display each value from 0 to 15 in sequence.

(c) Delay for 2 seconds between transitions using `HAL_Delay(2000)`.

Tip: You may optimize your code by writing a `display_number(uint8_t num)` function that maps each digit to the correct GPIO output combination.

3.4 Task 2: Display Each Digit of Student ID Using the On-board User Button

Objective: Use the USER push button (`PA0`) to display each digit of your student ID one at a time. Each press should increment to the next digit in order.

Instructions:

(a) Store your student ID in an array of integers.

(b) Detect if the USER button is pressed.

(c) Implement debouncing logic (e.g., software delay or flag-based).

(d) Reset to the first digit after the last.

3.5 Task 3: Dual Button Counter (USER Button = +1, External Button = –1)

Objective: Design and implement a counter system that increments or decrements its value based on user interaction through two push buttons:

- Onboard USER button (`PA0`) to increment the counter.
- External push button (e.g., `PB5`) to decrement the counter.

The current value of the counter is displayed on the seven-segment display.

Hardware Setup:

- USER button (`PA0`): built-in pull-down.
- External button (`PB5`): one terminal to 3V, the other to `PB5`, with a 10k pull-down to GND.

Note: The GPIO pin reads HIGH when the button is pressed (3V), and LOW when unpressed (pulled to ground).

3.6 Task 4: Random Digit Display (1 to 6) on Button Press

Objective: Each press of the USER button should display a random number between 1 and 6 on the seven-segment display.

Instructions:

(a) Use `rand()` from `stdlib.h` and limit output using modulus, e.g., `(rand() % 6) + 1`.

(b) Display the result using the same segment logic from Task 1.

3.7 GPIO Interrupts and Event-Driven Programming

A GPIO interrupt is a hardware mechanism that allows the microcontroller to respond immediately when a specific change occurs on a GPIO pin. This is done by executing a special function called an Interrupt Service Routine (ISR), instead of continuously checking the pin state using polling.

3.7.1 Advantages Over Polling

- **Reduces CPU Usage:** The microcontroller can perform other tasks instead of constantly checking input.
- **Increases Responsiveness:** Immediate reaction to events like button presses.
- **Better for Power Efficiency:** Allows entering low-power modes and waking up only on interrupts.

3.7.2 Interrupt Trigger Types

- **Rising Edge:** Triggered when signal transitions from LOW to HIGH.
- **Falling Edge:** Triggered when signal transitions from HIGH to LOW.
- **Both Edges:** Triggered on any signal change.

Example: Use Rising Edge if using a pull-down resistor with a button.

3.7.3 Configuring GPIO Interrupts in STM32CubeMX

(a) Select a GPIO pin (e.g., PA0) to act as the interrupt input.

(b) Set its mode to `GPIO_EXTI` (e.g., `EXTI0`).

(c) Choose the appropriate trigger condition (e.g., Rising Edge).

(d) Go to the NVIC tab and enable the EXTI line's interrupt.

(e) Generate the code.

3.7.4 Writing the Interrupt Handler

Use the callback provided by the HAL library:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) {
        // Example: Toggle LED
    }
}
```

HAL automatically links the interrupt request (IRQ) handler:

```
void EXTI0_IRQHandler(void) {
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

3.8 Common GPIO Function for STM32

3.8.1 HAL Functions

Function / Macro	Explanation
`HAL_GPIO_Init()`	Initializes the specified GPIO port and pins with mode, pull-up/down, speed, and alternate function settings.
`HAL_GPIO_WritePin()`	Sets or resets the output level of a specific GPIO pin.
`HAL_GPIO_ReadPin()`	Reads the current state (HIGH or LOW) of a specified GPIO input pin.
`HAL_GPIO_TogglePin()`	Toggles the output state of a specific GPIO pin (from HIGH to LOW or vice versa).
`HAL_GPIO_DeInit()`	Resets the specified GPIO pin configuration to default (analog mode, no pull-up/down).
`__HAL_RCC_GPIOx_CLK_ENABLE()`	Enables the clock for a given GPIO port (e.g., `GPIOA`, `GPIOB`). This is mandatory before any GPIO operation.
`__HAL_GPIO_EXTI_CLEAR_FLAG()`	Clears the EXTI line pending interrupt flag. Used when handling external interrupts.
`HAL_GPIO_EXTI_Callback()`	Weak user-defined callback that gets triggered when an external GPIO interrupt occurs.
`HAL_GPIO_LockPin()`	Locks the configuration of the selected GPIO pin(s) to prevent accidental changes until next reset.
`HAL_GPIO_EXTI_IRQHandler()`	Handles the EXTI interrupt for external GPIO pin events (e.g., button press).

Table 8.3: Common GPIO Functions in STM32F3 HAL Library

3.8.2 BSP Functions

BSP Function / Macro	Explanation
`BSP_LED_Init(LEDx)`	Initializes the specified on-board LED (e.g., `LED3`, `LED4`) by configuring the corresponding GPIO pin.
`BSP_LED_On(LEDx)`	Turns ON the specified LED.
`BSP_LED_Off(LEDx)`	Turns OFF the specified LED.
`BSP_LED_Toggle(LEDx)`	Toggles the state of the specified LED (from ON to OFF or vice versa).
`BSP_PB_Init(Button, Mode)`	Initializes the on-board push-button in GPIO or EXTI mode. `Mode` can be `BUTTON_MODE_GPIO` or `BUTTON_MODE_EXTI`.
`BSP_PB_GetState(Button)`	Returns the current state (pressed or not) of the specified button.
`LEDx`	Macro representing on-board LEDs. Common values: `LED3`, `LED4`, etc.
`BUTTON_USER`	Macro for the on-board user push-button.
`BUTTON_MODE_GPIO`	Configures the button as a simple input GPIO.
`BUTTON_MODE_EXTI`	Configures the button to trigger an external interrupt (EXTI).

Table 8.4: Common GPIO BSP Functions for STM32F3 Discovery Board

3.9 Segment Bit Patterns for 7-Segment Display

Use the following lookup table to define segment patterns for digits:

Table 8.5: Common Anode 7-Segment Display Bit Patterns

Digit	gfedcba	Binary Pattern	Hex Value
0	1000000	0b1000000	0x40
1	1111001	0b1111001	0x79
2	0100100	0b0100100	0x24
3	0110000	0b0110000	0x30
4	0011001	0b0011001	0x19
5	0010010	0b0010010	0x12
6	0000010	0b0000010	0x02
7	1111000	0b1111000	0x78
8	0000000	0b0000000	0x00
9	0010000	0b0010000	0x10
A	0001000	0b0001000	0x08
b	0000011	0b0000011	0x03
C	1000110	0b1000110	0x46
d	0100001	0b0100001	0x21
E	0000110	0b0000110	0x06
F	0001110	0b0001110	0x0E

3.10 Tips

1. Optimizing the Code for Seven Segment Output

Rather than writing seven separate `HAL_GPIO_WritePin()` calls for each digit, you can optimize your code using a helper function.

2. Using a Lookup Table and Bit Shifting Create a `display_number()` function:

```
void display_number(uint8_t num) {
    // Lookup table of 8-bit binary segment patterns for 0 to F
    uint8_t segment_map[16] = { ... };
    uint8_t pattern = segment_map[num];
    for (int i = 0; i < 7; i++) {
        HAL_GPIO_WritePin(GPIOx, GPIO_PIN_y[i], (pattern >> i) & 1);
    }
}
```

2. Directly Writing to ODR (Output Data Register)

If all segment pins are connected to consecutive bits of the same GPIO port, you can write all at once using bit masks:

```
    GPIOA->ODR = (GPIOA->ODR & ~0x7F) | pattern;
```

3.11 Evaluation Questions

(a) What is the key difference between polling and interrupt-based GPIO input handling, and when would you prefer using interrupts?
(b) Suppose you're connecting a push button to a GPIO pin. Why is a pull-down resistor necessary, and what might happen if it's missing?

3.12 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
2	1	/5	/10	/10	/5	–
	2	–	/10	/5		
	3	/5	/5	/5		
	4	–	/10	/5		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	-
Total						**/100**

Lab 4: Configuring LED Control Using Timers: Polling and Interrupts

4.1 Objectives

This lab introduces timers and their basic usage in microcontrollers. You will learn what timers are, why they are important, and how to configure and use them on the STM32F3 Discovery board to create time delays and generate periodic output signals using both polling and interrupt techniques.

4.2 Background

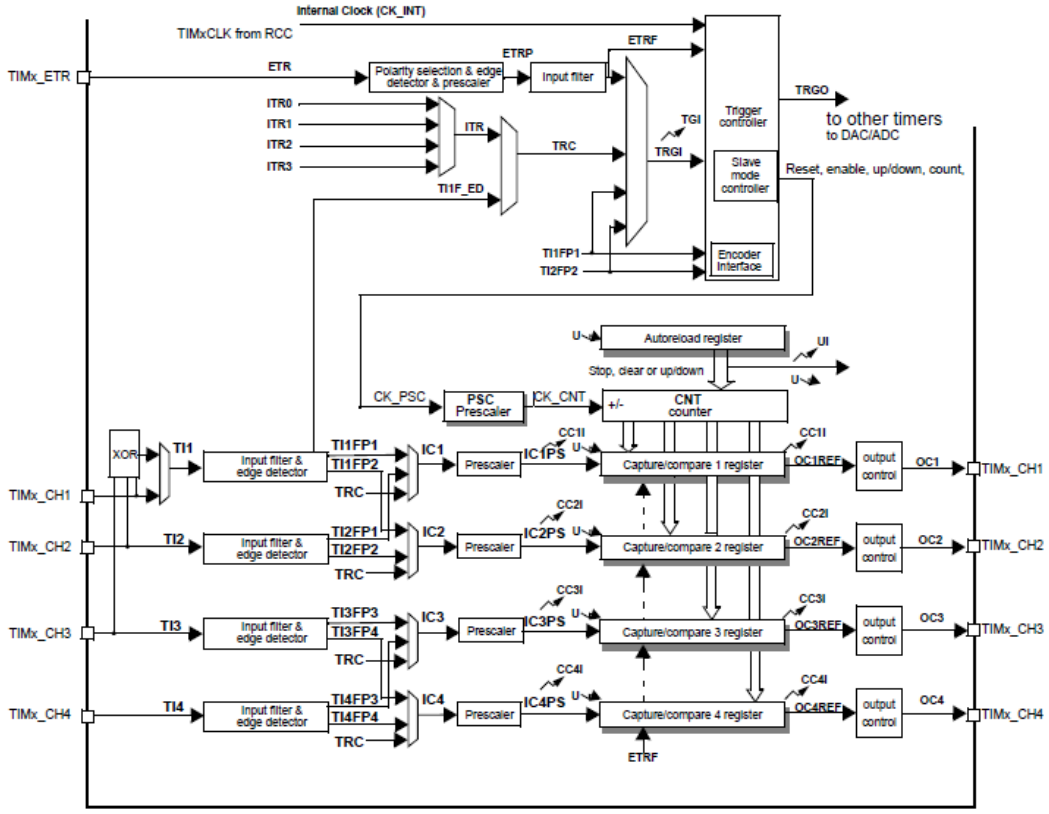


Figure 9.1: Timer Block Diagram

What are Timers?
In microcontrollers, timers are dedicated hardware peripherals that measure and control time-based operations. Timers operate independently of the main CPU and are crucial for generating precise time delays, measuring signal durations, counting external events, generating Pulse Width Modulated (PWM) signals, and triggering periodic interrupts. Their versatility makes them fundamental in real-time embedded systems, motor control, communication protocols, and signal processing tasks.

Why Are Timers Important?
Timers allow microcontrollers to perform precise, time-dependent tasks without burdening the main CPU. They facilitate correct scheduling, timekeeping, and event synchronization, which is crucial for real-time responsiveness. Whether it's generating PWM signals for motor control,

creating delays, or measuring the duration of an input pulse, timers provide the necessary accuracy and hardware-level efficiency. Without timers, software-based timing would consume more processing power and reduce reliability.

Timer Modes in STM32F3

The STM32F3 series microcontrollers provide flexible general-purpose and advanced timers that support various operating modes. These modes include:

- **Time-Base Mode**: The timer counts upward, downward, or in center-aligned mode, generating interrupts on overflow or underflow. This mode is typically used for periodic event generation.
- **Output Compare Mode**: Compares the timer counter value with a preset value to generate an output signal change (toggle, set, or clear) at precise timings.
- **PWM Mode (Pulse Width Modulation)**: A special case of output compare where the timer generates a periodic waveform with a variable duty cycle, widely used in motor control and signal modulation.
- **Input Capture Mode**: Captures and stores the current counter value upon detecting an input signal edge, useful for measuring pulse width, signal frequency, or time between events.
- **Encoder Interface Mode**: Uses two timer channels to decode quadrature signals from rotary encoders, allowing measurement of position, direction, and speed.
- **One-Pulse Mode**: Generates a single output pulse after a trigger event, suitable for one-shot signal generation or delayed response applications.

These modes can be configured independently or combined to perform complex timing, measurement, and control functions in embedded applications. In this course, we will be learning base mode, PWM mode and input capture mode.

Core Time-Base Registers

The STM32F3 timer's basic counting behavior is controlled by the following key registers:

- **TIMx_CR1 (Control Register 1)**: Configures core timer behavior. Important bits include:
  - `CEN` – Enables or disables the timer counter.
  - `DIR` – Selects counting direction: up or down.
  - `ARPE` – Enables auto-reload preload, allowing the ARR value to be buffered and updated only on an update event.
- **TIMx_PSC (Prescaler Register)**: Sets the division factor for the timer clock. The timer counter frequency is reduced according to:

$$f_{\text{counter}} = \frac{f_{\text{timer clock}}}{\text{PSC} + 1}$$

  This allows flexible timing adjustment without changing the main system clock.
- **TIMx_ARR (Auto-Reload Register)**: Determines the counter's upper (or lower) limit. Once the counter reaches this value (in upcounting mode), it resets and optionally triggers an update event:

$$f_{\text{update}} = \frac{f_{\text{timer clock}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

  With `ARPE` enabled, changes to ARR take effect only after an update event.
- **TIMx_CNT (Counter Register)**: Holds the current value of the timer counter. This register can be read at any time to track elapsed time or written to reset or preload a custom starting value. It increments (or decrements) based on the selected counting mode.

The STM32F3 series microcontrollers include multiple hardware timers, each designed to perform various timing, counting, and signal-generation functions. To reflect this modularity, the symbol `x` is used in register names (e.g., `TIMx_CR1`, `TIMx_CNT`), indicating that the same register structure applies to multiple timer instances (e.g., TIM2, TIM3, TIM4). These timers vary in their counter width: some are **16-bit** timers, capable of counting from 0 to 65,535, while others are **32-bit** timers, supporting counts up to 4,294,967,295. The bit-width affects both the maximum measurable time interval and the resolution of timing operations.

For the purpose of this course, we will focus on the **general-purpose timers**—specifically `TIM2`, `TIM3`, and `TIM4`. These timers provide essential features such as basic time-base generation, output compare, PWM generation, and input capture. They are suitable for a wide range of embedded

applications and provide a practical foundation for learning timer configuration and use in real-time systems. If you need, you can also use other timers as well.

This lab focuses on timer base mode, whereas next two modes will be covered in the upcoming weeks. A timer in base mode typically used to generate delays. This mode could be further sub divided into two sub modes.

In **up-counting mode** (Fig. 9.2), the timer counter (`TIMx_CNT`) starts from zero and increments on each timer clock cycle until it reaches the value in the Auto-Reload Register (`TIMx_ARR`). Once this value is reached, an update event is triggered, the counter resets to zero, and the cycle repeats. In **down-counting mode** ((Fig. 9.3)), the counter starts from the value in `ARR` and decrements down to zero, at which point an update event occurs and the counter is reloaded. The counting direction is configured using the `DIR` bit in the `TIMx_CR1` register. The choice between up and down modes affects how the timer behaves in applications such as signal generation and event timing but does not alter the timer frequency. These modes provide flexible control over how and when timer-based events are triggered.

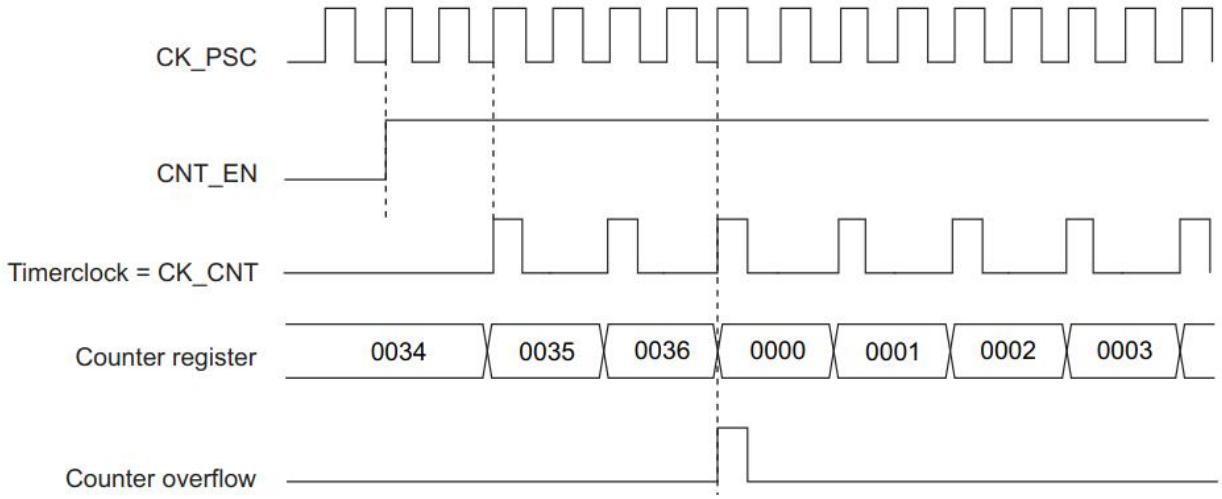


Figure 9.2: Timer as up-counter. CK_PSC is timer peripheral clock, same as STM32F3 system clock. CNT_EN bit enables the timer. CK_CNT is pre-scaled clock used by timer to increment the counter register. In this case, it is prescaled by a factor of 4. Counter register increments by 1 for each timer clock edge(x4 peripheral clock edges). Once it matches the value in ARR register, overflow is set which could be used e.g. to generate and interrupt.

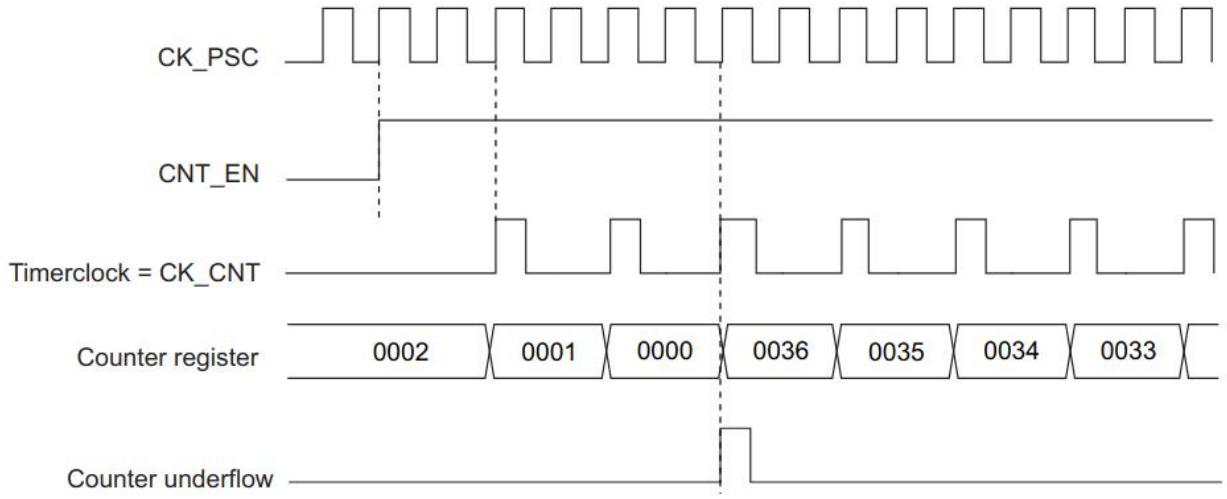


Figure 9.3: Timer as down-counter. All signals are similar to the previous figure, however, now counter register is decrementing by 1 on each timer clock instead of incrementing.

4.3 Task 1: Timer-Based Delay and LED Blinking Using Polling

Objective: Implement a timer-based delay function using polling and use it to blink an LED at 1-second intervals.

Instructions:

(a) Configure Timer:

- Open STM32CubeMX.
- Navigate to `Timers > TIM2`.
- Set the `Clock Source` to `Internal Clock`.
- CubeMX allows configuration of the following Timer settings:

Parameter	Description
Prescaler	Divides the input clock frequency to slow down the timer. For example, with a prescaler of 99, the timer clock is $\frac{f_{\text{timer}}}{100}$.
Counter Mode	Defines the counting direction: up, down, or center-aligned.
Period (ARR)	Sets the value at which the timer overflows and generates an update event or interrupt.
Clock Division	Determines whether the timer clock is further divided before being used for dead-time or digital filtering.
Auto-Reload Preload	When enabled, the period value is buffered and only updated on an update event, reducing glitches.
Interrupt Enable	Allows enabling update or capture/compare interrupts for timer events.
PWM Generation	Enables PWM mode by configuring Output Compare channels with specific duty cycles.
Input Capture/Output Compare	Allows precise measurement of input signal timing or generation of timed output signals.

Table 9.1: Timer Parameter Descriptions in STM32CubeMX

- Set the `Prescaler` such that each tick corresponds to 1 ms.

(b) Implement a reusable delay function:

```
1 void delay_ms(uint32_t ms);
2
```

Function	Description
`__HAL_TIM_SET_COUNTER(&htim2, 0)`	Resets the TIM2 counter value to 0.
`HAL_TIM_Base_Start(&htim2)`	Starts TIM2 in base mode. The counter begins counting based on the clock source and prescaler settings.
`__HAL_TIM_GET_COUNTER(&htim2)`	Reads the current counter value of TIM2.
`HAL_TIM_Base_Stop(&htim2)`	Stops TIM2 from counting.

Table 9.2: Timer Functions

(c) Initialize the onboard LED as a GPIO output.

(d) In the `main` loop, toggle the LED and insert `delay_ms(1000)` between toggles.

4.4 Timer Interrupts

In embedded systems, interrupts allow the microcontroller to respond to events asynchronously, without constantly polling or blocking program execution. A timer interrupt occurs when a timer overflows (reaches its auto-reload value), and it automatically triggers a function allowing the CPU to perform time-based actions without wasting cycles in a busy wait.

How do they work?
A timer generates an update event when its counter reaches the auto-reload value. If interrupts are enabled, this update event:

(a) Triggers an interrupt request (IRQ)

(b) Causes the MCU to jump to an interrupt service routine (ISR)

(c) Executes a user-defined callback function.

4.5 Task 2: LED Blinking Using Timer Interrupt

Objective: Use a timer interrupt to blink an LED at 1-second intervals without blocking the main loop.

Instructions:

(a) In STM32CubeMX, configure TIM2 to generate an interrupt every 1 second.

(b) Enable NVIC interrupt for TIM2.

(c) In `main.c`, implement the callback:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOx, GPIO_PIN_y);
    }
}
```

(d) In `main`, before the `while(1)`, start the timer using:

```
HAL_TIM_Base_Start_IT(&htim2);
```

4.6 Task 3: Blink 3 LEDs at Different Rates Using Timer Interrupt

Objective: Use a single timer interrupt and software counters to blink 3 LEDs at 3 different frequencies.

Instructions:

(a) Configure TIM2 to generate an interrupt every 1 millisecond.

(b) Inside the interrupt handler, increment three separate variables (e.g., `countA`, `countB`, `countC`).

(c) When a counter reaches its threshold, toggle the corresponding LED and reset the counter.

(d) Example thresholds:

- `countA` → 500 ms (1 Hz)
- `countB` → 200 ms (2.5 Hz)
- `countC` → 100 ms (5 Hz)

4.6.1 Extension: Using Timer Interrupts for Frequency Measurement (Input Capture Mode)

When using a timer to measure the frequency of an external signal (e.g., from a sensor or square wave generator), the timer is set up in **Input Capture mode**. This allows the timer to **record the timestamp** (i.e., current counter value) of rising or falling edges on a pin.

Key additions required:

- Configure one of the timer channels in **Input Capture** mode via CubeMX (e.g., TIM3_CH1).
- Enable the corresponding NVIC interrupt (e.g., TIM3 global interrupt).
- Implement the `HAL_TIM_IC_CaptureCallback()` in `main.c`.

Example Callback to Measure Period:

```
uint32_t last_capture = 0, period = 0;
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {
        uint32_t current_capture = HAL_TIM_ReadCapturedValue(htim,
   TIM_CHANNEL_1);
        period = current_capture - last_capture;
        last_capture = current_capture;
    }
}
```

Start the Input Capture Timer:

```
HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1);
```

Calculate Frequency:

$$\text{Frequency (Hz)} = \frac{\text{Timer Clock Frequency}}{\text{Prescaler} \times \text{Period}}$$

Note: Timer overflow handling may be required if the input signal is slow.

4.7 Tips

- Disable timer interrupt generation in polling-based tasks.

4.8 Evaluation Questions

(a) What is the advantage of using a hardware timer for delays instead of a for or while loop?

4.9 Assessment Rubric

CLO; LDL	Task No.	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
3	1	/10	/10	/5	–
	2	/15	/10		
	3	/15	/10		
	Evaluation Questions	–	–	–	/10
4	Git	–	–	/15	-
Total					**/100**

Lab 5: Using Timers for PWM Generation and Motor Speed Control

5.1 Objective

Understand Pulse Width Modulation (PWM) and DC Motor Control.

5.2 Introduction to Pulse Width Modulation (PWM)

5.2.1 Pulse Width Modulation (PWM)

In digital systems, output signals are typically restricted to two discrete states: logic high (e.g., 5V or 3.3V) and logic low (0V). However, many applications require intermediate voltages or variable power, such as controlling the brightness of an LED, the speed of a motor, or the volume of a buzzer.
Pulse Width Modulation (PWM) offers a simple yet powerful way to simulate analog behavior using digital means. Instead of outputting a steady analog voltage, a PWM signal rapidly toggles between high and low, and the proportion of time spent in the high state determines how much energy is delivered to the device.

5.2.2 Structure of a PWM Signal

A PWM signal is periodic, consisting of repeating cycles of ”on” and ”off” pulses. The two defining characteristics of any PWM waveform are:

- **Period** (T): the duration of one complete cycle (from one rising edge to the next).
- **Duty Cycle** (D): the fraction of one period during which the signal is high.

The frequency of the PWM signal is the reciprocal of the period:

$$f = \frac{1}{T}$$

and the duty cycle (as a percentage) is

$$D = \frac{t_{\text{on}}}{T} \times 100\%$$

where t_{on} is the high time in one cycle.

To better understand PWM behavior, visualize a digital waveform where only the width of the high pulse changes, while the overall period remains constant:

5.2.3 Effective Voltage and Power Delivery

Though digital, the average voltage over time of a PWM signal behaves like an analog voltage:

$$V_{\text{avg}} = D \times V_{\text{high}}$$

Where,

- D is the duty cycle (as a fraction between 0 and 1).
- V_{high} is the logic-high level of the digital output (e.g., 5V or 3.3V).

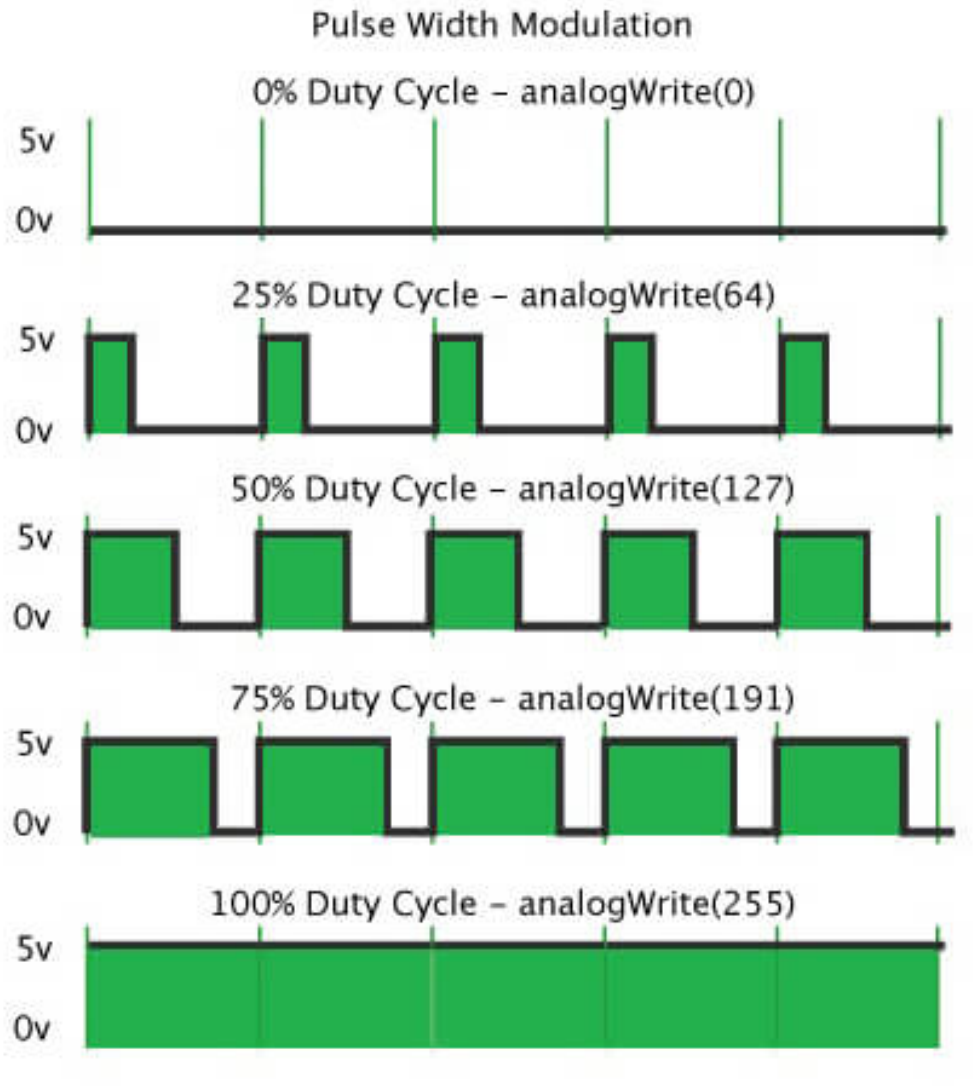


Figure 10.1: PWM waveforms with varying duty cycles

5.2.4 Mathematical Analysis of PWM Behavior

To design or interpret PWM behavior in a system, the following relationships are essential:

- **PWM Period:**

$$T = \frac{1}{f}$$

- **Pulse Width (High Time):**

$$t_{\text{on}} = D \cdot T$$

- **Average Output Voltage:**

$$V_{\text{avg}} = D \cdot V_{\text{high}}$$

These formulas describe how the frequency and duty cycle affect the physical behavior of devices connected to the PWM signal. They also allow engineers to calculate required timing values when designing PWM controllers.

5.3 PWM Configuration with STM32F3DISCOVERY Microcontroller

5.3.1 Overview of PWM in STM32 Timers

On STM32 microcontrollers, PWM generation is performed using general-purpose or advanced-control timers. Each timer has multiple channels capable of generating PWM signals. The signal appears on specific alternate function pins, mapped to timer channels through the GPIO peripheral. In CubeMX, PWM is configured by:

- Selecting a Timer Channel (e.g., TIM3 Channel 1)
- Setting its mode to PWM Generation CHx
- Selecting a compatible GPIO pin for output (e.g., PA6, PB4, etc.)

5.3.2 Timer Configuration for PWM Generation

To configure a timer for PWM, you must understand how the timer's internal counter determines both frequency and duty cycle:
The timer counts up from 0 to a value stored in the Auto-Reload Register (ARR). The Capture/Compare Register (CCR) determines how long the output stays high during this count.

Timer Input Clock

$$f_{\text{timer}} = \begin{cases} f_{\text{APB}} & \text{if prescaler} = 1, \\ 2 \times f_{\text{APB}} & \text{if prescaler} > 1 \end{cases}$$

The APB (Advanced Peripheral Bus) is part of the STM32 microcontroller's internal bus architecture and is used to connect peripheral modules such as timers, USART, SPI, etc. There are two buses: APB1 and APB2, each with its own clock and prescaler settings.

The equation above shows how the timer input clock (f_{timer}) is derived from the APB bus clock (f_{APB}), depending on the prescaler setting. If the prescaler is set to 1, the timer runs directly on the APB clock. If the prescaler is greater than 1, the timer receives twice the APB clock frequency. This behavior ensures that timers maintain high-resolution timing even when peripheral clocks are divided down. It is essential to consider this when calculating timer settings for delays, PWM, or other timing operations.

Selecting Prescaler (PSC) and ARR

To generate a PWM signal of frequency f_{PWM}, choose prescaler (PSC) and ARR such that:

$$f_{\text{PWM}} = \frac{f_{\text{timer}}}{(\text{PSC} + 1)(\text{ARR} + 1)}.$$

This gives you:

$$(\text{PSC} + 1)(\text{ARR} + 1) = \frac{f_{\text{timer}}}{f_{\text{PWM}}}$$

Once ARR is fixed, the duty cycle (%) is given by:

$$\text{Duty Cycle} = \frac{\text{CCR}}{\text{ARR} + 1} \times 100$$

Example:

Goal: Generate a 1 kHz PWM signal with 50% duty cycle.
Assume: $f_{\text{timer}} = 48\,\text{MHz}$

Step 1: Compute total ticks needed per cycle

$$\frac{48\,000\,000}{1\,000} = 48\,000 \quad \text{(total timer ticks)}$$

Step 2: Choose convenient PSC and ARR E.g.,

$$\text{PSC} = 47 \quad \Rightarrow \quad \text{timer now counts at 1 MHz}$$

$$\text{ARR} = 999 \quad \Rightarrow \quad \text{overflows every 1000 ticks (i.e., every 1 ms = 1 kHz)}$$

Step 3: Set CCR = 500 for 50% duty cycle.

5.3.3 CubeMX Configuration

In STM32CubeMX:

(a) Select TIM3 (or any timer) and enable "PWM Generation CHx".

(b) Assign the channel to a GPIO pin (e.g., PA6).

(c) Set Prescaler, Counter Period (ARR), and Pulse (CCR).

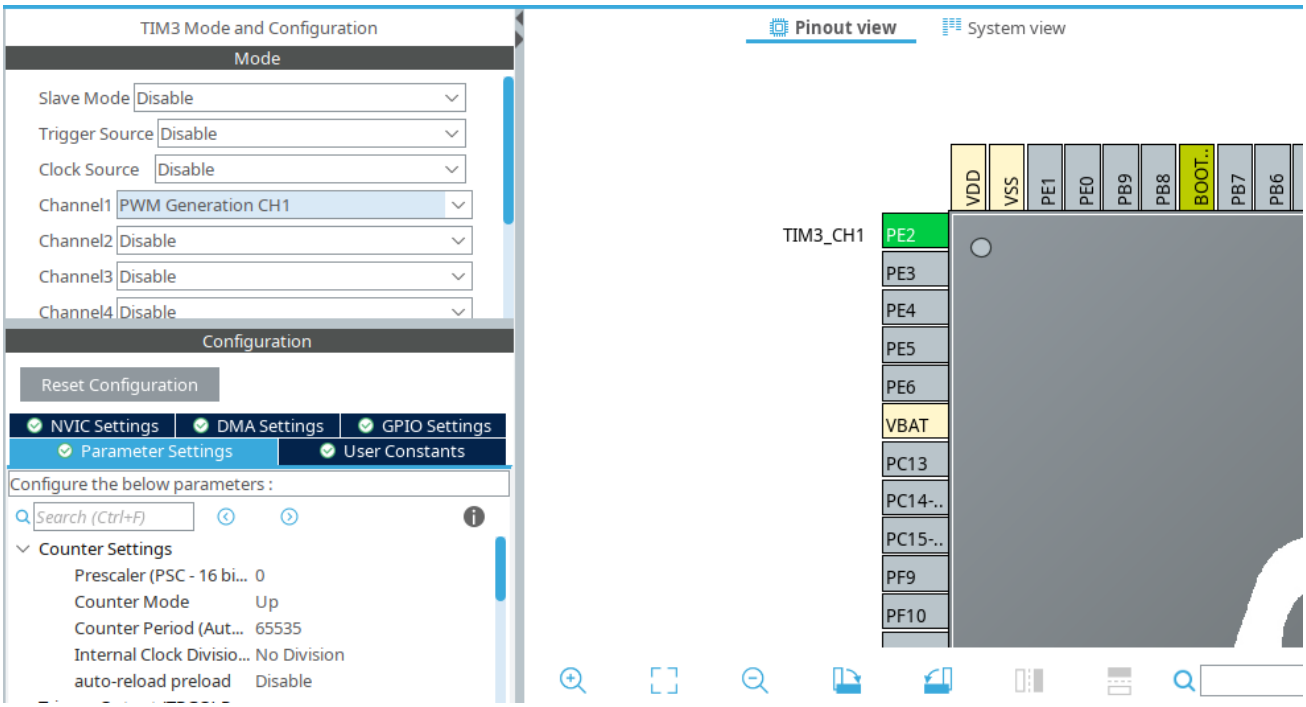


Figure 10.2: TIM3 Pinout Configuration in CubeMX

5.3.4 Code Integration Using STM32 HAL

Use the following code in `main.c` after peripheral initialization:

```
// Start PWM
HAL_TIM_PWM_Start(&htim_x, TIM_CHANNEL_y);

// Update duty cycle
__HAL_TIM_SET_COMPARE(&htim_x, TIM_CHANNEL_y, new_CCR_value);

// Stop PWM
HAL_TIM_PWM_Stop(&htim_x, TIM_CHANNEL_y);
```

5.4 Task 1: LED Brightness Control

Use PWM to fade an LED in and out:

(a) Configure a timer channel for PWM in CubeMX.

(b) Attach an LED to the PWM-capable pin.

(c) Set frequency $\geq$ 1 kHz to avoid flicker.

(d) In code, loop CCR from 0 to ARR and back, with delays.

5.5 Task 2: DC Motor Speed Control

Drive a DC motor at variable speeds:

(a) Connect motor driver to STM32 board.

(b) Use PWM to vary motor speed; at 0% duty cycle the motor stops.

(c) Implement direction control (CW and CCW).

D8	D12	D10 / PWM	Right Motor	D7	D6	D9 / PWM	Left Motor
HIGH	LOW	0–255	Go front	HIGH	LOW	0–255	Go front
LOW	HIGH	0–255	Go back	LOW	HIGH	0–255	Go back
HIGH	HIGH	/	Stop	HIGH	HIGH	/	Stop
LOW	LOW	/	Stop	LOW	LOW	/	Stop

Table 10.1: Motor control logic using digital and PWM pins

Note: Ground of STM32F3 Board must be connected to the GND of Motor shield.

5.6 Tips

- Disable timer interrupt generation in polling-based tasks.

5.7 Common PWM Related Functions for STM32

Function / Macro	Explanation
`HAL_TIM_PWM_Init()`	Initializes the timer in PWM mode. Must be called before starting PWM output.
`HAL_TIM_PWM_ConfigChannel()`	Configures the PWM parameters (pulse width, polarity, fast mode, etc.) for a specific timer channel.
`HAL_TIM_PWM_Start()`	Starts PWM signal generation on a specific timer channel in blocking mode.
`HAL_TIM_PWM_Start_IT()`	Starts PWM with interrupt mode enabled. Rarely used for basic PWM generation.
`HAL_TIM_PWM_Start_DMA()`	Starts PWM with DMA, typically used for advanced applications like audio or signal generation.
`HAL_TIM_PWM_Stop()`	Stops PWM generation on the specified timer channel.
`__HAL_TIM_SET_COMPARE()`	Updates the pulse width (duty cycle) by setting the compare register (e.g., `TIMx->CCR1`) dynamically at runtime.
`__HAL_TIM_SET_COUNTER()`	Sets the current timer counter value. Useful for resetting the counter or synchronization.
`__HAL_TIM_SET_AUTORELOAD()`	Sets the auto-reload value, which defines the PWM period.
`HAL_TIM_Base_Start()`	Starts the timer without enabling any specific channel. Sometimes used along with manual control.
`HAL_TIM_Base_Init()`	Initializes timer base settings (prescaler, counter mode, period) required before enabling PWM mode.

Table 10.2: Common PWM Functions Using STM32 HAL

5.8 Evaluation Questions

(a) How does increasing or decreasing the duty cycle affect the speed of a DC motor?

(b) Given a timer clock of 72 MHz, how would you configure PSC and ARR to generate a 2 kHz PWM signal?

5.9 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
3	1	/5	/15	/10	/5	–
	2	/10	/20	/10		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	-
Total						**/100**

Lab 6: Using Timers to Implement Capture/Compare Mode

6.1 Objectives

This lab explores the advanced features of microcontroller timers using the Capture/Compare mode. Students will:

- Use GPIO interrupts to start and stop a timer to measure signal frequency.
- Use the input capture mode to measure signal frequency.
- Write a program to read the rotary encoder frequency and calculate RPM using polling.
- Write a program to read the rotary encoder frequency and calculate RPM using input capture.

6.2 Background

6.2.1 Introduction

In order to measure an external signal's frequency, we can utilize timers in one of the following two modes:

Timers as Counters: When used as counters, timers increase their count based on external signals rather than an internal clock. This mode is particularly useful for applications that require the counting of external events such as pulses, encoder signals, or frequency measurements.

Input Capture Mode: Input Capture mode allows the timers to precisely measure the timing characteristics of external signals, such as pulse width, frequency, or period. In this mode, the timer captures and stores the current counter value in a capture/compare register (`TIMx_CCRn`) each time a configured edge (rising, falling, or both) is detected on the corresponding input pin. Each timer channel (usually up to four) can be configured independently to capture input events on specific physical pins, which must be set up in alternate function mode corresponding to the timer channel.

In addition to the above two built-in timer modes, we can also measure the frequency of an external signal using a combination of a timer and GPIO-based edge detection—either through polling or interrupts. In this approach, the timer is configured as a free-running counter, and software logic is used to start and stop timing on signal edges detected via GPIO. By measuring the time interval between consecutive rising or falling edges, we can calculate the signal's period and, hence, its frequency.

In this lab, we will implement two methods to measure the frequency of a rotary encoder signal, which is used to estimate the speed of a DC motor. The first method uses GPIO interrupts to detect edges and software timing logic to compute frequency. The second method leverages the hardware Input Capture feature of the timer, which provides more precise and CPU-efficient frequency measurement by capturing timer values directly on signal edges.

6.2.1 Rotary Encoder and Motor Speed Measurement

A rotary encoder generates a series of digital pulses as it rotates. By counting these pulses over time, one can calculate rotational speed.

6.3 Task 1: Frequency Measurement using GPIO Interrupt

Objective: Measure the frequency of a signal (e.g., from a function generator or square wave output) using GPIO Interrupt.
Hardware Setup:

- Connect the output of a function generator to a microcontroller pin(e.g., PD0).
- Ensure that the signal is compatible with 3.3V logic.

Instructions:

- In STM32CubeMX:
  - Initialize Tim2 with prescaler 9
  - Enable the function generation pin as `GPIO_EXTI`
  - Enable NVIC interrupt for the pin.
  - Set the trigger on the rising edge.
- In code:
  - Within the interrupt callback function, on the first interrupt, start the timer to begin counting.
  - On the second interrupt, capture the current timer value and store it in an array.
  - Repeat this process until the array is filled with multiple measurements for averaging (e.g., average sample size could be 10, 16, etc.).
  - Once the array is full, set a print flag to `true`.
  - In the `main()` loop, check every 100 ms if the print flag is set:
    - If `true`, calculate the average of the captured values.
    - Compute the frequency using the formula:

$$f = \frac{8 \times 10^6}{2 \times \text{average period}}$$

    - Add the error margin to account for measurement uncertainty.
    - Print the final frequency value.

6.4 Task 2: Frequency Measurement using Input Capture

Objective: Measure the frequency of a signal (e.g., from a function generator or square wave output) using Input Capture.

Hardware Setup:

- Connect the output of the function generator to the TIM2 CH1 input capture pin (e.g. PA0).
- Ensure proper pull-up/down configuration.
- Ensure that the signal is compatible with 3.3V logic.

Instructions:

- In STM32CubeMX:
  - Configure TIM2 Channel 1 for input capture.
  - Set the prescaler to 71 for 1 micro second tick.
  - Set to rising edge capture, and enable the interrupt.
- In code:
  - In interrupt callback function, capture time between pulses to period.
  - Compute the frequency by dividing the Tim2 frequency by the computed period.
  - Display result over UART.

6.5 Task 3: Measure Motor Speed Using Encoder and Polling

Objective: Write a program to read the rotary encoder frequency and calculate the RPM using polling.

Hardware Setup:

Shield Pin	STM32 Pin	Description
D4	GPIO (Encoder Input)	Right motor encoder output. Reads encoder pulses from the right motor to measure speed or position.
D5	GPIO (Encoder Input)	Left motor encoder output. Reads encoder pulses from the left motor to measure speed or position.
5V	5V	5V power connection between the motor shield and STM32 board to power the encoder modules. Do not connect the vin pin of the shield to 5V.
GND	GND	Common ground connection. The ground pins of the motor shield and STM32 board must be connected together for correct operation.

Table 11.1: Pin-level connectivity between Encoder Module and STM32F3 board

Note: Connections for the motor PWM and direction pins must also be made as in Lab 5. Ensure that a common ground is maintained between the STM32 board and the motor shield. You are required to read the encoder value for any one motor.

- Set the desired PWM and duty cycle using Lab05 Task 2.
- Configure a general-purpose timer (e.g., `TIM3`) for time measurement in microseconds.

Instructions:

- In STM32CubeMX:
  - Configure the GPIO pin as input.
- In code:
  - Use polling to detect two consecutive falling edges of the input signal.
  - Start the timer on the first edge and stop it on the second to measure the time between pulses.
  - Calculate the signal frequency using the formula:

$$\text{Frequency (Hz)} = \frac{\text{Timers Frequency}}{\text{Captured Ticks}}$$

  - Determine the PPR (Pulses Per Revolution) either by counting pulses over one complete rotation or by referring to the motor's datasheet.

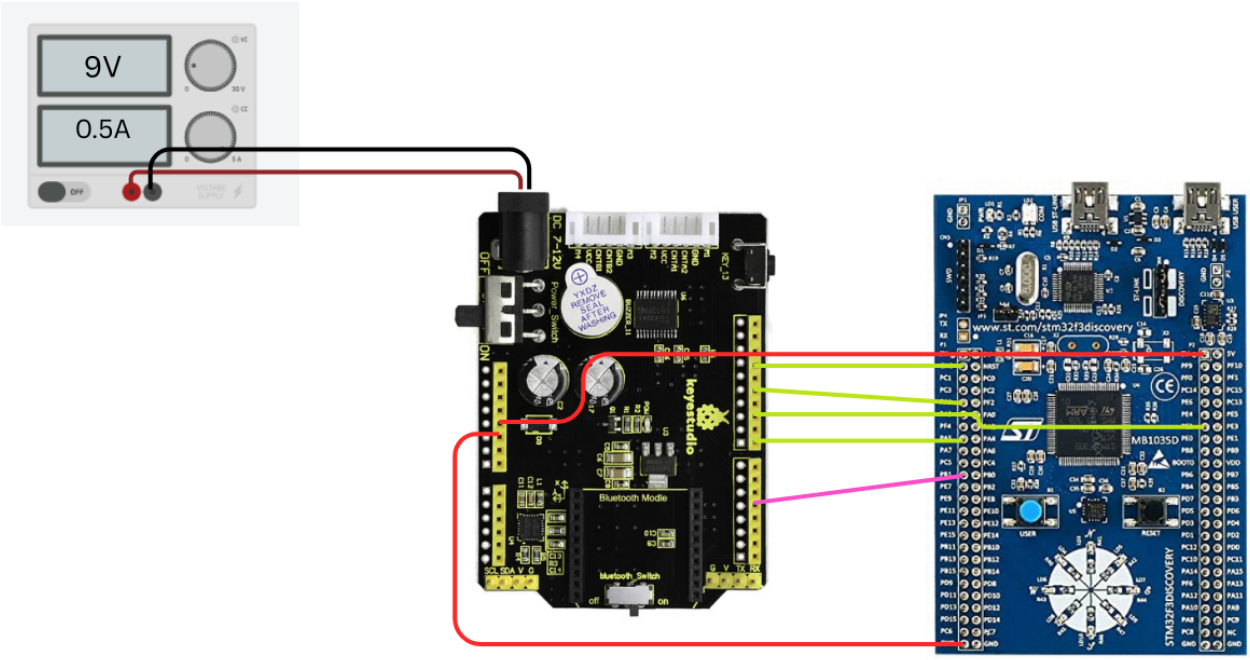


Figure 11.1: schematic for right motor + encoder module

– Calculate the RPM using the following formula:

$$RPM = \frac{60 \times \text{frequency}}{\text{PPR}}$$

– Display the result over UART. .

6.6 Task 4: Measure Motor Speed Using Encoder and Input Capture

Objective: Write a program to read the rotary encoder frequency and calculate the RPM using input capture.

Hardware Setup:

Shield Pin	STM32 Pin	Description
D4	TIM2 Channel 1 e.g. PA0	Right motor encoder output.
D5	TIM2 Channel 1 e.g. PA0	Left motor encoder output.
5V	5V	5V power connection between the motor shield and STM32 board to power the encoder modules. Do not connect the vin pin of the shield to 5V.
GND	GND	Common ground connection. The ground pins of the motor shield and STM32 board must be connected together for correct operation.

Table 11.2: Pin-level connectivity between Encoder Module and STM32F3 board

Note: Connections for the motor PWM and direction pins must also be made as in Lab 5. Ensure that a common ground is maintained between the STM32 board and the motor shield. You are required to read the encoder value for any one motor.

- Set the desired PWM and duty cycle using Lab05 Task 2.

Instructions:

- In STM32CubeMX:
  - Configure TIM2 Channel 1 for input capture.
  - Set to rising edge capture, and enable the interrupt.
- In code:
  - Calculate the encoder frequency by repeating Task 2.
  - Determine the PPR (Pulses Per Revolution) either by counting pulses over one complete rotation or by referring to the motor's datasheet.
  - Calculate the RPM using the following formula:

$$RPM = \frac{60 \times \text{frequency}}{\text{PPR}}$$

  - Display the result over UART.

6.7 Tips

- Use an oscilloscope to verify the frequency and signal integrity of the encoder output.
- Use LEDs onboard to indicate successful interrupt triggering or edge detection.
- Ensure that all input pins connected to the encoder are 3.3V tolerant to avoid damage to the microcontroller.
- Some useful functions for this lab:

Function	Description
`__HAL_TIM_SET_COUNTER(&htim2, 0)`	Resets TIM2 counter to 0 at the beginning of a measurement.
`HAL_TIM_Base_Start(&htim2)`	Starts TIM2 in base mode. The counter begins counting based on the clock source and prescaler settings.
`__HAL_TIM_GET_COUNTER(&htim2)`	Reads the current counter value of TIM2 to determine the elapsed time since the last edge.
`HAL_TIM_Base_Stop(&htim2)`	Stops TIM2 after capturing the counter value to finalize the measurement.
`HAL_TIM_ReadCapturedValue(htim, channel)`	Reads the captured counter value from the specified channel when an input edge occurs.
`HAL_TIM_IC_CaptureCallback(...)`	Callback function automatically invoked by HAL on each capture event. Custom logic to calculate frequency is written here.
`HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)`	Reads the logic level (HIGH or LOW) of a specific GPIO pin. Used here for polling signal edges manually.

Table 11.3: Functions Used in Frequency Measurement

6.8 Evaluation Questions

(a) What is the difference between capture mode and compare mode in a timer? Give an example use case for each.

(b) How can you use input capture to measure the frequency of a signal? What factors could affect the accuracy of this measurement?

(c) During your lab, the frequency readings from input capture mode were inconsistent. List at least two possible reasons for the fluctuation and explain how you would address them.

6.9 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
3	1	-	/5	/10	/5	–
	2	–	/10	/5		
	3	/5	10	/5		
	4	-	/10	/10		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	-
Total						**/100**

Lab 7: Acquire and Filter Analog Signals Using ADC and CMSIS DSP

7.1 Objectives

This lab introduces the basics of Analog to Digital Conversion (ADC) on the STM32F3 Discovery board. Students will:

- Understand how ADC works and how to configure it.
- Read and interpret analog values from sensors.
- Transmit sensor data over UART.
- Implement a basic low-pass filter using CMSIS DSP library.

7.2 Introduction to Analog-to-Digital Conversion

In the physical world, most phenomena—such as temperature, light intensity, voltage, or sound—exist in analog form. These analog signals are continuous in both time and amplitude. However, modern digital systems like microcontrollers and processors operate on discrete binary values. The Analog-to-Digital Converter (ADC) serves as the bridge between the continuous analog world and the discrete digital domain.

7.0.1: What is ADC?

An Analog-to-Digital Converter (ADC) is an electronic subsystem that transforms an analog input signal (typically voltage) into a digital representation of that signal. This digital value can then be stored, processed, or transmitted by digital systems.

For example, a varying analog voltage from a temperature sensor can be sampled by an ADC and represented as a corresponding binary number (e.g., `01010110`), which a microcontroller can then use in computations.

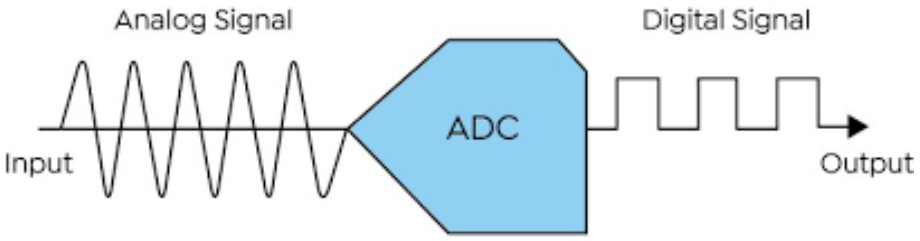


Figure 12.1: Analog to Digital Conversion

7.0.2: The Process of Analog-to-Digital Conversion

The ADC process involves two primary steps:

1. Sampling: Sampling is the process of measuring the amplitude of an analog signal at discrete intervals of time. Instead of recording every tiny variation in the signal, we take "snapshots" at regular time steps.

- **Sampling Rate (Sampling Frequency):** This is the number of samples taken per second, measured in Hertz (Hz). A higher sampling rate captures more details of the original signal.

- **Nyquist Theorem:** To accurately represent an analog signal, the sampling rate should be at least twice the maximum frequency present in the analog signal.

Example: To sample a 1 kHz signal, you should use at least a 2 kHz sampling rate.

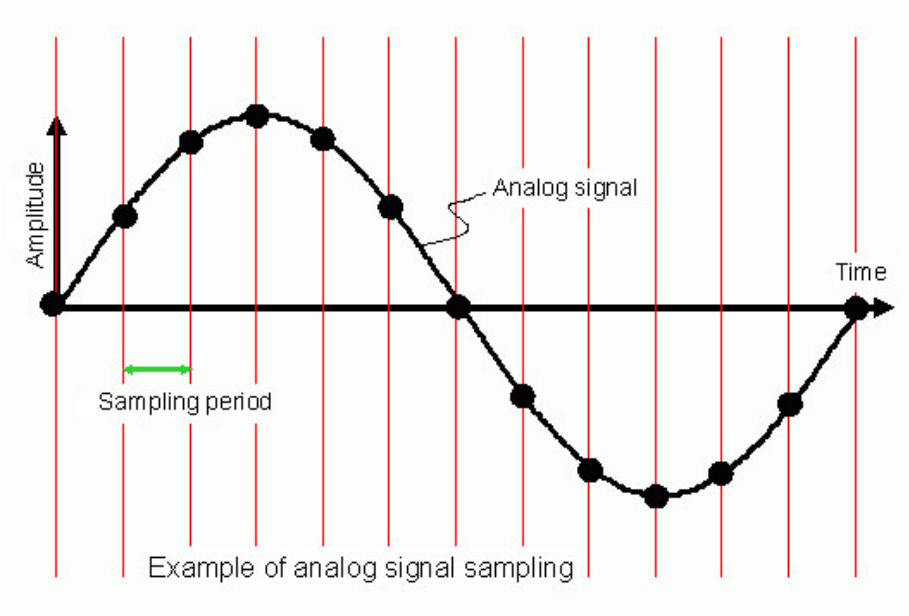


Figure 12.2: Sampling

2. Quantization and Encoding: After sampling, the next step is quantization, where each sample's amplitude is rounded to the nearest value from a finite set of levels.

- **Resolution (Bits):** This determines how many levels you can quantize the signal into. If an ADC has N-bit resolution, it means it can represent values using 2^N levels.

Example: An 8-bit ADC $\rightarrow 2^8 = 256$ levels A 12-bit ADC $\rightarrow 2^{12} = 4096$ levels

If your ADC can measure voltages from 0 V to V_{ref}, then each quantization level (also called LSB, Least Significant Bit) represents:

$$\text{LSB (Step Size)} = \frac{V_{\text{ref}}}{2^n}$$

Encoding is the final step in the ADC process where each quantized value is converted into binary form for digital processing.

Example: If the quantized level corresponds to level 5 in a 3-bit ADC, it will be encoded as `101`.

7.0.3: Key ADC Parameters

- **a. Resolution**
  Defined as the number of bits used to represent the analog value. An n-bit ADC divides the input range into 2^n levels. Higher resolution allows for finer precision.

  Example: A 10-bit ADC can represent 1024 discrete levels (from 0 to 1023).

- **b. Reference Voltage (V_{ref})**
  The maximum analog input voltage that corresponds to the highest digital output. The digital output is proportional to the input voltage relative to V_{ref}.

- **c. Quantization Step Size (ΔV)**
  The minimum change in voltage that the ADC can detect.

$$\Delta V = \frac{V_{\text{ref}}}{2^n}$$

  Where:
  n = ADC resolution in bits
  V_{ref} = Reference voltage

- **d. Input Range**
  The range of input voltages that the ADC can convert, typically from 0 V to V_{ref}.

- **e. Sampling Rate / Conversion Time**
  How fast the ADC can convert one analog sample into a digital value. Determines how frequently new data can be captured.

- **f. Accuracy & Error**
  Influenced by noise, quantization error, non-linearity, and reference voltage fluctuations.

7.0.4: Digital Output Calculation

The output of an ideal ADC can be estimated using:

$$D = \left\lfloor \frac{V_{\text{in}}}{V_{\text{ref}}} \cdot (2^n - 1) \right\rfloor$$

Where:

- V_{in} = Input Analog Voltage
- V_{ref} = Reference Voltage
- n = Resolution (in bits)
- D = Digital output (an integer between 0 and $2^n - 1$)

Example: For a 10-bit ADC with $V_{\text{ref}} = 3.3\,\text{V}$ and $V_{\text{in}} = 1.65\,\text{V}$:

$$D = \left\lfloor \frac{1.65}{3.3} \cdot 1023 \right\rfloor = \lfloor 0.5 \cdot 1023 \rfloor = 511$$

7.0.5: Real-World Applications of ADC

- Reading sensors (temperature, light, pressure)
- Audio signal processing (microphones, musical instruments)
- Image sensors in cameras
- Industrial automation
- Biomedical devices (e.g., ECG, EEG)
- Robotics (sensing position, velocity, etc.)

7.3 ADC on STM32F3 Discovery - Configuration and Implementation

7.1.1: Overview

The STM32F3 Discovery board features multiple ADC peripherals that allow conversion of analog signals into digital values. These ADCs can operate in single, continuous, scan, or discontinuous conversion modes and can be configured through STM32CubeMX.

In this section, we explore how to configure the ADC in STM32F3, understand the available parameters, and examine how to implement ADC functionalities through both polling and interrupt-driven approaches.

7.1.2: CubeMX Configuration

When configuring the ADC in STM32CubeMX for the STM32F3 Discovery board, the following steps and parameters are crucial:

a) Enabling ADC and Selecting Channels

- Open CubeMX and enable an ADC (e.g., ADC1).
- Select the desired ADC channel by clicking on the corresponding pin (e.g., `PA0` for `ADC1_IN1`).
- You may also configure multiple channels for scan conversion if needed.

b) Parameter Configuration
CubeMX allows configuration of the following ADC settings:

Parameter	Description
Resolution	Defines the number of bits of the conversion result (e.g., 12-bit, 10-bit). A 12-bit resolution provides values from 0 to 4095.
Data Alignment	Determines whether the data is right-aligned or left-aligned in the data register.
Scan Conversion Mode	Enables scanning across multiple ADC channels in a sequence.
Continuous Conversion Mode	Automatically starts a new conversion after the previous one finishes.
Discontinuous Mode	Allows limited conversions in a group during a scan.
External Trigger Conversion	Specifies external events that trigger the ADC conversion, e.g., timer output.
DMA Continuous Requests	Enables use of DMA to automatically transfer ADC results to memory.
Sampling Time	Controls how long the analog signal is sampled before conversion. A longer sampling time increases accuracy but reduces speed.

Table 12.1: ADC Parameter Descriptions in STM32CubeMX

c) ADC Clock Configuration
In the *Clock Configuration* tab, ensure that the ADC clock is set appropriately. The ADC clock frequency impacts the maximum conversion speed.

7.1.3: Modes of Operation

a) Polling Mode
In this mode, the CPU actively waits for the conversion to complete before reading the value.

Function Flow:

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
uint32_t value = HAL_ADC_GetValue(&hadc1);
HAL_ADC_Stop(&hadc1);
```

b) Interrupt Mode
The ADC triggers an interrupt once conversion completes, allowing the CPU to continue other operations.

CubeMX Setup:

- Enable the ADC global interrupt in the NVIC tab.
- Enable "Interrupt Conversion" in the ADC settings.

Code Snippet:

```
HAL_ADC_Start_IT(&hadc1);
```

Callback Function:

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc->Instance == ADC1) {
        uint32_t adc_val = HAL_ADC_GetValue(hadc);
        // Use adc_val as needed
    }
}
```

7.1.4: Additional Functions

Function	Description
HAL_ADC_Init()	Initializes ADC peripheral with configured settings.
HAL_ADC_Start()	Starts ADC conversion in polling mode.
HAL_ADC_Start_IT()	Starts ADC conversion with interrupt enabled.
HAL_ADC_GetValue()	Returns the digital value after conversion.
HAL_ADC_Stop()	Stops ADC in polling mode.
HAL_ADC_Stop_IT()	Stops ADC in interrupt mode.

Table 12.2: Common HAL ADC Functions and Their Descriptions

7.4 Task 1: Reading an Analog Sensor and Transmitting the Value via UART

7.2.1: Objective

The goal of this task is to develop a program that reads the value from an analog sensor (a potentiometer) using the ADC of the STM32F3 Discovery board and transmits the corresponding digital value over UART to a serial terminal. This task consolidates the students' understanding of ADC configuration and data transmission using UART.

7.2.2: Overview of the Sensor: Potentiometer

A potentiometer is a three-terminal analog device that acts as a variable voltage divider. By adjusting the knob of the potentiometer, the voltage at the middle terminal changes continuously between the supply voltage (usually 3.3 V or 5 V) and ground (0 V). This makes it an ideal analog input for demonstrating the functionality of an ADC.

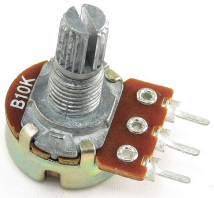


Figure 12.3: Potentiometer

Wiring the potentiometer:

- One side pin → Connect to 3.3 V
- Other side pin → Connect to GND
- Middle pin (wiper) → Connect to ADC input pin on the STM32F3 board (e.g., `PA0` for `ADC1_IN1`)

7.2.3: Instructions

Configure ADC in STM32CubeMX

- Enable the `ADC1` peripheral.
- Select the appropriate channel corresponding to the pin connected to the potentiometer (e.g., `IN1` for `PA0`).
- Set the resolution (e.g., 12-bit).
- Set the data alignment (right-aligned).
- Choose the conversion mode (single or continuous).
- Enable Interrupt mode (optional, but recommended for efficiency).
- Configure ADC clock and sampling time based on desired precision and speed.

Generate code and implement logic

- Start ADC conversion.
- Read the digital values from the ADC.
- Print those values via UART.
- Observe the digital value changing as you rotate the potentiometer.

7.5 Task 2: Signal Filtering Using CMSIS DSP Library

7.3.1: Objective

The goal of this task is to develop a program that applies a Moving Average Low-Pass Filter to a sampled sine wave input acquired via ADC and visualize the result using a provided Python plotting script. This introduces the fundamental concept of signal filtering and its practical application using the CMSIS DSP Library.

7.3.2: Introduction to Filtering

Filtering in signal processing refers to the technique of modifying or removing unwanted components from a signal. It is crucial in embedded systems where raw sensor signals often contain noise or high-frequency fluctuations that must be suppressed for reliable readings.

Types of Filters:

- **Low-Pass Filter (LPF):** Allows low-frequency components to pass through while attenuating higher-frequency components. Commonly used to smooth signals.
- **High-Pass Filter (HPF):** Allows high-frequency components through, attenuating the lower frequencies.
- **Band-Pass Filter:** Passes a specific frequency range while blocking others.
- **Moving Average Filter:** A type of low-pass filter that computes the average of the most recent N samples. It is simple, non-recursive, and effective for noise reduction.

In this lab, we will use a 10-sample moving average to filter out high-frequency noise from a sine wave input.

7.3.3: CMSIS DSP Library

The Cortex Microcontroller Software Interface Standard (CMSIS) provides optimized signal processing functions for ARM Cortex-M microcontrollers. The CMSIS DSP module includes mathematical operations, filtering, transforms (e.g., FFT), statistics, and more.

7.3.4: CubeMX Configuration for CMSIS DSP and ADC Data Acquisition

Enabling CMSIS DSP Library
To perform digital signal processing (DSP) operations on the STM32F3 Discovery board, we utilize the CMSIS DSP library provided by ARM. This library offers a comprehensive collection of DSP functions, including filters, transforms, statistical operations, and more.

To enable the CMSIS DSP library:

- Open STM32CubeMX.
- Navigate to `Software Packs > CMSIS > DSP`.
- Enable the CMSIS DSP package to include the library in your project.
- Generate the code to include the necessary headers and dependencies.

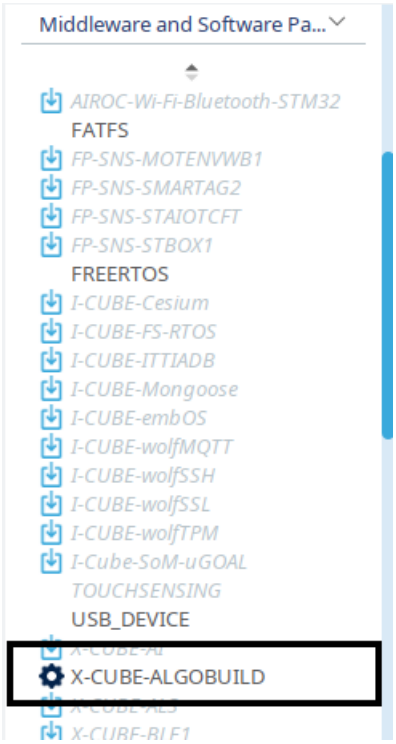


Figure 12.4

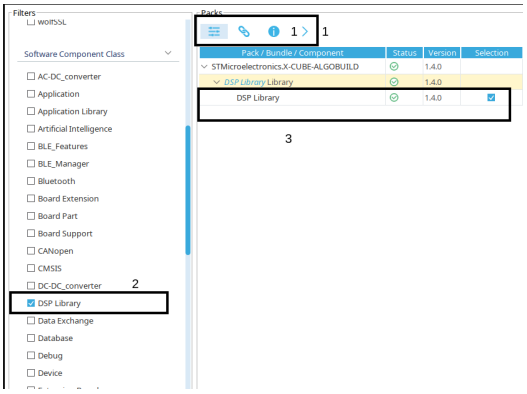


Figure 12.5

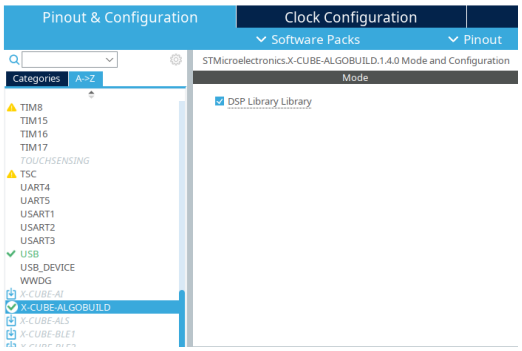


Figure 12.6

ADC Configuration
We will use the ADC (Analog-to-Digital Converter) to sample analog signals, such as those generated by a function generator. The ADC must be configured for continuous sampling to support real-time filtering.

Recommended settings:

- Mode: Continuous Conversion Mode
- Resolution: 12-bit (0–4095 for 0–3.3 V input)
- Sampling Time: Choose based on the input signal frequency

7.3.5: Implementing a Moving Average Filter using CMSIS DSP

We implement a moving average filter using the CMSIS DSP function `arm_mean_f32`, which calculates the arithmetic mean of an array of `float32_t` values.

Conceptual Formula
A moving average filter of length N computes the output $y[n]$ as:

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]$$

Where:

- $y[n]$ is the filtered output at sample index n
- $x[n-k]$ represents the most recent N samples of input
- N is the filter window length (10 samples in this lab)

Code Snippet

```
#include "arm_math.h"
#define FILTER_LEN 10

float32_t inputBuffer[FILTER_LEN];
float32_t output;

void apply_moving_average(float32_t new_sample) {
    // Store the input ADC value in the buffer:
    // Your logic here to increment the index and store new_sample (ADC output) in the buffer

    // Use the DSP function to calculate the moving average of buffer and store in output:
    arm_mean_f32(inputBuffer, FILTER_LEN, &output);
}
```

where,

- `inputBuffer[]` stores the most recent ADC samples (circular buffer).
- `arm_mean_f32()` computes the average of the buffer values.
- `output` holds the filtered value, which can then be transmitted via UART.

7.3.6: Task: Filtering a Sine Wave Input and Visualizing Results

Objective

In this task, you will acquire a sine wave using the ADC, apply a moving average low-pass filter using the CMSIS DSP library, and visualize both raw and filtered signals via UART on a PC using a Python plotting script.
This task is to be completed in two parts as outlined below:

(i) **Polling Mode:** In the first part of the task, the Analog-to-Digital Converter (ADC) should be configured and operated in *polling mode*. In this mode, the microcontroller continuously checks the ADC status flag to determine whether the conversion has been completed before reading the converted value.

(ii) **Interrupt Mode:** In the second part, the ADC should be configured in *interrupt mode*. Here, the ADC generates an interrupt signal once the conversion is complete, allowing the microcontroller to perform other tasks and respond to the conversion result asynchronously through an interrupt service routine (ISR).

Procedure

Hardware Setup

- Connect a function generator to the configured ADC channel of the STM32F3 Discovery board.
- Configure the function generator to output a sine wave, with a frequency ranging between 1 Hz and 100 Hz.
- Ensure that the amplitude does not exceed the ADC's input range (0–3.3 V).

Firmware Implementation

- Initialize ADC in continuous conversion mode.
- Read the ADC value continuously in interrupt mode.
- Convert the ADC result to a floating-point voltage value.
- Apply the 10-point moving average filter to the ADC value.
- Transmit both raw and filtered values over UART.

Python Visualization

- Use the provided Python script (based on `pyserial` and `matplotlib`) to:
  - Read UART data in real-time.
  - Plot both the raw ADC signal and the filtered output on a graph.
- **Note: There may be a slight delay between modifying the settings on the function generator and observing the corresponding changes in the waveform visualization. Please allow a few seconds for the system to reflect any updates.**

Required Outcome

When the frequency of the input signal from the function generator is increased, you will observe a gradual decrease in the amplitude of the output signal after filtering. This behavior indicates that the filter is attenuating higher-frequency components of the input signal. Such a response is characteristic of a low-pass filter, which is designed to allow signals with frequencies below a certain cutoff frequency to pass through relatively unaffected, while attenuating signals with frequencies above that threshold.

7.6 Evaluation Questions

(a) Explain the relationship between ADC resolution and quantization error. Why does increasing resolution reduce error?

7.7 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
2	1	/5	/20	/10	/5	–
	2	–	/20	/15		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	-
Total						**/100**

Lab 8: Establish SPI Communication with the L3GD20D Gyroscope Sensor

8.1 Objectives

This lab introduces the Serial Peripheral Interface (SPI) protocol and demonstrates how to interface the STM32F3 Discovery board with SPI-based peripherals. Students will:

- Understand the SPI protocol and its key features.
- Set up SPI communication on the STM32 microcontroller.
- Write a program to exchange data with an SPI-based sensor
- Learn inter-MCU communication using SPI.

8.2 Communication Protocols

When you connect a microcontroller to a sensor, display, or other module, do you ever think about how the two devices talk to each other? What exactly are they saying? How are they able to understand each other?

Communication between electronic devices is like communication between humans. Both sides need to speak the same language. In electronics, these languages are called **communication protocols**. Luckily for us, there are only a few communication protocols we need to know when building most electronics projects. The three most common protocols are:

- *Serial Peripheral Interface (SPI)*
- *Inter-Integrated Circuit (I2C)*
- *Universal Asynchronous Receiver/Transmitter (UART)*

We have already studied UART-driven communication in previous labs. In this manual, we'll begin with some basic concepts about electronic communication, then explain in detail how SPI works. In the next lab, we'll dive into I2C.

SPI, I2C, and UART are quite a bit slower than protocols like USB, Ethernet, Bluetooth, and WiFi, but they're a lot simpler and use less hardware and system resources. SPI, I2C, and UART are ideal for communication between microcontrollers and between microcontrollers and sensors where large amounts of high-speed data don't need to be transferred.

8.2.1 Serial vs Parallel Communication

Electronic devices talk to each other by sending bits of data through wires physically connected between devices. A bit is like a letter in a word, except instead of the 26 letters (in the English alphabet), a bit is binary and can only be a 1 or 0. Bits are transferred from one device to another by quick changes in voltage. In a system operating at 5 V, a 0 bit is communicated as a short pulse of 0 V, and a 1 bit is communicated by a short pulse of 5 V.

The bits of data can be transmitted either in **parallel** or **serial** form. In parallel communication, the bits of data are sent all at the same time, each through a separate wire. The following diagram shows the parallel transmission of the letter "C" in binary (01000011):

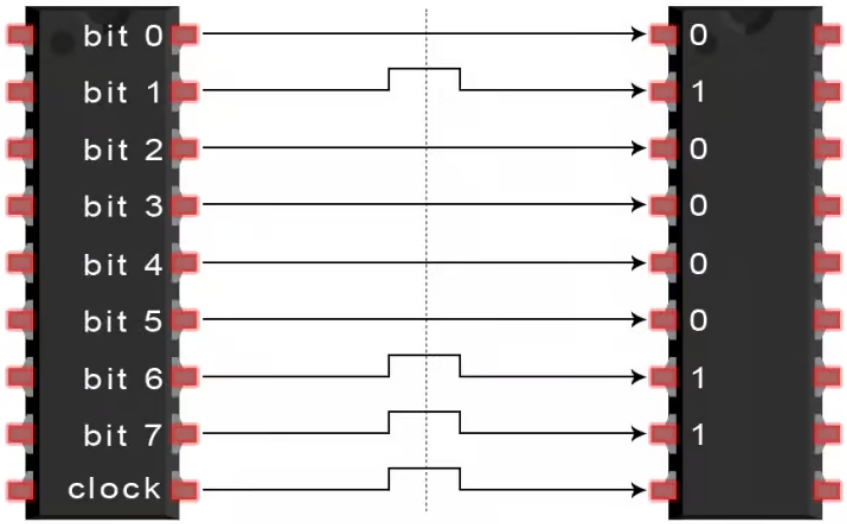


Figure 13.1: Parallel transmission of the letter “C” (01000011)

In serial communication, the bits are sent one by one through a single wire. The following diagram shows the serial transmission of the letter “C” in binary (01000011):

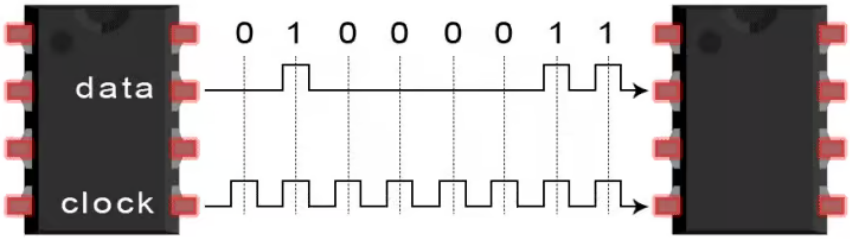


Figure 13.2: Serial transmission of the letter “C” (01000011)

8.3 Introduction to SPI Communication

SPI is a common communication protocol used by many different devices. For example, SD card reader modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers all use SPI to communicate with microcontrollers.

One unique benefit of SPI is the fact that data can be transferred without interruption. Any number of bits can be sent or received in a continuous stream. With I2C and UART, data is sent in packets, limited to a specific number of bits. Start and stop conditions define the beginning and end of each packet, so the data is interrupted during transmission.

Devices communicating via SPI are in a **master-slave** relationship. The master is the controlling device (usually a microcontroller), while the slave (usually a sensor, display, or memory chip) takes instruction from the master. The simplest configuration of SPI is a single master, single slave system, but one master can control more than one slave (more on this below).

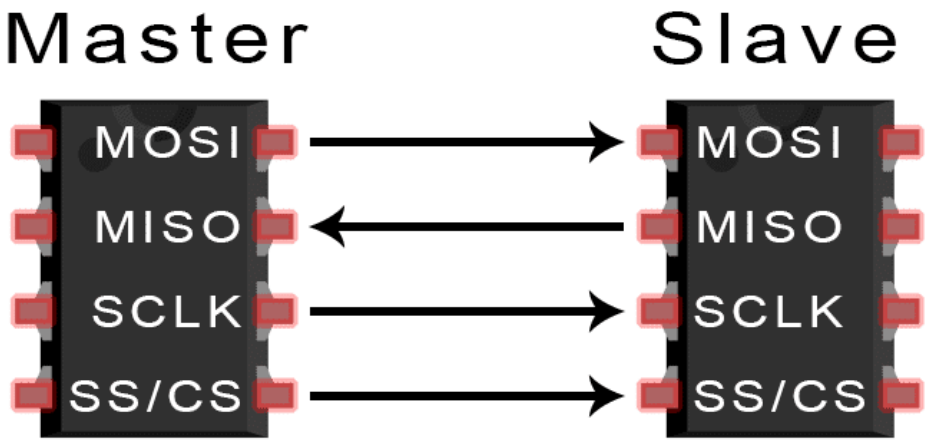


Figure 13.3: SPI communication between master and slave devices

SPI Lines:

- **MOSI (Master Output / Slave Input)** – Line for the master to send data to the slave.
- **MISO (Master Input / Slave Output)** – Line for the slave to send data to the master.
- **SCLK (Clock)** – Line for the clock signal.
- **SS/CS (Slave Select / Chip Select)** – Line for the master to select which slave to send data to.

Figure 13.4: Typical SPI bus wiring with MOSI, MISO, SCLK, and SS lines

Note: In practice, the number of slaves is limited by the load capacitance of the system, which reduces the ability of the master to accurately switch between voltage levels.

8.3.1 How SPI Works

The Clock
The clock signal synchronizes the output of data bits from the master to the sampling of bits by the slave. One bit of data is transferred in each clock cycle, so the speed of data transfer is determined by the frequency of the clock signal. SPI communication is always initiated by the master since the master configures and generates the clock signal.

Any communication protocol where devices share a clock signal is known as **synchronous**. SPI is a synchronous communication protocol. There are also **asynchronous** methods that don't use a clock signal. For example, in UART communication, both sides are set to a pre-configured baud rate that dictates the speed and timing of data transmission.

The clock signal in SPI can be modified using the properties of **clock polarity** and **clock phase**. These two properties work together to define when the bits are output and when they are sampled. Clock polarity can be set by the master to allow for bits to be output and sampled on either the rising or falling edge of the clock cycle. Clock phase can be set for output and sampling to occur on either the first edge or second edge of the clock cycle, regardless of whether it is rising or falling.

Slave Select
The master can choose which slave it wants to talk to by setting the slave's CS/SS line to a low voltage level. In the idle, non-transmitting state, the slave select line is kept at a high voltage level. Multiple CS/SS pins may be available on the master, which allows for multiple slaves to be wired in parallel. If only one CS/SS pin is present, multiple slaves can be wired to the master by daisy-chaining.

Multiple Slaves
SPI can be set up to operate with a single master and a single slave, or with multiple slaves controlled by a single master. There are two ways to connect multiple slaves to the master:

- **Parallel wiring using multiple CS/SS pins:**

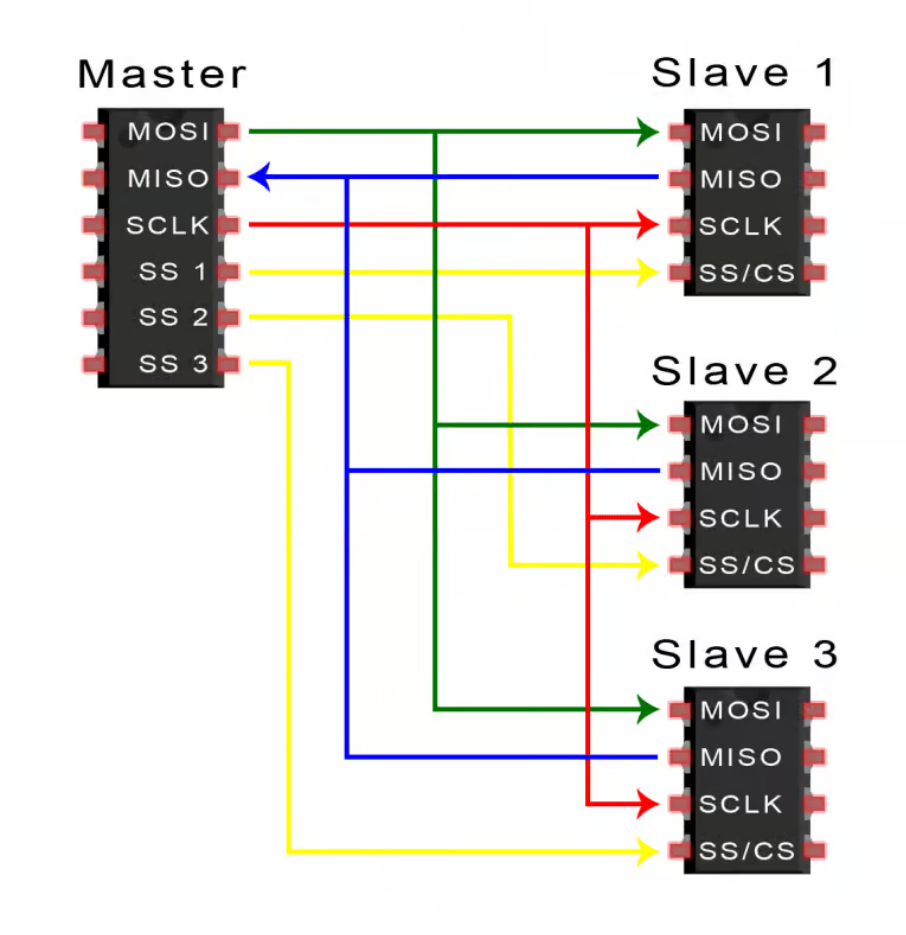


Figure 13.5: Multiple slaves connected in parallel with separate CS/SS lines

- **Daisy-chaining with a single CS/SS pin:**

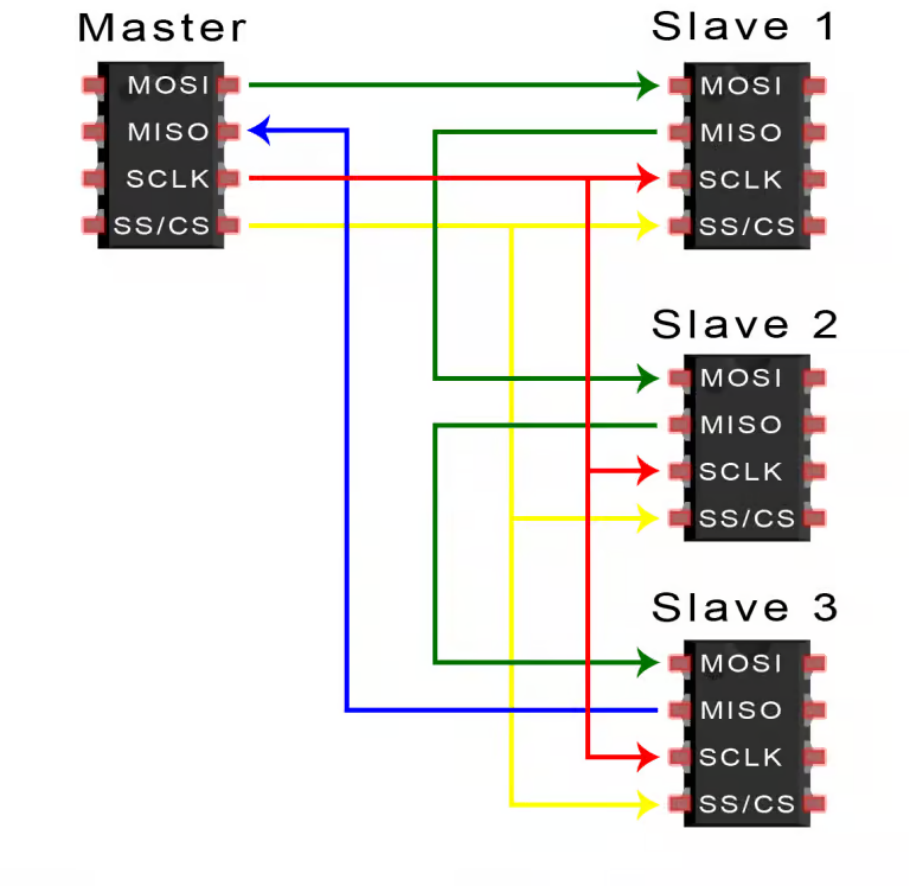


Figure 13.6: Multiple slaves connected in series using daisy-chaining

MOSI and MISO

The master sends data to the slave bit by bit, in serial through the **MOSI (Master Out Slave In)** line. The slave receives the data sent from the master at the MOSI pin. Data sent from the master to the slave is usually sent with the *most significant bit (MSB)* first.

The slave can also send data back to the master through the **MISO (Master In Slave Out)** line in serial. The data sent from the slave back to the master is usually sent with the *least significant bit (LSB)* first.

8.1.2: Steps of SPI Data Transmission

(a) **The master outputs the clock signal:**

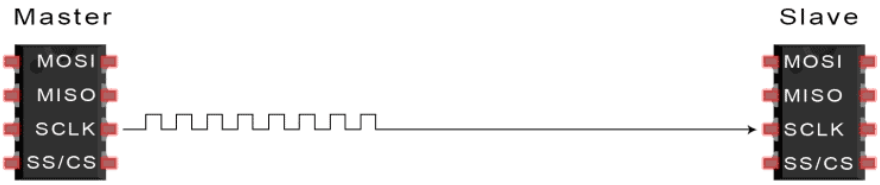


Figure 13.7: Clock signal generated by the master

(b) **The master switches the SS/CS pin to a low voltage state, which activates the slave:**

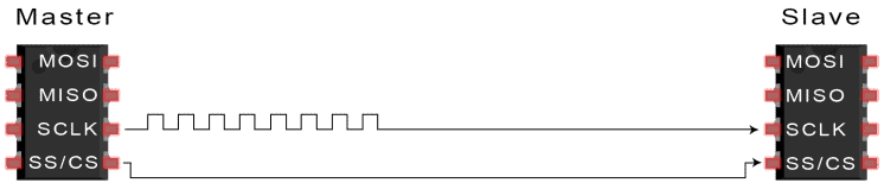


Figure 13.8: Slave Select (CS) line pulled low to activate the slave

(c) **The master sends the data one bit at a time to the slave along the MOSI line. The slave reads the bits as they are received:**

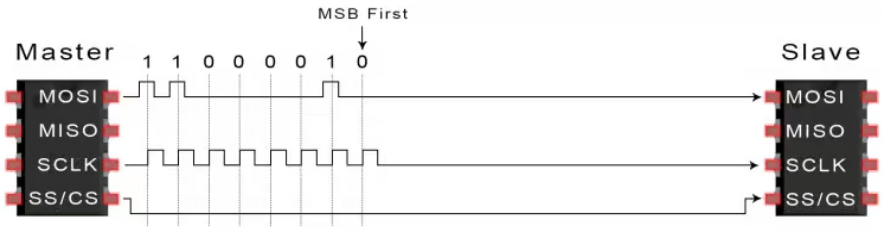


Figure 13.9: Data transmission from master to slave via MOSI

(d) **If a response is needed, the slave returns data one bit at a time to the master along the MISO line. The master reads the bits as they are received:**

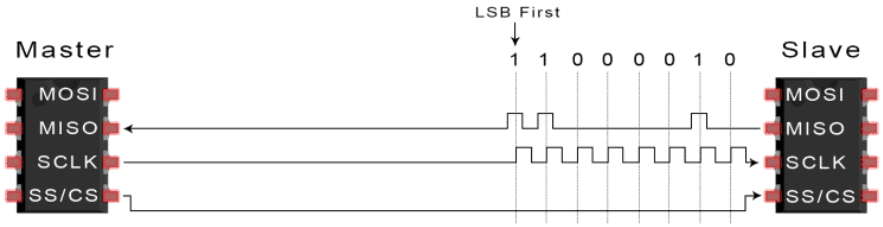


Figure 13.10: Data response from slave to master via MISO

8.3.2 The 4 SPI Modes

There are four possible clocking configurations of CPOL and CPHA, often referred to as **SPI modes**:

- **Mode 0** (CPOL = 0, CPHA = 0): CLK idle state = low, data sampled on rising edge and shifted on falling edge.
- **Mode 1** (CPOL = 0, CPHA = 1): CLK idle state = low, data sampled on falling edge and shifted on rising edge.
- **Mode 2** (CPOL = 1, CPHA = 0): CLK idle state = high, data sampled on falling edge and shifted on rising edge.
- **Mode 3** (CPOL = 1, CPHA = 1): CLK idle state = high, data sampled on rising edge and shifted on falling edge.

8.3.3 Advantages and Disadvantages of SPI

There are some advantages and disadvantages to using SPI, and if given the choice between different communication protocols, you should know when to use SPI according to the requirements of your project:

Advantages

- No start and stop bits, so the data can be streamed continuously without interruption.
- No complicated slave addressing system like I2C.
- Higher data transfer rate than I2C (almost twice as fast).
- Separate MISO and MOSI lines, so data can be sent and received at the same time.

Disadvantages

- Uses four wires (I2C and UART use only two).
- No acknowledgement that the data has been successfully received (I2C has this).
- No form of error checking like the parity bit in UART.
- Only allows for a single master.

8.4 Configuring SPI Using STM32CubeMX

The STM32F3 Discovery board includes an SPI-compatible gyroscope sensor (L3GD20) that communicates with the microcontroller via `SPI1`. To enable communication with such devices, SPI must be correctly configured using STM32CubeMX, which abstracts register-level settings into a graphical user interface.

8.4.1 Procedure: SPI Configuration in STM32CubeMX

(a) **Enabling SPI Peripheral**

- Open STM32CubeMX and create a new project for the `STM32F303VCT6` microcontroller (used on the STM32F3 Discovery board).
- In the *Pinout & Configuration* tab, locate and enable the `SPI1` peripheral.
- Set SPI Mode to **Full Duplex Master**.
- The following pins will be automatically assigned:
  - `PA5` – SCK (Serial Clock)
  - `PA6` – MISO (Master In Slave Out)
  - `PA7` – MOSI (Master Out Slave In)
  - `PE3` – CS/SS (Chip Select / Slave Select)

8.4.2 SPI Configuration Parameters

Parameter	Description
Mode	Selects the role of the SPI peripheral: Master or Slave.
Direction	Defines the data flow: Full Duplex, Half Duplex, or Simplex.
Data Size	Typically 8 bits; determines the size of each SPI frame.
First Bit	Transmission order: Most Significant Bit (MSB) or Least Significant Bit (LSB).
Clock Polarity (CPOL)	Defines the idle state of the clock line (Low or High).
Clock Phase (CPHA)	Specifies whether data is captured on the first or second clock edge.
NSS Signal Type	Determines how the Chip Select (CS) is managed: Hardware or Software control.
Baud Rate Prescaler	Divides the APB clock to set the SPI clock speed (e.g., ÷8, ÷16).
CRC Calculation	Enables/disables Cyclic Redundancy Check for error detection (typically disabled).

Table 13.1: SPI Configuration Parameters in STM32CubeMX

8.4.3 Recommended Configuration for L3GD20 or I3G4250D Gyroscope Sensor

Setting	Value
Mode	Master
Direction	2 Lines (Full Duplex)
Data Size	8-bit
First Bit	MSB First
Clock Polarity (CPOL)	High (CPOL = 1)
Clock Phase (CPHA)	2nd Edge (CPHA = 1)
Baud Rate Prescaler	8 or 16 ($\leq$ 10 MHz for sensor compatibility)
NSS Signal Type	Software (NSS disabled)
CRC Calculation	Disabled
NSSP Mode	Disabled

Table 13.2: Recommended SPI Settings for L3GD20 or I3G4250D Gyroscope

8.4.4 Chip Select (NSS) Pin Handling

In the STM32F3 Discovery board, the `L3GD20` gyroscope is connected to the `SPI1` interface. The Chip Select (CS) line for the gyroscope is connected to pin `PE3`.

When `SPI1` is enabled in STM32CubeMX, `PE3` is automatically assigned as the NSS (CS) pin. However, because the gyroscope requires manual control of CS, this pin must be handled in software.

No additional configuration is needed in CubeMX for `PE3`, but you must use it as a **GPIO Output** in code.

Basic GPIO Functions Used:

- Pull CS Low (start communication):

```
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);

```

- Pull CS High (end communication):

```
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);

```

8.4.5 SPI Communication Modes

STM32 HAL offers two primary modes for SPI data transfer:

Mode	Description
Polling	Blocking, waits until the operation completes.
Interrupt	Non-blocking, uses callbacks when complete.

Table 13.3: SPI Communication Modes

These modes can be selected based on the complexity and real-time requirements of the application.

Polling Mode:
In Polling mode, the microcontroller waits (or ”polls”) for the SPI transfer to finish before proceeding. This method is simple and often sufficient for basic applications.

CubeMX Settings:

- SPI1 Mode: Full Duplex Master
- NSS Signal Type: Software
- No need to enable SPI interrupts in NVIC

Basic SPI Functions Used:

- Transmit:

```
HAL_SPI_Transmit(&hspi1, tx_buffer, size_in_bytes, timeout);

```

- Receive:

```
HAL_SPI_Receive(&hspi1, rx_buffer, size_in_bytes, timeout);

```

Interrupt Mode:
In Interrupt mode, the SPI operations are handled in the background. Once a transfer is triggered, the processor can perform other tasks until an interrupt signals completion.

CubeMX Settings:

- Same as polling mode, plus:
- Enable SPI1 global interrupt under NVIC Settings

Basic SPI Functions Used:

- Transmit:

```
HAL_SPI_Transmit_IT(&hspi1, tx_buffer, size_in_bytes);

```

- Receive:

```
HAL_SPI_Receive_IT(&hspi1, rx_buffer, size_in_bytes);

```

Callback Functions Required:

- Transmission Complete:

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi);

```

- Reception Complete:

```
void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi);

```

These callbacks are automatically invoked by the HAL when the corresponding SPI event completes.

8.5 Task 1: Reading the WHO_AM_I Register from the I3G4250D Gyroscope Sensor

When communicating with SPI-based sensors, each internal register is accessed by sending a specific register address along with a read/write bit in the command byte. The WHO_AM_I register of the gyroscope contains a predefined identifier used to verify communication with the device.

Depending on the STM32F3 Discovery board revision, the on-board gyroscope may be either the I3G4250D (blue board) or the L3GD20 (green board). The procedure is identical; only expected IDs differ.

Instructions:

(a) Download and open the official datasheet for the I3G4250D gyroscope:

https://www.st.com/resource/en/datasheet/i3g4250d.pdf

(b) Locate the WHO_AM_I register in the Register Description table. You will find:

- Register Name: WHO_AM_I
- Its address in hexadecimal
- Its default value in binary

(c) Configure your SPI communication to read from this register.

(d) After reading, verify the return value via UART.

Understanding the Read Command Format:

When communicating with the gyroscope over SPI, the first byte sent must contain:

- Bit 7 = 1 for Read operation
- Bit 6 = 1 to enable Auto-increment (optional for multi-byte reads)
- Bits [5:0] = Register address

Since we are only reading one register, auto-increment is not strictly necessary.

Example: To read from register address 0x0F:

- Set bit 7 (MSB) to 1 → read mode
- Final command byte becomes:

```
// Bitwise OR operation:
0x80 | 0x0F = 0x8F

// Binary Representation:
0x80 = 10000000    // Read bit = 1
0x0F = 00001111    // Register address (e.g., 0x0F)
-------------------------
0x8F = 10001111    // Final command byte
```

This final byte `0x8F` tells the sensor: *"I want to read from the register address 0x0F."*

Implementation Guidelines:

- Use `HAL_SPI_Transmit()` to send the command byte.
- Use `HAL_SPI_Receive()` to read one byte of data.
- Control the Chip Select (PE3) line manually:
  - Set LOW before communication
  - Set HIGH after communication

8.6 Task 2: Reading Temperature from the Gyroscope Sensor in Interrupt Mode

Objective: In this task, you will read the internal temperature sensor data from the `I3G4250D` gyroscope using the SPI communication protocol in interrupt mode. The temperature data should be read continuously, transmitted via UART to your PC, and then plotted in real-time using the provided Python script. The Python plotting script will be provided on the LMS.

Instructions

STM32F3 Board	Gyroscope	Datasheet Link
Blue Board	I3G4250D	I3G4250D Datasheet
Green Board	L3GD20	L3GD20 Datasheet

Table 13.4: Gyroscopes used on STM32F3 Discovery boards

Determine:

- The register address corresponding to the temperature output.
- The expected format and range of values.

2. Enable SPI in Interrupt Mode

- Ensure `SPI1` is configured in interrupt mode using STM32CubeMX:
  - Enable the SPI1 global interrupt from NVIC settings.
  - Set SPI parameters (prescaler, mode, etc.) as done in Task 1.

- Configure PE3 as GPIO Output to manually handle CS.

3. Implement the SPI Interrupt Handlers

You must use the following callback functions:

- `HAL_SPI_TxCpltCallback()` – called after transmitting the register address.
- `HAL_SPI_RxCpltCallback()` – called after receiving the temperature value.

4. Transmit the Temperature via UART

- After successfully reading the temperature byte, convert it into a signed integer.
- Transmit it over UART using:

```
HAL_UART_Transmit(&huart2, &temperature, 1, HAL_MAX_DELAY);

```

5. Plot Temperature Using Python A Python script has been provided to visualize the live temperature data sent over UART. Use it to monitor the temperature readings and verify correct functionality.

Important: Sensor Initialization Code

Before reading any sensor values, the gyroscope must be properly powered on and its output axes enabled. This is done by writing to CTRL_REG1, where:

- Bit 3 (PD) enables active power mode.
- Bits 0–2 enable X, Y, and Z axes.

Code Example:

```
#define CTRL_REG1     0x20
#define CTRL_REG1_VAL 0b00001111  // PD=1 (Power on), Xen/Yen/Zen enabled

void gyro_init()
{
    uint8_t tx[2] = { CTRL_REG1, CTRL_REG1_VAL };

    // Pull CS low to begin communication
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);

    // Send control register configuration
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);

    // Pull CS high to end communication
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);
}
```

Note: You must call the above function once in `main()` outside the `while(1)` loop.

Expected Outcome:

- Your code should continuously read and transmit the temperature value via UART every 100–200 ms.
- The temperature should appear as a signed integer and be plotted in real-time using the Python script.
- **Proper use of interrupt-based SPI is required; polling mode will not be accepted for this task.**

8.7 Task 3: Reading Angular Velocity (X, Y, Z Axes) from the Gyroscope Sensor

In this task, you will read the angular velocity data from the X, Y, and Z axes of the `I3G4250D` gyroscope. These values, provided as raw 16-bit signed integers, must be converted into physical units of degrees per second (dps) using the sensor's sensitivity settings.

You will also read the internal temperature as in Task 2, and transmit all values via UART to be visualized using the provided Python plotting script.

Unlike Task 2, polling mode is acceptable for this task.

Sensor Initialization
As in the previous task, it is essential to enable the sensor by writing to the `CTRL_REG1` register:

```
#define CTRL_REG1     0x20
#define CTRL_REG1_VAL 0b10001111  // Power on, enable X, Y, Z axes

void gyro_init()
{
    uint8_t tx[2] = { CTRL_REG1, CTRL_REG1_VAL };
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);  // CS LOW
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);    // CS HIGH
}
```

Without this configuration, the sensor remains in power-down mode, and valid readings from the gyroscope or temperature sensor cannot be obtained.

Refer to the datasheet to understand what each register value bit represents and how it affects sensor behavior.

Configuring Sensitivity (Optional)
By default, the gyroscope operates at a full-scale range of ±245 dps, which corresponds to a sensitivity of 8.75 mdps/LSB.

To optionally configure sensitivity in the future:

```
#define CTRL_REG4     0x23
#define CTRL_REG4_VAL 0b00000000  // FS[1:0] = 00 => +/- 245 dps

void gyro_set_ctrl_reg4()
{
    uint8_t tx[2] = { CTRL_REG4, CTRL_REG4_VAL };
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);  // CS LOW
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);    // CS HIGH
}
```

Note: This step is optional in this lab, since default settings are used.

Reading Gyroscope Output Registers
You must read six registers to obtain angular velocity data from all three axes:

Axis	Low Byte	High Byte
X	OUT_X_L (0x28)	OUT_X_H (0x29)
Y	OUT_Y_L (0x2A)	OUT_Y_H (0x2B)
Z	OUT_Z_L (0x2C)	OUT_Z_H (0x2D)

Table 13.5: Gyroscope Output Register Addresses

Merging High and Low Bytes

Each axis value is stored in two 8-bit registers (low and high). Merge them to form a 16-bit signed integer:

```
int16_t x = (int16_t)((OUT_X_H << 8) | OUT_X_L);
```

Note: The gyroscope uses little-endian format — low byte first.

Converting to Degrees per Second (dps)

Using the sensitivity value for ±245 dps:

$$8.75 \text{ mdps/LSB} = 0.00875 \text{ dps/LSB}$$

Apply the conversion:

```
float x_dps = x_raw * 0.00875f;
float y_dps = y_raw * 0.00875f;
float z_dps = z_raw * 0.00875f;
```

Reading Temperature
You must also read the temperature register `OUT_TEMP` (0x26) as done in Task 2. The returned value is a signed 8-bit integer.

Output via UART
Transmit the values in the following comma-separated format:

```
<temp>, <x_dps>, <y_dps>, <z_dps>\n
```

Example Output:

```
25, 0.23, -1.74, 3.14
```

One line per reading will be parsed and plotted in real time using the provided Python script.

Enabling Floating-Point Support in `printf()`

By default, floating-point support is disabled in the standard `printf()` implementation to reduce code size. To enable printing of floating-point values (for example, angles and control variables), an additional linker option must be added to the project.

After generating the project and adding the required libraries, navigate to the following directory:

Open the `CMakeLists.txt` file located in the `cmake -> stm32cubemx` directory and add the following line:

```
target_link_options(${CMAKE_PROJECT_NAME} PRIVATE -u _printf_float)
```

Listing 13.1: Enabling floating-point support for `printf()`

Implementation Notes

- You may implement this using either:
  - Polling mode (recommended for simplicity)
  - Interrupt mode (more efficient, slightly more complex)
- Manually control CS pin (`PE3`):
  - Set `LOW` before each SPI transaction
  - Set `HIGH` after completion
- Add delay between reads:

```
HAL_Delay(100);

```

Expected Outcome

- Temperature and angular velocity values are read continuously.
- Values are converted to dps using the sensitivity factor.
- Output is transmitted via UART in a comma-separated format.
- The Python script plots values in real time.

8.8 Common SPI Functions for STM32

Function / Macro	Explanation
`HAL_SPI_Init()`	Initializes the SPI peripheral with user-defined settings (mode, baud rate, data size, etc.).
`HAL_SPI_Transmit()`	Sends data via SPI in blocking mode (polling). The function waits until transmission is complete.
`HAL_SPI_Receive()`	Receives data via SPI in blocking mode. The function waits until reception is complete.
`HAL_SPI_TransmitReceive()`	Performs full-duplex communication: transmits and receives data simultaneously in blocking mode.
`HAL_SPI_Transmit_IT()`	Sends data via SPI using interrupt mode (non-blocking).
`HAL_SPI_Receive_IT()`	Receives data using SPI interrupts.
`HAL_SPI_TransmitReceive_IT()`	Sends and receives SPI data simultaneously using interrupt mode.
`HAL_SPI_Transmit_DMA()`	Sends SPI data using Direct Memory Access (DMA) for non-blocking and efficient transfer.
`HAL_SPI_Receive_DMA()`	Receives SPI data using DMA.
`HAL_SPI_TransmitReceive_DMA()`	Full-duplex SPI transfer using DMA.
`HAL_SPI_IRQHandler()`	Handles SPI interrupts. Typically called inside the ISR defined in `stm32f3xx_it.c`.
`HAL_SPI_Abort()`	Aborts ongoing SPI transfer in any mode (blocking, interrupt, or DMA).
`HAL_SPI_DeInit()`	Deinitializes the SPI peripheral, resetting its configuration.
`__HAL_SPI_ENABLE()`	Macro to enable the SPI peripheral manually (e.g., after configuring CR1).
`__HAL_SPI_DISABLE()`	Disables the SPI peripheral. Often used before reconfiguring settings.

Table 13.6: Common SPI Functions Using STM32 HAL

8.9 Evaluation Question

(a) You read a WHO_AM_I value of 0x00 from the gyroscope sensor instead of the expected ID. List two possible causes for this issue.

8.10 Assessment Rubric

CLO; LDL	Task No.	LR2 – Code	LR5 – Results	LR9 – Report	LR7 – Viva
2	1	/10	/10	/5	–
	2	/15	/10		
	3	/15	/10		
	Evaluation Questions	–	–	–	/10
4	Git	–	–	/15	-
Total					**/100**

Lab 9: Integrate I²C Communication with the MPU-6050 or LSM303AGR for Sensor Data Acquisition

9.1 Objectives

This lab introduces the Inter-Integrated Circuit (I²C) communication protocol. You will learn how to configure the STM32F3 Discovery board to communicate with I²C-compatible sensors. In this lab, you can use either the MPU-6050 (located on the Keyestudio shield) or the LSM303AGR/ LSM303DLHCC (integrated on the STM32F303 Discovery board). The MPU-6050 includes a 3-axis accelerometer and a 3-axis gyroscope, while the LSM303AGR includes a 3-axis accelerometer and a 3-axis magnetometer.

Board / Shield	Sensor	Sensor Features
STM32F3 Discovery (Green Board)	LSM303DLHC	3-axis accelerometer and 3-axis magnetometer (I²C-compatible).
STM32F3 Discovery (Blue Board)	LSM303AGR	3-axis accelerometer and 3-axis magnetometer (I²C-compatible).
Keyestudio Motor Shield	MPU-6050	3-axis accelerometer and 3-axis gyroscope (I²C-compatible).

Table 14.1: I²C-compatible sensors available for this lab

- Understand how the I²C protocol works.
- Configure the STM32 microcontroller as an I²C master.
- Establish communication with one of the following sensors:
  - **MPU-6050 (Keyestudio Shield)**: Read `WHO_AM_I` register, accelerometer, and gyroscope data.
  - **LSM303AGR/LSM303DLHC(on-board STM32F303)**: Read `WHO_AM_I` register for accelerometer and magnetometer, and read accelerometer data.
- Transmit sensor data over UART.
- Use a complementary filter to calculate the angle (MPU-6050: accelerometer + gyroscope; LSM303AGR/LSM303DLHC: accelerometer only) and plot it.

9.2 Background

9.2.1 What is I²C?

I²C is a synchronous, multi-master, multi-slave, serial communication protocol. It uses two lines:

- **SDA** – Serial Data Line (for data transfer)
- **SCL** – Serial Clock Line (for synchronization)

Key Features:

- Two-wire communication

- Master-slave architecture
- Supports multiple devices using 7-bit or 10-bit addresses
- Requires fewer pins than SPI
- Moderate speed (typically up to 400 kHz in Fast Mode)

9.2.2 Difference Between I^2C and SPI Communication Protocols

1. Number of Wires

- **SPI** requires at least 4 wires: MOSI, MISO, SCK, and CS (per slave).
- **I^2C** uses only 2 wires: SDA (data) and SCL (clock), shared among all devices.

2. Master-Slave Architecture

- **SPI:** One master can communicate with multiple slaves, but each slave needs a dedicated Chip Select (CS) line.
- **I^2C:** Uses a unique 7- or 10-bit address for each slave, allowing communication over shared lines.

3. Communication Type

- **SPI:** Full-duplex (can send and receive simultaneously).
- **I^2C:** Half-duplex (data flows in one direction at a time).

4. Speed

- **SPI:** Typically faster; supports clock speeds up to tens of MHz.
- **I^2C:** Slower; standard mode is 100 kHz, fast mode is 400 kHz, and high-speed mode can go up to 3.4 MHz.

5. Complexity and Overhead

- **SPI:** Simple protocol, minimal overhead, no built-in acknowledgment or addressing.
- **I^2C:** More complex; supports acknowledgments, arbitration, and addressing, which increases robustness but also overhead.

6. Use Cases

- **SPI:** Used where speed is critical (e.g., display drivers, fast sensors, memory).
- **I^2C:** Used where simplicity and wiring efficiency are preferred (e.g., temperature sensors, RTCs, low-speed communication).

Summary Table

Feature	SPI	I^2C
Wires Required	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
Speed	Higher (up to 50+ MHz)	Moderate (up to 3.4 MHz)
Duplex Mode	Full-duplex	Half-duplex
Multi-slave Support	Yes (with separate CS lines)	Yes (with addressing)
Complexity	Simple	More complex
Hardware Overhead	Higher (more pins)	Lower (fewer pins)
Acknowledgments	No	Yes

9.2.3 The MPU-6050 Sensor

The MPU-6050 is a 6-axis motion tracking device that combines a 3-axis accelerometer and a 3-axis gyroscope on a single chip. It communicates over I^2C (or optionally SPI on MPU-6000).

Relevant Registers:

- `WHO_AM_I` (0x75): Used to verify communication with the device. It returns the fixed device ID (expected value: 0x68).
- `ACCEL_CONFIG` (0x1C): Used to configure the accelerometer's full scale range. Common settings include $\pm 2g$, $\pm 4g$, $\pm 8g$, and $\pm 16g$.
- `GYRO_CONFIG` (0x1B): Used to configure the gyroscope's full scale range. Common settings include $\pm 250^\circ/s$, $\pm 500^\circ/s$, $\pm 1000^\circ/s$, and $\pm 2000^\circ/s$.
- `ACCEL_XOUT_H` (0x3B): High byte of the X-axis accelerometer output.
- `GYRO_XOUT_H` (0x43): High byte of the X-axis gyroscope output.

9.2.4 The LSM303AGR/LSM303DLHC Sensor

The LSM303AGR is a 6-axis motion sensor that combines a 3-axis accelerometer and a 3-axis magnetometer in a single package. It communicates over the I^2C interface and is integrated directly on the STM32F303 Discovery board, eliminating the need for external wiring. **Relevant Registers:**

Register Name	Address (Hex)	Description
WHO_AM_I_A	0x0F	Accelerometer identification register. Returns a fixed device ID (expected value: 0x33).
CTRL_REG1_A	0x20	Controls the accelerometer data rate, power mode, and axis enable configuration.
CTRL_REG4_A	0x23	Configures the accelerometer operating mode (low-power, normal power, or high-resolution modes).
OUT_X_L_A	0x28	Low byte of the X-axis accelerometer output data.
OUT_X_H_A	0x29	High byte of the X-axis accelerometer output data.
OUT_Y_L_A	0x2A	Low byte of the X-axis accelerometer output data.
OUT_Y_H_A	0x2B	High byte of the X-axis accelerometer output data.

Table 14.2: Relevant accelerometer registers and their functions

9.3 Accelerometer Axis Orientation

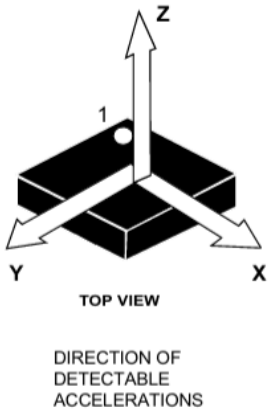


Figure 14.1: Direction of detectable accelerations for the accelerometer on the Blue STM32F3 Discovery board

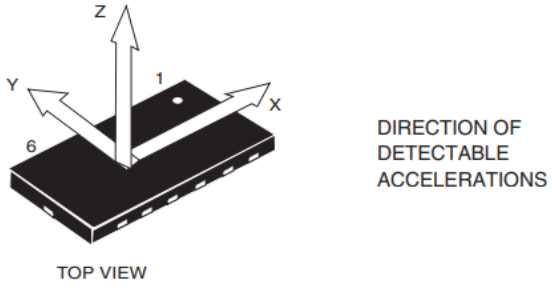


Figure 14.2: Direction of detectable accelerations for the accelerometer on the Green STM32F3 Discovery board

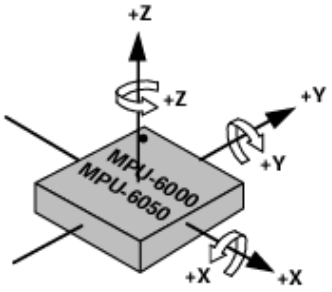


Figure 14.3: Direction of detectable accelerations for the accelerometer on the shield

9.4 Configuring I^{2}C in STM32CubeMX

- Enable the I^{2}C1 peripheral.
- Set SDA and SCL to appropriate GPIO pins with Alternate Function (AF4).
- Enable internal pull-up resistors if external ones are not connected.
- Optionally configure UART for debugging/printing.
- Some useful functions:

Function	Description
`HAL_I2C_Init(&hi2c)`	Initializes the I2C peripheral with parameters such as timing, addressing mode, and own address.
`HAL_I2C_Master_Transmit(...)`	Sends data from the master to the slave device using the 7-bit slave address.
`HAL_I2C_Master_Receive(...)`	Receives data from a slave device in master mode.
`HAL_I2C_Mem_Write(...)`	Writes data to a specific memory/register address on a slave (e.g., EEPROM, sensor register).
`HAL_I2C_Mem_Read(...)`	Reads data from a memory/register address on the slave device.
`HAL_I2CEx_ConfigAnalogFilter(...)`	Enables or disables the analog noise filter on the I2C lines.
`HAL_I2CEx_ConfigDigitalFilter(...)`	Configures digital noise filter (0–15) to filter spikes on SDA/SCL.
`HAL_I2C_IsDeviceReady(...)`	Checks if a device is ready/responding at a given address using retries and timeout.

Table 14.3: Common I2C Functions in STM32 HAL

9.5 Task 1: I^{2}C Communication with Selected Sensor

Objective: Interface the STM32F3 Discovery board with your chosen sensor (MPU-6050 on the Keyestudio Shield or the on-board LSM303AGR) over I^{2}C to read from the identification register(s) and verify the device ID.

Instructions:

(a) In STM32CubeMX:

- **MPU-6050:**
  - Under the Connectivity tab, select I^{2}C1 and configure it as I2C.
- **LSM303AGR:**
  - First, enable pins **PB6** (SCL) and **PB7** (SDA) in the Pinout view, then under the Connectivity tab select I^{2}C1 and configure it as I2C.
- Enable UART for debugging (e.g., USART1).

(b) Hardware Setup:

- **MPU-6050 (Keyestudio Shield)**:
  - Connect the SCL and SDA lines according to the STM32CubeMX pinout.
  - Connect GND to board ground and VCC to 3.3V.
- **LSM303AGR (On-board STM32F303)**:
  - No additional wiring is required; the sensor is already connected to I^{2}C1 internally.

(c) In your code:

- **MPU-6050:**

  - Device address: `0x68 << 1` (shift the address by one when reading or writing from it).
  - Use `HAL_I2C_Mem_Read()` to read from `WHO_AM_I` (0x75).
  - Expected output: `0x68`.
- **LSM303AGR:**
  - Device address: `0x33`.
  - Use `HAL_I2C_Mem_Read()` to read from `WHO_AM_I` (0x0F).
  - Expected output: `0x33`.
- Format and print the results over UART.

9.6 Task 2: Print Accelerometer and Gyroscope Values.

Objective: Interface the STM32F3 Discovery board with the MPU-6050 or LSM303AGR sensor over I^2 C to read the values of the accelerometer and gyroscope and print them.

9.6.1 MPU-6050

Note:

- For the MPU-6050 address is shifted by 1 bit when reading or writing from it.

Instructions:

(a) In STM32CubeMX:

- Under the Connectivity tab, select I^2C1 and configure it as I2C.
- Enable UART for debugging (e.g., USART1).

(b) Hardware Setup:

- Connect the SCL and SDA lines using the pinout view in STM32CubeMX.
- Connect the MPU-6050 ground to the board's ground, and VCC to 3.3V.

(c) In your code:

- This task requires you to implement four key functions:
  - `Init_MPU`: Initializes the MPU-6050 by writing appropriate values to its power, accelerometer, and gyroscope configuration registers.
  - `Read_MPU`: Reads raw accelerometer and gyroscope data from the sensor, scales them to meaningful physical units, subtracts offset values, and converts acceleration from radians to degrees for angle estimation.
  - `Print_MPU`: Sends the processed accelerometer, gyroscope, and optionally angle data to a serial terminal over UART, formatted as comma-separated values.
  - `Offset_MPU`: Calculates and stores the offset values for both accelerometer and gyroscope by averaging multiple stationary readings.
- `Init_MPU`:
  - Wakes up the MPU-6050 by writing `0x00` to the `PWR_MGMT_1` `0x6B` register.
  - Sets the accelerometer full-scale range to $\pm 2g$ by writing `0x00` to the `ACCEL_CONFIG` register.
  - Sets the gyroscope full-scale range to $\pm 250^\circ/s$ by writing `0x00` to the `GYRO_CONFIG` register.
- `Read_MPU`:

  - To read sensor values, both the high and low bytes of each axis register must be read.
  - These two bytes are combined as a signed 16-bit integer using: `raw = (high << 8) | low`.
  - Raw accelerometer values must be divided by `16384.0` to convert to acceleration in g units (for $\pm 2g$).
  - Raw gyroscope values must be divided by `131.0` to convert to angular velocity in degrees per second (for $\pm 250^\circ/s$).
  - Accelerometer values used for tilt estimation should be converted from radians to degrees using the constant `RAD_TO_DEG = 57.2958`.
  - Offset values should be subtracted from the scaled readings to correct bias in the sensor.
- `Print_MPU`:
  - The final processed values (e.g., accelerometer X, gyroscope X) should be printed as comma-separated values via UART.
  - Example output: `1.25,-0.45`
  - This format would be later helpful in plotting the values in next task.
- `Offset_MPU` (Calibration):
  - Place the sensor on a flat, stationary surface to ensure no movement.
  - Take multiple readings (e.g., 20 samples) of raw accelerometer and gyroscope data.
  - Convert the raw values to physical units using the same scaling factors as in `Read_MPU`.
  - Accumulate the scaled values and divide by the number of samples to calculate the average offset.
  - Store these average values as `acc_offset` and `gyro_offset`.
- A `struct` is used to store:
  - Raw and scaled accelerometer and gyroscope values.
  - Calculated offset values for calibration.
- In `main.c`:
  - Initialize the MPU-6050 data structure
  - Call the initialization function
  - Call the offset calibration function
  - Inside the `while(1)` loop,
    - Read accelerometer and gyroscope data.
    - Display the readings via UART in comma-separated format.
    - Add a delay of 100ms.

9.6.2 LSM303AGR

Note:

- Since the sensor only includes an accelerometer and magnetometer, to obtain gyroscope values refer to `Lab08`; you can integrate the initialization and reading processes with the accelerometer functions.
- For the LSM303AGR/LSM303DLHC, use `0x33` for read operations and `0x32` for write operations as the device address.

Command	SAD[7:1]	R/W	SAD+R/W
Read	0011001	1	00110011 (33h)
Write	0011001	0	00110010 (32h)

Figure 14.4: Read/Write Addresses for LSM303AGR and LSM303DLHC

Note: The STM32 HAL expects the 7-bit address left-shifted by one bit. Therefore, `0x32` (write) and `0x33` (read) correspond to the same 7-bit device address.

Instructions:

(a) In STM32CubeMX:

- Assign PB6 as I2C1_SCL and PB7 as I2C1_SDA.
- Under the Connectivity tab, select I^2C1 and configure it as I2C.
- Enable UART for debugging (e.g., USART1).

(b) Hardware Setup:

- The LSM303AGR is embedded on the STM32F303 board; no external wiring is required.

(c) In your code:

- This task requires you to implement four key functions:
  - `Init_LSM`: Initializes the LSM303AGR by writing appropriate values to its CTRL_REG1_A, CTRL_REG4_A registers.
  - `Read_LSM`: Reads raw accelerometer from the sensor, scales them to meaningful physical units, subtracts offset values, and converts acceleration from radians to degrees for angle estimation.
  - `Print_LSM`: Sends the processed accelerometer, gyroscope, and optionally angle data to a serial terminal over UART, formatted as comma-separated values.
  - `Offset_LSM`: Calculates and stores the offset values for both accelerometer and gyroscope by averaging multiple stationary readings.
- `Init_LSM`:
  - Wakes up the LSM303AGR accelerometer by writing `0x67` to the `CTRL_REG1_A` register.
  - Sets the accelerometer Normal mode by writing `0x00` to the `CTRL_REG4_A` register.
  - Use HAL_I2C_MemWrite() to write to the registers.

  For more information on register values, refer to the datasheet of the respective sensor.
- `Read_LSM`:
  - To read sensor values, both the high and low bytes of each axis register must be read using HAL_I2C_MemRead();
  - These two bytes are combined as a signed 16-bit integer using:
    `raw = (high << 8) | low`.
  - Raw accelerometer values must be multiplied by `3.9` (LSM303AGR) or `1.0` (LSM303DLHC) to convert them to acceleration in g units when operating in normal mode.
  - Accelerometer values used for tilt estimation should be converted from radians to degrees using the constant `RAD_TO_DEG = 57.2958`.
  - Offset values should be subtracted from the scaled readings to correct bias in the sensor.
- `Print_LSM`:
  - The final processed values (e.g., accelerometer X, gyroscope X) should be printed as comma-separated values via UART.
  - Example output: `1.25,-0.45`

  - This format would be later helpful in plotting the values in next task.

- `Offset_LSM` (Calibration):
  - Place the sensor on a flat, stationary surface to ensure no movement.
  - Take multiple readings (e.g., 20 samples) of raw accelerometer and gyroscope data.
  - Convert the raw values to physical units using the same scaling factors as in `Read_LSM`.
  - Accumulate the scaled values and divide by the number of samples to calculate the average offset.
  - Store these average values as `acc_offset` and `gyro_offset`.
- A `struct` is used to store:
  - Raw and scaled accelerometer and gyroscope values.
  - Calculated offset values for calibration.
- In `main.c`:
  - Initialize the LSM data structure
  - Call the initialization function
  - Call the offset calibration function
  - Inside the `while(1)` loop,
    * Read accelerometer and gyroscope data.
    * Display the readings via UART in comma-separated format.
    * Add a delay of 100ms.

9.7 Tips

- If you are not getting correct values, print the value of the `WHO_AM_I` register to verify communication.
- Tilt the sensor to observe changes in the accelerometer and gyroscope values.
- For the MPU-6050, ensure that all connections are properly wired and secure.
- Floating-point output using `printf` requires linking `_printf_float` in `CMakeLists.txt`.
- For further details, refer to the
  * MPU-6050 Datasheet.
  * LSM303AGR Datasheet
  * LSM303DLHC Datasheet

9.8 Evaluation Question

(a) Compare I^2C and SPI protocols in terms of speed, complexity, and use cases.

9.9 Assessment Rubric

CLO; LDL	Task No.	LR1 – Circuit Layout	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
3	1	/5	/15	/15	/5	-
	2	–	/20	/15		
	Evaluation Questions	–	–	–	–	/10
4	Git	–	–	–	/15	-
Total						**/100**

Lab 10: Investigate Dynamic Memory Management and Heap Allocation Techniques in Embedded C

10.1 Objectives

In this lab, students will explore stack vs. heap memory usage and gain hands-on experience with dynamic memory allocation in C. Students will write programs using `malloc()`, `realloc()`, and `free()`, while learning to avoid memory leaks and dangling pointers.

- Understand the difference between stack and heap memory.
- Use dynamic memory allocation and deallocation functions.
- Prevent memory leaks through proper resource management.
- Practice resizing memory dynamically.

10.2 Overview of Stack and Heap Memory

10.2.1 General Concept of Stack and Heap Memory

In the context of computer systems and embedded programming, memory is broadly divided into two main regions for dynamic allocation during program execution: **stack** and **heap**. Both serve different purposes and are managed differently by the system.

Stack Memory
The stack is a region of memory that stores temporary data related to function execution, such as:

- Local variables,
- Function parameters, and
- Return addresses.

It operates on the **Last In, First Out (LIFO)** principle. Whenever a function is called, a new ”stack frame” is pushed onto the stack, and it is popped off when the function returns. The stack is managed automatically by the compiler and has a fixed size, defined at the time of program startup.

Due to its predictable structure and low overhead, stack memory access is typically faster than heap access.

However, the stack is limited in size, and excessive use—especially through deep or recursive function calls—can lead to a **stack overflow**, which may crash the system or cause unpredictable behavior.

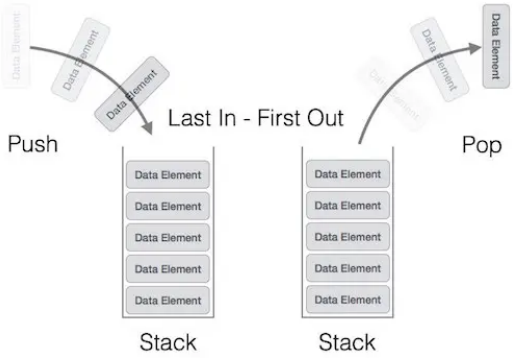


Figure 15.1: Stack (LIFO)

Heap Memory
The heap is a larger, dynamically managed memory region intended for variables that need to persist beyond the scope of a function.

Memory on the heap must be:

- Explicitly allocated (e.g., using `malloc` in C), and
- Explicitly freed (e.g., using `free`).

The heap provides flexibility but comes at the cost of:

- Slower access time,
- Fragmentation risks, and
- Greater potential for memory leaks if allocations are not properly managed.

Efficient use of the heap is essential in long-running applications or those requiring dynamic data structures (e.g., linked lists, trees, buffers).

10.2.2 Stack and Heap in STM32F3 Microcontrollers

On microcontrollers such as the `STM32F3VCT6`, memory resources are considerably more constrained compared to desktop systems, which necessitates a careful approach to memory management.

The `STM32F3VCT6` microcontroller features:

- 256 KB Flash memory (for storing program code), and
- 40 KB SRAM (for run-time data, including stack and heap).

Stack in STM32F3
The stack in STM32 microcontrollers is typically initialized at the top of the SRAM. On system reset, the **Main Stack Pointer (MSP)** is loaded from the vector table and used to manage stack operations unless the **Process Stack Pointer (PSP)** is explicitly configured (e.g., for an RTOS).

- Stack size is defined in the linker script (typically `startup_stm32f3xx.s` or `.ld` files).
- Stack size can be adjusted depending on the application requirements.
- Stack overflows are difficult to detect in bare-metal systems without proper safeguards and can overwrite adjacent memory, including the heap.

Heap in STM32F3
The heap resides just above the statically allocated global and static variables, growing upward in memory.

- The size and starting address of the heap are also defined in the linker script.
- Embedded developers often implement or use minimal custom allocators since standard `malloc` implementations are not always efficient or even included by default.
- In real-time systems, dynamic memory allocation on the heap is sometimes avoided altogether due to fragmentation risks and non-deterministic timing.

Important: Unlike high-level platforms, embedded systems like the STM32F3 do not come with an operating system by default. Therefore, the responsibility for stack/heap configuration, overflow prevention, and memory cleanup lies entirely with the programmer or the runtime support provided by the toolchain.

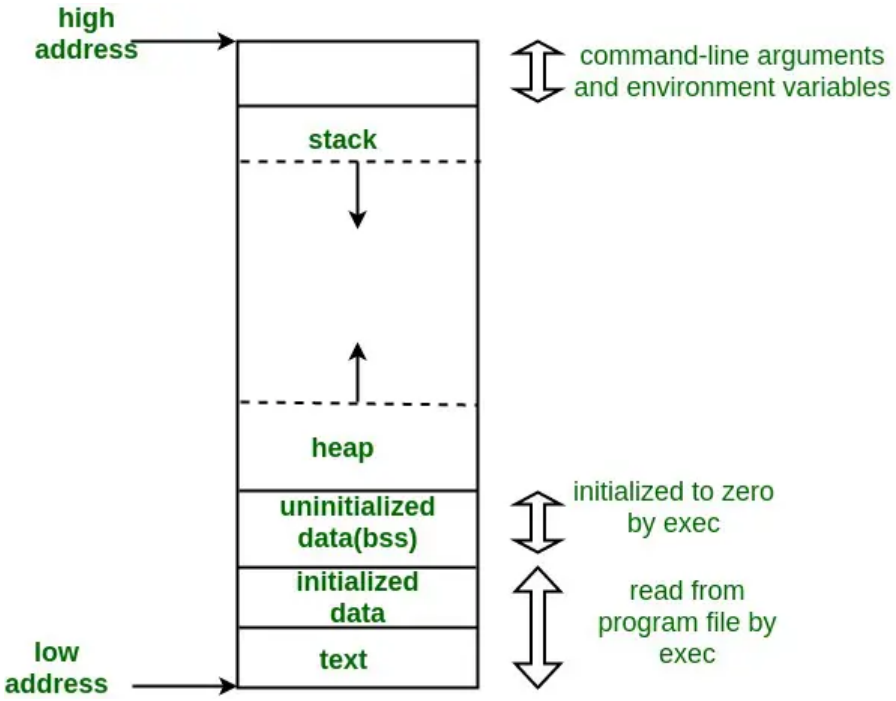


Figure 15.2: Stack and Heap

10.3 Dynamic Memory Allocation in Embedded Systems

10.3.1 What is Dynamic Memory Allocation

Dynamic memory allocation is the process of assigning memory at runtime for data whose size may not be known at compile time. Unlike static memory allocation (for global or local variables), dynamic allocation enables greater flexibility in how a program handles memory. This is especially useful when working with data structures like variable-length arrays, linked lists, or buffers whose size depends on user input or changing conditions during execution.

In C programming, dynamic memory is allocated from a special memory region called the **heap**, and the memory must be manually freed when it is no longer needed. This allows developers to reuse memory and manage memory usage efficiently—if done carefully.

10.3.2 Dynamic Memory Allocation in C

In C, dynamic memory allocation is handled through the standard library <stdlib.h> using the following key functions:

Function	Description
`malloc(size_t size)`	Allocates a block of memory of the specified `size` in bytes. It returns a pointer to the beginning of the block, or `NULL` if the allocation fails. The memory is uninitialized, meaning it contains garbage values. Example: `int* arr = (int*) malloc(5 * sizeof(int));`
`calloc(size_t num, size_t size)`	Allocates memory for an array of `num` elements, each of `size` bytes, and initializes all bits to zero. It returns a pointer to the allocated memory or `NULL` on failure. Example: `int* arr = (int*) calloc(5, sizeof(int));`
`realloc(void* ptr, size_t size)`	Changes the size of the memory block pointed to by `ptr` to `new_size` bytes. The contents will be preserved up to the lesser of the old and new sizes. It may return a new pointer; if `NULL` is returned, the original memory is not freed. It is safer to use a temporary pointer when calling `realloc()` to avoid memory loss. Example: `int* temp = realloc(arr, 10 * sizeof(int));`
`free(void* ptr)`	Releases memory previously allocated using `malloc()`, `calloc()`, or `realloc()`. Calling `free()` on a `NULL` pointer has no effect. After freeing, always set the pointer to `NULL` to avoid accidental use of a dangling pointer. Example: `free(arr); arr = NULL;`

Table 15.1: Standard C Dynamic Memory Allocation Functions

Example:

```c
int* buffer = (int*) malloc(100 * sizeof(int)); // allocate memory for 100 integers
if (buffer != NULL) {
    // Use the buffer...
    free(buffer); // Always free dynamically allocated memory
}
```

Important: Always check the return value of `malloc()` or `calloc()`. If memory allocation fails, these functions return `NULL`.

10.3.3 How It Works on STM32F3 Microcontrollers

Microcontrollers like the `STM32F3VCT6` have a constrained memory model, without an operating system to manage resources. When a program running on STM32 allocates memory dynamically using `malloc()` or `calloc()`, the memory is carved out from the SRAM—specifically, from a designated heap region within the SRAM.

SRAM Memory Map (STM32F3VCT6):

- Total SRAM: 40 KB (Address Range: `0x2000_0000` to `0x2000_9FFF`)
- Heap location: Starts just after the static variables (`.data` and `.bss`), and grows upward in memory.
- Stack location: Starts from the top of SRAM and grows downward.

This setup means:

- Memory allocated dynamically comes from a limited pool shared with the rest of the application (including the stack).
- If dynamic allocation is excessive or untracked, it can cause **heap-stack collision**, leading to unpredictable behavior or system crashes.

Behavior of Allocation Functions:

- `malloc()` looks for a free block of memory of the requested size within the heap. If found, it marks it as used and returns its address.
- `free()` releases a previously allocated block back to the heap, allowing future reuse.

These operations are handled internally by the runtime, and their success depends on the available free space in SRAM at that time.

10.3.4 Best Practices in Embedded Context

Because of the limited memory and lack of built-in protection, embedded developers must follow best practices when using dynamic memory allocation:

- **Minimize Usage:** Use `malloc()` only when absolutely necessary. Prefer static or stack memory when possible.
- **Always Free Memory:** Every `malloc()` or `calloc()` must have a corresponding `free()` once the memory is no longer needed.
- **Check for NULL:** Always check whether the allocation was successful before using the memory.
- **Avoid Fragmentation:** Repeated allocations and deallocations can fragment the heap over time, making it harder to find large blocks of memory.
- **Monitor Usage:** Use debugging tools (such as STM32CubeIDE's memory viewer) to monitor heap and stack usage.

10.4 Memory Leaks and How to Prevent Them

10.4.1 What is a Memory Leak?

A memory leak occurs when dynamically allocated memory is not properly deallocated, resulting in permanent loss of that memory during the program's execution. This typically happens when memory is allocated using `malloc()` or `calloc()`, but the pointer to that memory is either:

- Lost (e.g., overwritten), or
- Never passed to `free()`.

In embedded systems with limited RAM, even a small memory leak can cause heap exhaustion, system instability, or application failure over time.

10.4.2 Why Memory Leaks Are Critical in Embedded Systems?

Unlike desktop or server applications, microcontrollers do not have an operating system or a background garbage collector to reclaim unused memory. Once memory is leaked:

- It remains allocated until a reset or power cycle.
- The system may eventually run out of heap space.
- Stack and heap may collide, leading to undefined behavior.

Because RAM is typically measured in kilobytes on STM32 (e.g., 40 KB on STM32F3VCT6), preventing memory leaks is essential to system reliability.

10.4.3 Common Causes of Memory Leaks

Cause	**Example**	**Description**
Not calling `free()`	`ptr = malloc(100);` but never calling `free(ptr);`	Allocating memory without releasing it.
Pointer overwrite	`ptr = malloc(100); ptr = malloc(50);`	Assigning a new value to a pointer before freeing the old one — the first block is leaked.
Losing reference	Passing a pointer to a function and not storing or freeing it	Data is allocated and forgotten.
Improper error handling	Returning from a function before calling `free()`	Memory is allocated but not freed in error paths.

Table 15.2: Common Causes of Memory Leaks

10.4.4 Detecting Memory Leaks on STM32

Because bare-metal STM32 systems lack tools like Valgrind or OS logging, detecting memory leaks requires manual methods:

(a) **Monitor Heap Usage:**

- Use STM32CubeIDE or GDB memory viewer to inspect heap.
- Set breakpoints and inspect if the heap pointer keeps increasing after multiple runs.
- Use semihosting or UART to print heap usage at runtime.

(b) **Add Runtime Checks:** Maintain a global counter or tracking list to monitor active allocations:

```c
static size_t allocation_count = 0;

void* my_malloc(size_t size) {
    allocation_count++;
    return malloc(size);
}
void my_free(void* ptr) {
    if (ptr) allocation_count--;
    free(ptr);
}
```

You can then print `allocation_count` to check for mismatches.

10.4.5 How to Prevent Memory Leaks

Strategy	Explanation
Always call `free()`	Release memory as soon as it's no longer needed.
Initialize pointers to `NULL`	Helps in checking whether memory was allocated.
Set pointers to `NULL` after `free()`	Prevents accidental double-free or use-after-free bugs.
Use local variables where possible	Avoid dynamic memory unless necessary.
Use static buffers for fixed-size needs	Prefer static allocation if data size is known.
Use custom wrappers	Track allocation/deallocation centrally for debugging.
Avoid unnecessary `malloc()`	Reuse memory when possible or allocate once at startup.

Table 15.3: Best Practices to Prevent Memory Leaks

10.5 Task 1: Dynamic Memory Allocation Using Standard C Functions

Objective:
To understand and apply dynamic memory allocation in embedded systems by using standard C functions (`malloc()`, `calloc()`, and `free()`) to allocate and manage memory on the STM32F3Discovery board.

Task Description:
In this task, you will write a C program that dynamically allocates and uses memory on the heap using both `malloc()` and `calloc()`. A fixed number of elements will be allocated for each array, and you will observe the behavior of the allocated memory before and after initialization.

Steps:

(a) Define a constant `n` representing the number of integer elements to allocate (e.g., `n = 10`).

(b) Dynamically allocate an integer array of size `n` using `malloc()`:

- Check whether the memory allocation was successful.
- If successful, initialize the array such that each element at index `i` contains the value `i * 2`.

(c) Dynamically allocate another integer array of size `n` using `calloc()`:

- Confirm that all elements are initially zero.
- Assign the value `i + 1` to each element at index `i`.

(d) Print the contents of both arrays using UART.

(e) Deallocate both arrays using `free()`.

(f) Set both pointers to `NULL` after deallocation to prevent dangling references.

(g) Display a final message indicating that memory has been successfully freed and pointers cleared.

Expected Outcome:

- Memory is dynamically allocated from the heap using both `malloc()` and `calloc()`.
- The arrays are initialized and their contents are correctly displayed.
- Memory is safely deallocated using `free()`, and pointers are reset to `NULL`.
- Students understand how heap memory is used and managed on the STM32F3 microcontroller.

Implementation Guidelines:

- Include the appropriate headers: `<stdlib.h>`, `<string.h>`, and any HAL/UART headers as needed.
- Always check the return value of `malloc()` and `calloc()` to ensure that memory was allocated successfully.
- Use a UART interface or debugging tool (e.g., STM32CubeIDE memory viewer) to examine the array contents.
- Ensure that the total size of allocated memory does not exceed the available heap space.

10.6 Task 2: Writing a Heap Memory Driver Using Direct SRAM Addressing

Overview:
In this task, you will implement a custom heap memory allocator for the STM32F3Discovery (STM32F303VCT6) by managing RAM directly via addresses. You will not use `malloc()`, `calloc()`, or any standard library allocators. Instead, you will write your own driver (`heap_driver.c`) guided by the provided header (`heap_driver.h`) and example application (`main.c`).

By the end of this task, you should be able to:

- Define and locate a heap region within SRAM using absolute addresses.
- Manage fixed-size memory blocks with a bitmap (`block_map`).
- Write and document three core functions: `heap_init()`, `heap_alloc()`, and `heap_free()`.
- Integrate your driver into the build system (CMake) and verify functionality via UART.

1. Project Structure:

```
ProjectRoot/
   CMakeLists.txt
   Core/
        Inc/
             heap_driver.h   <-- Provided to students
        Src/
            main.c        <-- Example "test harness" provided
           heap_driver.c   <-- Students implement this
    Drivers/
     ... (HAL, CMSIS, USB, etc.)
```

```
    ... (other CubeMX folders)
```

2. CMakeLists.txt Changes:
Locate the section where application sources are listed:

```
set(MX_Application_Src
    Core/Src/main.c
    Core/Src/heap_driver.c       # <-- Add this line
    Core/Src/stm32f3xx_it.c
    Core/Src/stm32f3xx_hal_msp.c
    Core/Src/sysmem.c
    Core/Src/syscalls.c
    startup_stm32f303xc.s
)
```

3. Header File (`heap_driver.h`): This file declares your functions. It is provided and commented for clarity.

Useful types:

- `size_t`: unsigned type for sizes (from `<stddef.h>`).
- `void*`: generic pointer to allocated memory.
- `uintptr_t`: integer type that can store a pointer for arithmetic.

4. `main.c` as Test Harness: You must not modify this file. It demonstrates usage of your driver.

Expected UART Output:

```
=== Custom Heap Driver (Direct SRAM) ===
Data in Block 1
Text from Block 2
Blocks freed.
```

5. Implementing `heap_driver.c`:
Key Parameters and Components:

- `HEAP_START_ADDR` = 0x20001000 (avoids overwriting `.data` and `.bss`)
- `HEAP_SIZE` = 4096 bytes (4 KB)
- `BLOCK_SIZE` = 16 bytes
- `BLOCK_COUNT` = 256
- `block_map[]` = array of `uint8_t`, 0 means free, 1 means used

Function Behavior:

- `heap_init()` — initializes the block map (all zeros)
- `heap_alloc(size_t size)`:
  - Calculates how many blocks are needed.
  - Scans for a contiguous sequence of free blocks.
  - If found, marks them and returns pointer. If not, returns `NULL`.
- `heap_free(void* ptr)`:

- Validates and aligns the pointer.
- Frees blocks by scanning until a free block is encountered.

6. Tips & Verification

- **UART Tracing:** Add debug prints inside your allocator to show progress.
- **Boundary Checks:** Ensure returned addresses stay within `[HEAP_START_ADDR, HEAP_START_ADDR + HEAP_SIZE)`.

10.7 Evaluation Questions

(a) What are the risks of using `malloc()` without checking for `NULL`?

(b) What happens when you try to allocate more memory than is available on the heap?

(c) How would you detect or debug a memory leak in your system?

10.8 Assessment Rubric

CLO; LDL	Task No.	LR2 – Program Code	LR5 – Results	LR9 – Report	LR7 – Viva
1	1	/20	/15	/5	-
	2	/20	/15		
	Evaluation Questions	–	–	–	/10
4	Git	–	–	/15	-
Total					**/100**

Lab 11: Open Ended Lab: Design and Validate a Feedback-Controlled Motor Speed System

11.1 Objectives

In this lab, students must independently determine how to compute the robot's tilt angle and implement a PID-based feedback controller using previously learned embedded system concepts. The lab is intentionally open-ended, with no guided instructions. Students are expected to demonstrate initiative, reasoning, and problem-solving skills throughout the implementation.

11.2 Task 1: Angle Estimation using IMU

Objective: Design and implement a sensor-based tilt angle estimation approach using accelerometer and gyroscope measurements.

What is a complementary filter?
A complementary filter is a technique used to calculate the angle using gyroscope and accelerometer readings. Gyroscope provides short term changes in angle whereas accelerometer provides stable long term tilt based on gravity. The filter uses both these values through weighted average, favoring gyro over accelerometer to calculate a stable and accurate angle. For additional background on accelerometer-based inclination sensing and its complementary use with gyroscope measurements, refer to the following resources:

- Using an Accelerometer for Sensing
- Complementary Filter Reference Document

Instructions:

(a) **For plotting:**

- Download the `sensorplot.py` script from the LMS.

(b) **In your code:**

- Compute the tilt angle using the equation below. You are required to independently determine the appropriate sensor axis and modify the equation accordingly.

$$\text{angle} = 0.98 \times (\text{angle} + \text{gyro}_x \times dt) + 0.02 \times \text{acc}_x$$

  where:

  - `gyro_x` is the angular rate obtained from the gyroscope (in degrees per second),
  - `acc_x` is the tilt angle estimated from the accelerometer (in degrees),
  - `dt` is the sampling interval (in seconds).

- In your print function, print the angle in a comma-separated format (e.g., `1.25,-0.45,23`).
- Ensure that the value of `dt` matches the delay in your `while` loop (e.g., for a 100 ms delay, `dt = 0.1`).
- Build and flash the code to the board. Afterward, run the Python script. A plotting window should open displaying the live data.

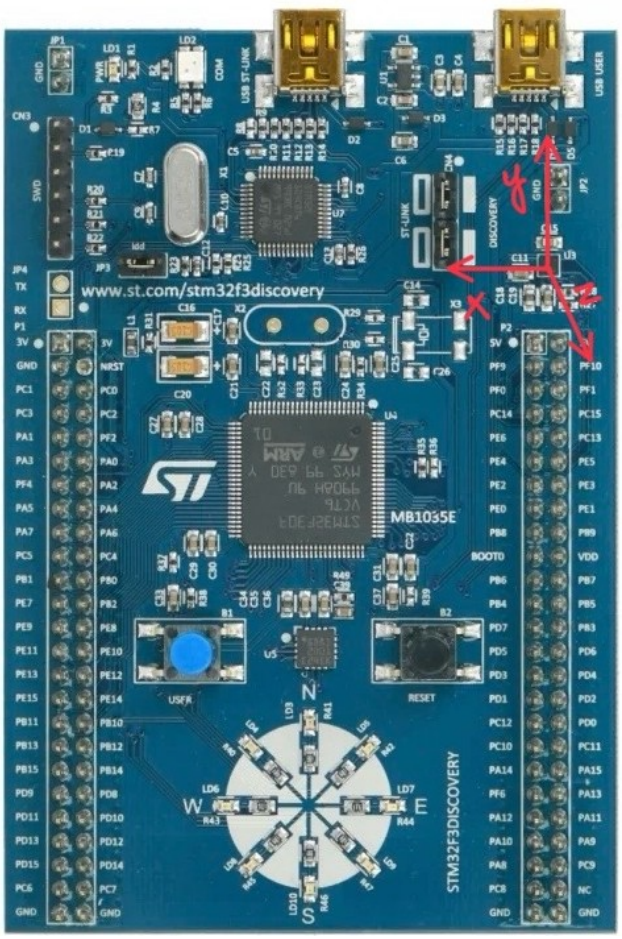


Figure 16.1: Accelerometer Axis Orientation on Blue Board)

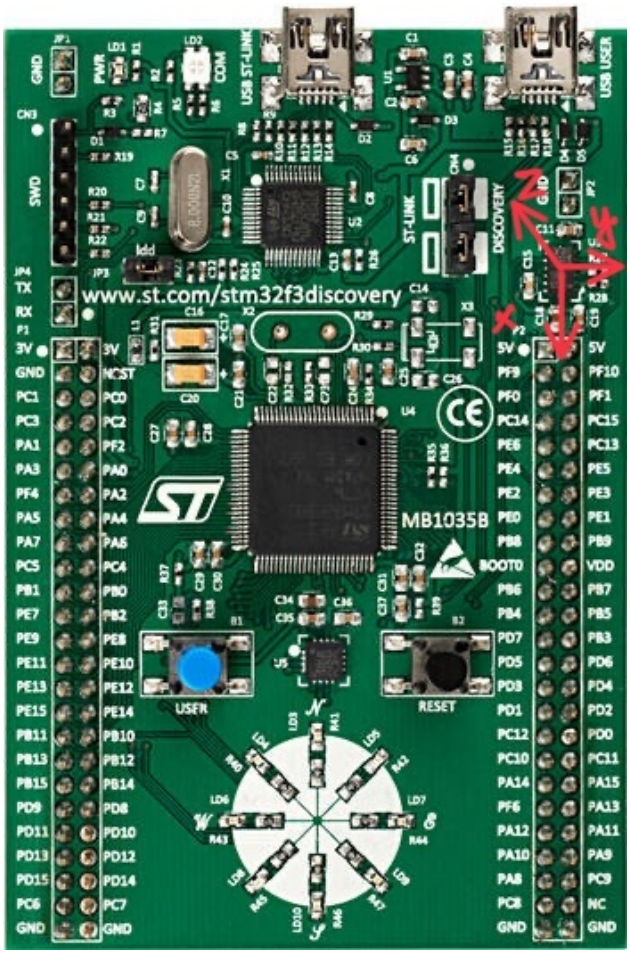


Figure 16.2: Accelerometer Axis Orientation on Green Board

Figure 16.3: STM32F3 Discovery board with axis labeling

11.3 Task 2: Position PID Controller Implementation

Objective: Design and implement a basic feedback controller (PI, PD, or PID) to regulate the robot's balance. A purely proportional controller is not permitted.

For the position PID Controller task, students must integrate the filtered angle with the controller.Both the angle estimation and the PID computation must execute within the same fixed-rate control loop at runtime. In addition, they must present and explain their code to the RA, detailing:

- How angle feedback is utilized in the control loop
- The controller's update rate (dt)
- The chosen set-point value

While achieving stable balancing is not a requirement for this lab, the motors must respond correctly to angle input. Specifically, for positive angle values, both motors should rotate clockwise; for negative angles, they should rotate counterclockwise (or vice-versa).

11.4 Closed-Loop Control and PID Controllers

Closed-loop control systems are fundamental in embedded and industrial applications to maintain desired behavior despite disturbances or changes in the environment. In this lab, students will implement a closed-loop controller for a DC motor, using the robot's tilt angle (relative to the ground)

as feedback.

At the core of many control systems is the **PID controller**, which stands for **Proportional–Integral–Derivative**. This widely used feedback control algorithm helps drive system outputs toward a desired setpoint by continuously correcting the error between the setpoint and the measured process variable (in this case, the robot's angle).

PID Control Components

- **Proportional (P)**: Responds proportionally to the current error (difference between the desired and measured value). A higher proportional gain increases response speed but may cause overshoot or instability.
- **Integral (I)**: Accumulates past errors over time. This helps eliminate steady-state error, ensuring the output eventually reaches the target even if a constant disturbance is present.
- **Derivative (D)**: Predicts future error based on its rate of change. It adds damping to the system and helps reduce overshoot and oscillations.

Students are required to implement a feedback controller using at least two of the three PID components (i.e., PI, PD, or full PID). A controller using only the proportional term is not allowed in this lab.

11.4.1 Position vs Speed PID Control

There are two main ways to apply PID control in motor systems:

- **Position PID Control**: The controller computes the control signal based on the error in position (or angle). In the context of this lab, the "position" refers to the robot's angle relative to the vertical. The control output attempts to minimize the difference between the current angle and the desired upright position. This type of control directly stabilizes the orientation of the robot.
- **Speed PID Control**: Here, the controller focuses on the rate of change of the variable — in this case, angular velocity. Speed control is typically used when a consistent rotational speed is desired (e.g., motor RPM regulation). In balance-related applications, speed control may be layered beneath position control to smooth out responses or handle dynamic behavior more effectively.

  Key Difference: Position PID uses the angle as the primary feedback variable, while speed PID uses the rate of angle change (angular velocity). For balancing tasks, position control ensures the robot maintains an upright stance, while speed control can be used for fine-tuned adjustments or nested control loops.

11.5 Angle Calculation within Control Loop and Sampling Time (`dt`)

In the previous tasks of this lab, the robot's tilt angle was computed using a complementary filter and then used as the feedback variable for the PID controller. For the controller to behave correctly, the angle estimation must be updated at a consistent and well-defined rate. This update interval is denoted by the sampling time, `dt`.

The complementary filter relies on integrating gyroscope data over time and blending it with accelerometer-based tilt estimates. As a result, the accuracy of the computed angle depends directly on the value of `dt`. Any variation in the sampling interval leads to incorrect angle estimation and degraded control performance.

In this lab, the same sampling time `dt` must be used consistently for both the angle estimation and the PID controller update. A typical and effective control-loop frequency for this system is 100–200 Hz, corresponding to `dt` values between 0.01 s and 0.005 s.

If angle computation and control updates are placed inside a simple `while(1)` loop, the effective sampling time becomes inconsistent due to UART transmissions, blocking delays, and other background operations. This results in jitter in `dt`, which negatively impacts both the complementary filter and the controller.

To avoid this issue, the angle calculation and controller update should be executed inside a **timer interrupt service routine (ISR)**. The timer ensures that these operations occur at fixed, regular intervals, independent of other software activities.

Within the timer ISR:

- IMU sensor readings are acquired (non-blocking).

- The tilt angle is updated using the complementary filter.
- The PID controller uses the updated angle to compute the control output.
- A counter is incremented to regulate how often diagnostic or UART data is transmitted.

This approach allows the control loop to run at a high, stable frequency while keeping serial communication at a lower rate for monitoring purposes. Maintaining a fixed sampling time is essential for reliable angle estimation and predictable closed-loop behavior.

Pseudocode

```
/* Timer ISR (runs at 200 Hz) */
void TIMER_ISR_callback(void)
{
    static float tilt_angle = 0.0f;
    static int counter = 0;

    // Step 1: Read IMU sensors (fast, non-blocking)
    float acc_y  = read_accelerometer_y();
    float gyro_y = read_gyroscope_y();

    // Step 2: Apply complementary filter
    tilt_angle = 0.98f * (tilt_angle + gyro_y * dt) + 0.02f * (acc_y);

    // Step 3: Store shared variable (for main loop / PID)
    shared_angle = tilt_angle;

    // Step 4: Increment counter for UART display
    counter++;
    if (counter >= 20) {        // e.g., 200Hz / 20 = 10Hz
        display_flag = 1;
        counter = 0;
    }
}

/* Main function (while(1)) */
int main(void)
{
    initialize_system();
    initialize_IMU();
    initialize_UART();
    configure_timer_interrupt(200);   // 200 Hz
    start_timer_interrupt();

    while (1) {
        if (display_flag == 1) {
            display_flag = 0;
            float current_angle = shared_angle;
            UART_transmit_angle(current_angle);
        }

```

```
41          // Other background tasks can be added here
42      }
43  }
```

Listing 16.1: Timer ISR and main loop structure for fixed-rate angle estimation

Lab Implementation Notes

In this lab, students are expected to:

- Demonstrate real-time angle and filtered angle transmission
- Design PID controller using the robot's tilt angle as feedback. Tuning the controller for the balancing is NOT required for this lab. This is a task which will be evaluated in the final project.
- Demonstrate an understanding of how each PID term influences system behavior.
- Students can explore speed-based control using rotary encoders but it is NOT required for the approval.

While the robot is not expected to achieve full balance in this lab, the motors must respond in the correct direction relative to the angle input. That is, when the robot tilts forward (positive angle), the motors should rotate clockwise; when tilting backward (negative angle), they should rotate counterclockwise.

11.6 Hardware Used

- STM32F3 Discovery Board
- Ks0193 Keyestudio Self-balancing shield
- One or both of the motors of the Keyestudio Self-balancing robot kit
- Power supply

11.6.1 Hints and Tips

- Begin with simple control logic i.e. P only before transitioning to more advanced strategies such as full PID or PI/PD. This helps with easier debugging and tuning.
- **Sampling time (`dt`)** is a critical parameter in control systems. Choose a value that reflects the dynamics of your system—too fast may introduce noise, too slow may lead to instability or sluggish response.
- Ensure the sampling time used in the controller is consistent with that used in the angle filter. It is recommended to define a global macro (e.g., `DT`) and use it consistently across the main loop, the controller logic, and the filtering algorithm.
- Implement anti-windup techniques or overflow protection for the integral term and final control output to prevent saturation and instability due to prolonged error accumulation.
- Use real-time plotting of the filtered angle and controller output to better understand the system's behavior and facilitate tuning.
- Toggle GPIO pins and observe them on an oscilloscope to assist with timing verification.
- The ODR defines how frequently new measurement data is produced by the accelerometer and gyroscope. This value is configured through the sensor's control registers (e.g., CTRL registers) and should be chosen to align with the system's sampling and control loop frequency.

11.7 Evaluation Questions

(a) What role does each PID term play in the controller's response?

(b) How does changing K_p, K_i, or K_d affect system performance?

(c) What challenges did you face in initial tuning the controller? What will be your strategy for the final fine tuning of the controller?

(d) What are the limitations of using a simple PID for real-world systems?

11.8 Assessment Rubric

OEL	Criteria	Points	CLO
OEL2	Accelerometer, gyroscope and filtered angle data is transmitted in real-time and demonstrated clearly.	/25	CLO 2
OEL2	PI, PD or PID controller is implemented in firmware with filtered angle used as a feedback variable.	/10	
OEL4	Logical values for the gain parameters and sampling time (DT) are chosen based on system behavior and are justified with reasoning.	/15	
OE5	For both tasks, code is correct, well-structured, and modular, with clear comments and appropriate use of functions.	/20	
OE6	Evaluation Questions	/20	
Git		/10	CLO 4